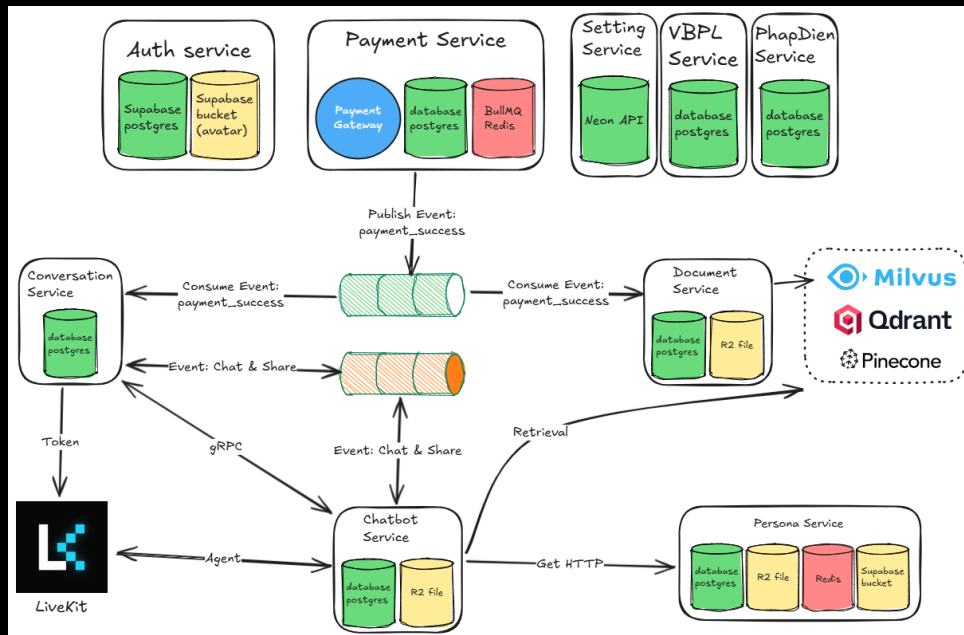


1 Triển khai hệ thống

1.1 Triển khai các dịch vụ

1.1.1 Tổng quan các dịch vụ của hệ thống



Hình 1.1: Sơ đồ tổng quan các dịch vụ của hệ thống

Tất cả các dịch vụ đều dùng supabase để xác thực thông tin JWT của người dùng. Trong JWT có các thông tin sub, exp, email, aal, app_metadata, Các dịch vụ giải mã để lấy thông tin người dùng. Kết quả thông tin payload của JWT từ Supabase:

```
JSON Claims Breakdown

{
  "iss": "https://tcfxhnmtyutbgmrdmvs.supabase.co/auth/v1",
  "sub": "b81a248d-90fd-4ae7-aa17-c7da0f210da1",
  "aud": "authenticated",
  "exp": 1781309512,
  "iat": 1781266312,
  "email": "vuvannghia.work@gmail.com",
  "phone": "",
  "app_metadata": {
    "provider": "google",
    "providers": [
      "google"
    ],
    "role": "admin"
  },
  "user_metadata": {
    "avatar_url": "https://lh3.googleusercontent.com/a/ACg8ocLyooaTGy-j35PwQQR1hcnXAmV013-CJTdTlQ8LWAKU0M26sjQ=s96-c",
    "email": "vuvannghia.work@gmail.com",
    "email_verified": true,
    "full_name": "Nghia Vu",
    "iss": "https://accounts.google.com",
    "name": "Nghia Vu",
    "phone_verified": false,
    "picture": "https://lh3.googleusercontent.com/a/ACg8ocLyooaTGy-j35PwQQR1hcnXAmV013-CJTdTlQ8LWAKU0M26sjQ=s96-c",
    "provider_id": "101416124558740354620",
    "sub": "101416124558740354620"
  },
  "role": "authenticated",
  "aal": "aal1",
  "amr": [
    {
      "method": "oauth",
      "timestamp": 1781266312
    }
  ],
  "session_id": "833402b8-d622-4f78-8af3-20cdc4dc08ea",
  "is_anonymous": false
}
```

Hình 1.2: Thông tin JWT Payload từ Supabase

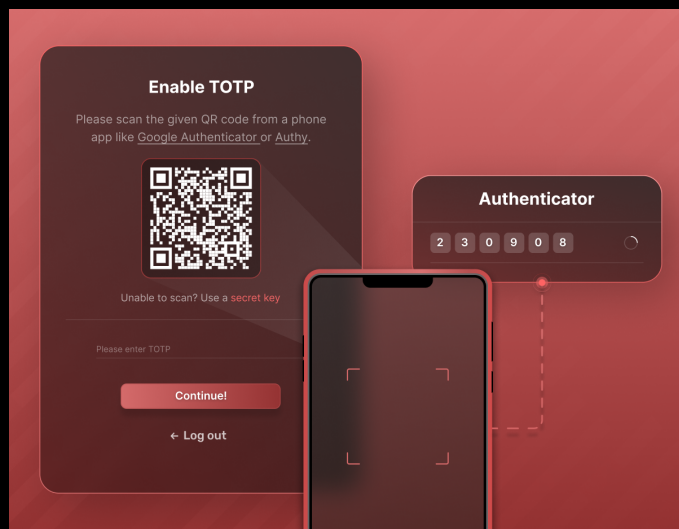
Thuật toán TOTP (Time-based One-time Password) sử dụng yếu tố thời gian Unix (thay vì dùng bộ đếm) (thời gian T_0 của Unix là ngày 1 tháng 1 năm 1970). Giá trị của bộ đếm trong thuật toán TOTP được tính bằng công thức:

$$C_T = \left\lceil \frac{T - T_0}{T_X} \right\rceil$$

Trong đó:

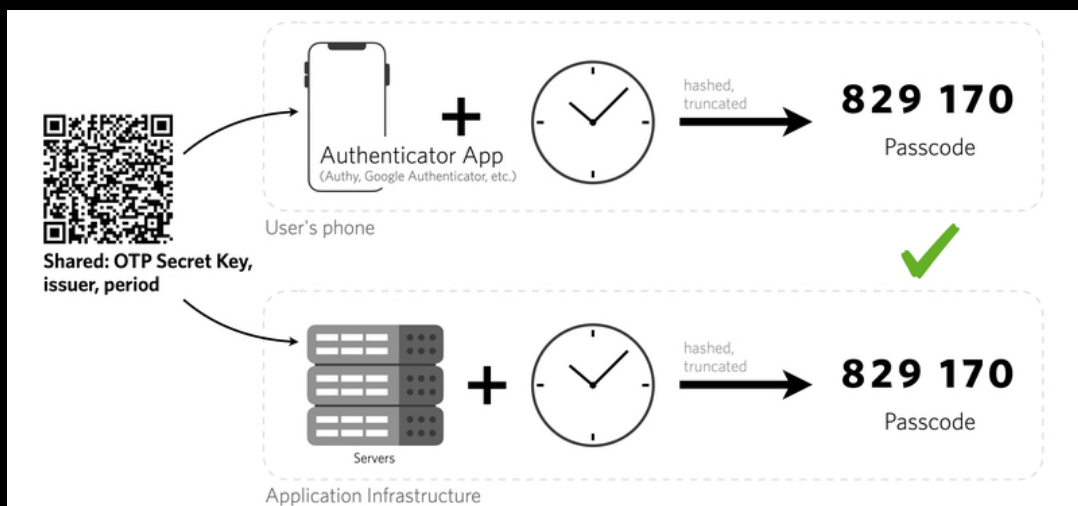
- T là thời gian hiện tại
- T_0 là thời gian đầu tiên
- T_X là chu kỳ của mã TOTP (mặc định là 30 giây)

Sau khi thực hiện xong thuật toán TOTP, lần đầu tiên máy chủ cần trao đổi khóa bí mật với người dùng. Vì hiện nay điện thoại thông minh đều được trang bị camera, máy chủ có thể chuyển đổi khóa bí mật thành mã QR và yêu cầu người dùng phần mềm Authenticator quét mã QR để lấy khóa bí mật.



Hình 1.3: Trao đổi khóa bí mật bằng cách quét mã QR

TOTP sử dụng khóa bí mật có thể hiển thị dưới dạng mã QR cho người dùng. Từ đó, máy chủ và phần mềm Authenticator đều có thể tính được giá trị của mã TOTP theo thuật toán. Cuối cùng, khi cần xác thực thì người dùng nhập giá trị TOTP hiển thị trên phần mềm Authenticator để máy chủ tiến hành so sánh.



Hình 1.4: Cách hoạt động của TOTP

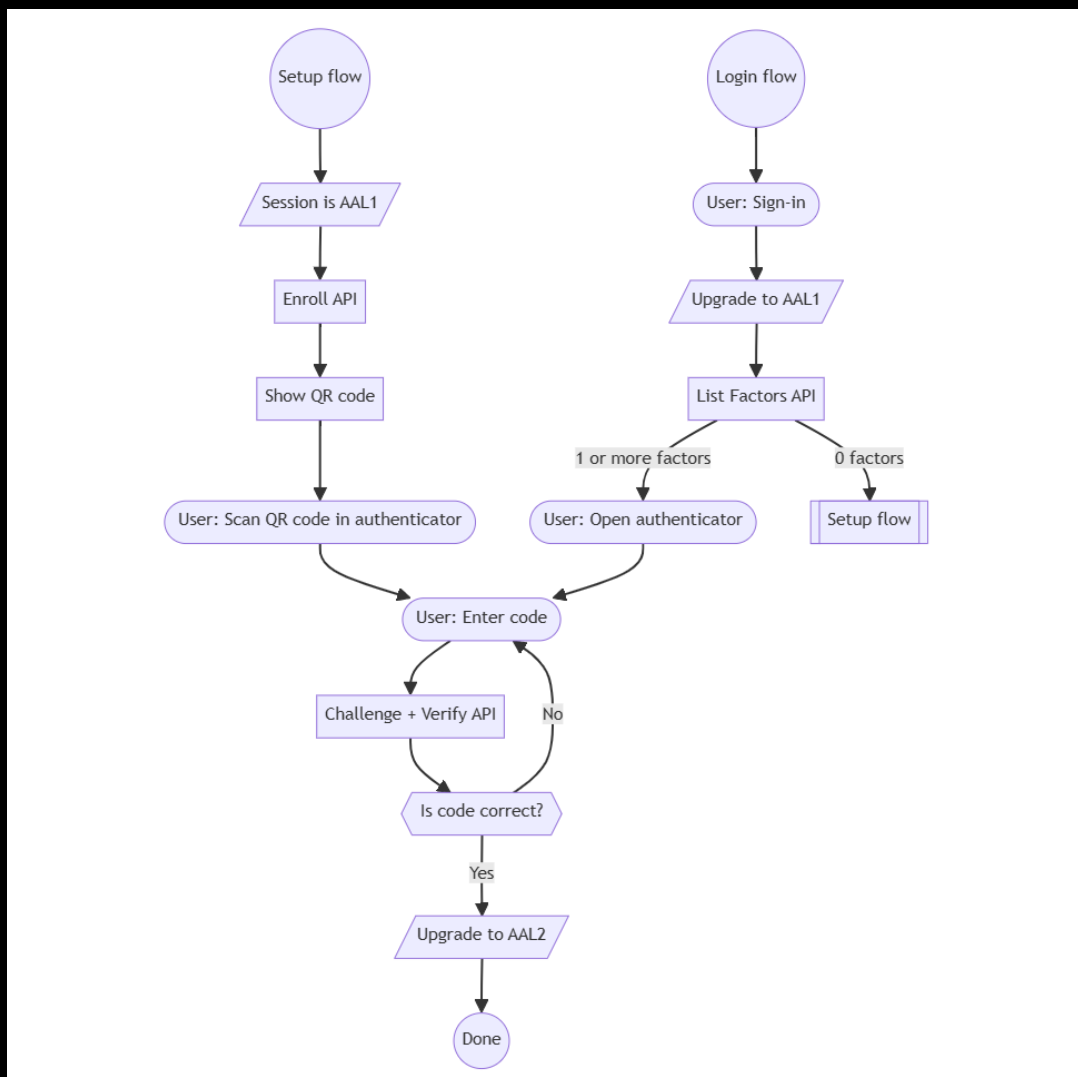
Tham khảo tài liệu tại: <https://supabase.com/docs/guides/auth/auth-mfa/totp>. Trang tài liệu này của Supabase hướng dẫn cách triển khai tính năng Xác thực Đa Yếu Tố (MFA) bằng ứng dụng Authenticator (TOTP). Tính năng này hiện được cung cấp miễn phí và bật mặc định cho mọi dự án Supabase. Nội dung chính xoay quanh hai luồng hoạt động cơ bản:

Quy trình đăng ký yếu tố bảo mật (Enrollment Flow)

1. Khởi tạo quá trình đăng ký.
2. Hệ thống sẽ trả về một mã QR và một chuỗi mã bí mật.
3. Người dùng sẽ dùng ứng dụng xác thực (như Google Authenticator) để quét mã QR này.
4. Chuẩn bị hệ thống để tiếp nhận mã xác minh, đồng thời trả về một challenge ID.
5. Đối chiếu mã 6 chữ số mà người dùng nhập vào từ ứng dụng xác thực.
6. Nếu mã hợp lệ, yếu tố bảo mật MFA này sẽ chính thức được kích hoạt cho tài khoản.

Quy trình kiểm tra khi đăng nhập (Challenge Flow)

1. Trích xuất danh sách các yếu tố MFA đã được người dùng đăng ký để lấy `factorId`.
2. Khi đã có `factorId`, ứng dụng tiếp tục gọi `challenge()` để mở phiên thử thách.
3. Dùng `verify()` cùng với mã TOTP người dùng nhập vào để hoàn tất đăng nhập và nâng cấp phiên làm việc lên `aal2`.



Hình 1.5: Sơ đồ luồng xử lý xác thực MFA Supabase

Tất cả các dịch vụ đều dùng CSDL Postgres của Neon để làm CSDL riêng theo đúng mẫu CSDL cho mỗi dịch vụ (database per service pattern). Mỗi dịch vụ sẽ sở hữu và quản lý hoàn toàn CSDL của riêng nó.

Name	Region	Created at	Storage	Compute last active at	Branches	Integrations
prod-code-persona-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.86 MB	An hour ago	1	Add ⊕
dev-code-persona-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.85 MB	16 hours ago	1	Add ⊕
dev-code-document-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.6 MB	11 days ago	1	Add ⊕
prod-code-document-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.33 MB	12 hours ago	1	Add ⊕
dev-code-chatbot-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.3 MB	2 days ago	1	Add ⊕
prod-code-chatbot-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.93 MB	12 hours ago	1	Add ⊕
prod-code-conversation-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.1 MB	A few seconds ago	1	Add ⊕
dev-code-conversation-service	AWS Asia Pacific 1 (Singapore)	A month ago	32.15 MB	2 days ago	1	Add ⊕
dev-code-payment-service	AWS Asia Pacific 1 (Singapore)	A month ago	31.99 MB	3 minutes ago	1	Add ⊕
prod-code-payment-service	AWS Asia Pacific 1 (Singapore)	A month ago	33.04 MB	A few seconds ago	1	Add ⊕

Hình 1.6: Bảng điều khiển CSDL trong Neon

Các dịch vụ ngoại trừ auth service và setting service được cấu hình trong Kong API Gateway đơn giản chỉ cần tạo Service và Route để định tuyến yêu cầu. Dịch vụ auth service và setting service sẽ được mô tả chi tiết về cách cấu hình Kong API Gateway trong nội dung trình bày của chính dịch vụ đó. Vì trang thống kê thông tin đường ống dữ liệu (Data Pipeline) dành cho quản trị viên (ADMIN) có tần suất truy vấn thấp nên sẽ được triển khai trên Vercel.

Project Name	Repository	Commit Message	Time Ago
code-fe-ui	20206205.tech	chore(deps): bump @radix-ui/react-progress from 1.1.8 to 1.1.9 (...)	2d ago on main
data-pipeline-phapdien-service	data-pipeline-phapdien-service.2020620...	Bump cryptography from 48.0.0 to 48.0.1 (#13)	2d ago on main
data-pipeline-vbplnew-service	data-pipeline-vbplnew-service.2020620...	Bump cryptography from 48.0.0 to 48.0.1 (#12)	2d ago on main
api-gateway-http-log	api-gateway-http-log.20206205.tech	Update deploy-nestjs-vercel-cloudflare.yml	Jun 8 on main

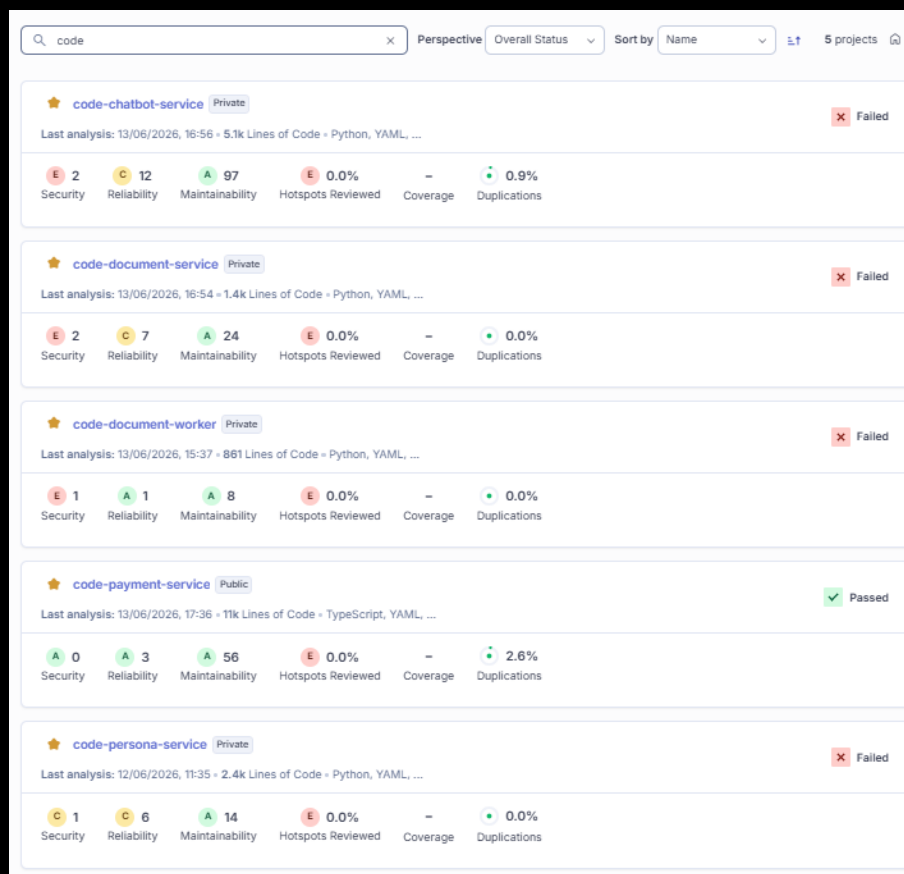
Hình 1.7: Bảng điều khiển các dự án trên Vercel

Các dịch vụ NestJS sử dụng Codecov để theo dõi độ bao phủ kiểm thử (test coverage).

Name	Last updated	Tracked lines
20206205Tech / 20206205tech-nestjs-auth	2 minutes ago	117
20206205Tech / 20206205tech-nestjs-common	3 minutes ago	150
20206205Tech / code-payment-service	about 23 hours ago	2220
20206205Tech / code-conversation-service	1 day ago	1998
20206205Tech / api-gateway-http-log	8 days ago	109

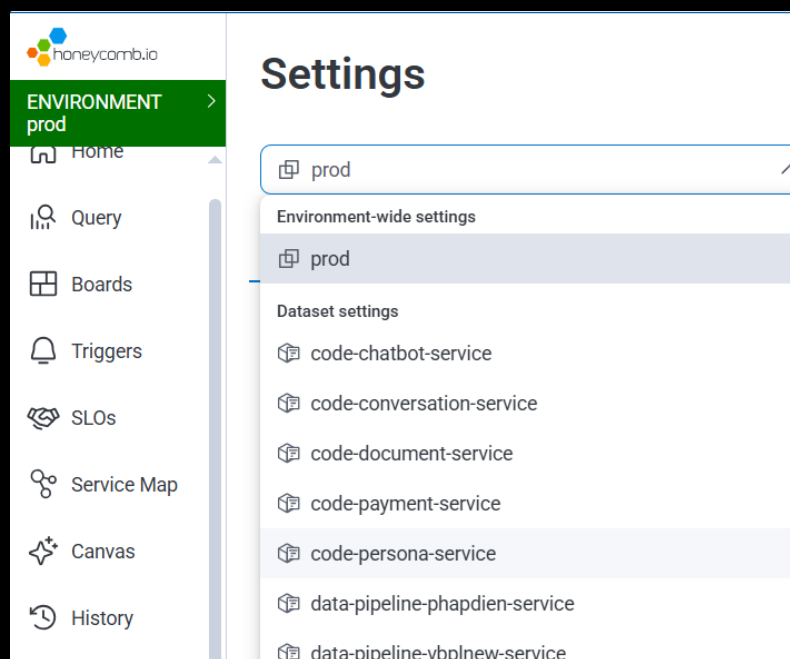
Hình 1.8: Sử dụng CodeCov theo dõi độ bao phủ kiểm thử

Tích hợp SonarQube để phân tích chất lượng mã nguồn và các lỗ hổng bảo mật.



Hình 1.9: Giao diện phân tích mã nguồn trên SonarQube

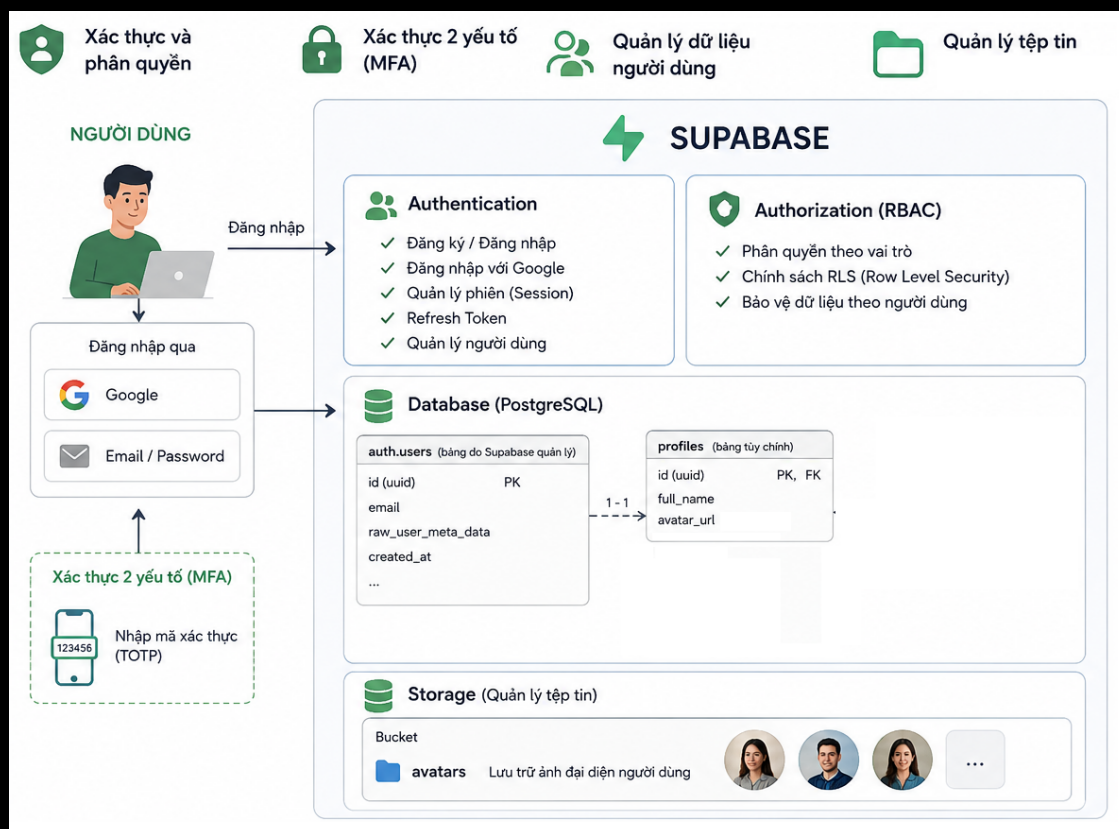
Hệ thống tích hợp Honeycomb để theo dõi vết (distributed tracing) giữa các dịch vụ.



Hình 1.10: Giao diện theo dõi vết distributed tracing trên Honeycomb

1.1.2 Dịch vụ xác thực (auth service - dùng Supabase)

Hệ thống sử dụng Supabase để xử lý các nhóm chức năng chính bao gồm: xác thực và phân quyền, xác thực 2 yếu tố (MFA), quản lý dữ liệu người dùng và quản lý tệp tin (lưu trữ các file ảnh đại diện do người dùng tải lên tại bucket *avatars*). Do mỗi lần người dùng đăng nhập lại, Supabase/Google sẽ ghi đè lại thông tin gốc từ tài khoản liên kết, bảng *profiles* được thiết kế riêng để lưu trữ và duy trì thông tin của người dùng. Dưới đây là mô tả chi tiết các chức năng:



Hình 1.11: Tổng quan các chức năng của Supabase

Xác thực và quản lý tài khoản Hệ thống sử dụng các API Auth của Supabase để xử lý toàn bộ luồng đăng nhập, đăng ký và bảo mật tài khoản:

- **Đăng nhập bằng Google:** Tạo URL và chuyển hướng người dùng đến trang ủy quyền của Google. Endpoint sử dụng: `/authorize?provider=google`
- **Đăng nhập bằng Email và Mật khẩu:** Cho phép người dùng đăng nhập bằng tài khoản email. Trả về thông tin Access Token và Refresh Token nếu thành công. Endpoint sử dụng: `/token?grant_type=password`
- **Đăng ký tài khoản mới:** Tạo tài khoản mới bằng Email và Mật khẩu, hỗ trợ gửi email xác nhận tài khoản thông qua đường dẫn chuyển hướng sau khi kích hoạt thành công (`redirect_to`). Endpoint sử dụng: `/signup`
- **Khôi phục mật khẩu:** Gửi yêu cầu khôi phục mật khẩu tới email của người dùng. Hệ thống sẽ gửi một email chứa liên kết để xác thực và chuyển hướng người dùng về trang đổi mật khẩu. Endpoint sử dụng: `/recover`
- **Cập nhật mật khẩu mới:** Cho phép người dùng đổi mật khẩu mới sau khi đã đăng nhập (hoặc sau khi click vào liên kết khôi phục mật khẩu). Endpoint sử dụng: `/user`
- **Làm mới Access Token:** Sử dụng `refresh_token` để lấy một `access_token` mới khi token cũ hết hạn. Endpoint sử dụng: `/token?grant_type=refresh_token`
- **Đăng xuất:** Hủy phiên làm việc hiện tại của người dùng trên hệ thống Supabase. Endpoint sử dụng: `/logout`

Xác thực 2 yếu tố (MFA) Hệ thống hỗ trợ phương thức xác thực hai yếu tố TOTP (Time-based One-Time Password - như Google Authenticator hay Authy) bằng các API nâng cao của Supabase:

- **Đăng ký thiết bị xác thực MFA mới:** Khởi tạo quá trình liên kết thiết bị xác thực mới. API trả về mã QR Code (`qr_code`), mã bí mật (`secret`) và chuỗi URI để người dùng quét trên ứng dụng Authenticator. Endpoint sử dụng: `/factors`
- **Tạo thử thách xác thực:** Tạo một thử thách xác thực (challenge) dựa trên ID thiết bị MFA (`factorId`) để chuẩn bị so khớp với mã TOTP người dùng nhập. Endpoint sử dụng: `/factors/{factorId}/challenge`
- **Xác thực mã TOTP:** Kiểm tra mã TOTP gồm 6 chữ số người dùng nhập có khớp với thử thách hiện tại không. Nếu đúng, Supabase sẽ nâng cấp phiên làm việc của người dùng (tăng cấp bảo mật cho Access Token lên mức độ xác thực `aal2`). Endpoint sử dụng: `/factors/{factorId}/verify`
- **Lấy danh sách các yếu tố MFA:** Lấy thông tin tài khoản người dùng để lọc ra danh sách các thiết bị/yếu tố xác thực đã liên kết, phân loại thành: tất cả thiết bị (`all`) và các thiết bị đã kích hoạt thành công (`active`). Endpoint sử dụng: `/user`
- **Hủy liên kết/Xóa thiết bị MFA:** Xóa một thiết bị xác thực MFA khỏi tài khoản người dùng. Endpoint sử dụng: `/factors/{factorId}`
- **Cập nhật tên hiển thị của thiết bị MFA:** Đổi tên hiển thị (`friendly_name`) của thiết bị xác thực (ví dụ: "Điện thoại cá nhân"). Endpoint sử dụng: `/factors/{factorId}`

Quản lý Hồ sơ người dùng (Supabase Database - PostgREST API) Hệ thống sử dụng cơ chế RESTful API tự động sinh từ PostgreSQL của Supabase (PostgREST) để thực hiện các thao tác CRUD trên bảng CSDL `profiles`:

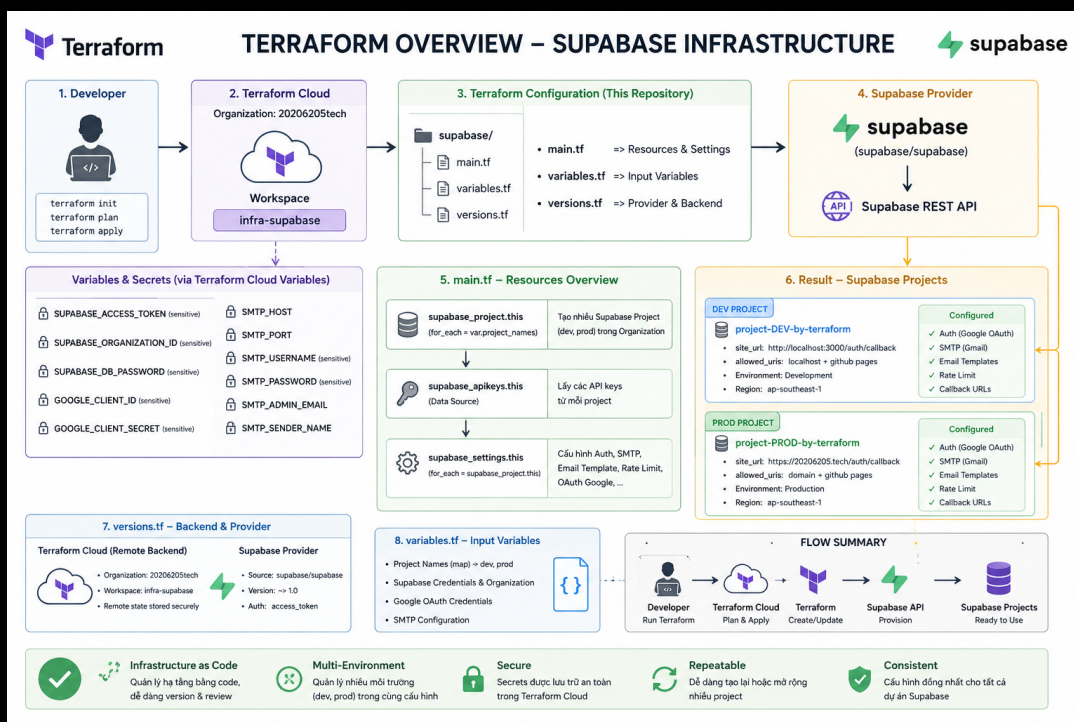
- **Lấy thông tin hồ sơ:** Truy vấn thông tin chi tiết hồ sơ người dùng (như tên đầy đủ, ảnh đại diện) từ bảng `profiles` dựa vào `userId`. Endpoint sử dụng: `/rest/v1/profiles`
- **Cập nhật thông tin hồ sơ:** Cập nhật các trường thông tin hồ sơ như tên hiển thị hoặc đường dẫn ảnh đại diện. Endpoint sử dụng: `/rest/v1/profiles`

Quản lý Ảnh đại diện (Supabase Storage) Hệ thống sử dụng dịch vụ lưu trữ tệp (Storage) của Supabase để quản lý hình ảnh của người dùng:

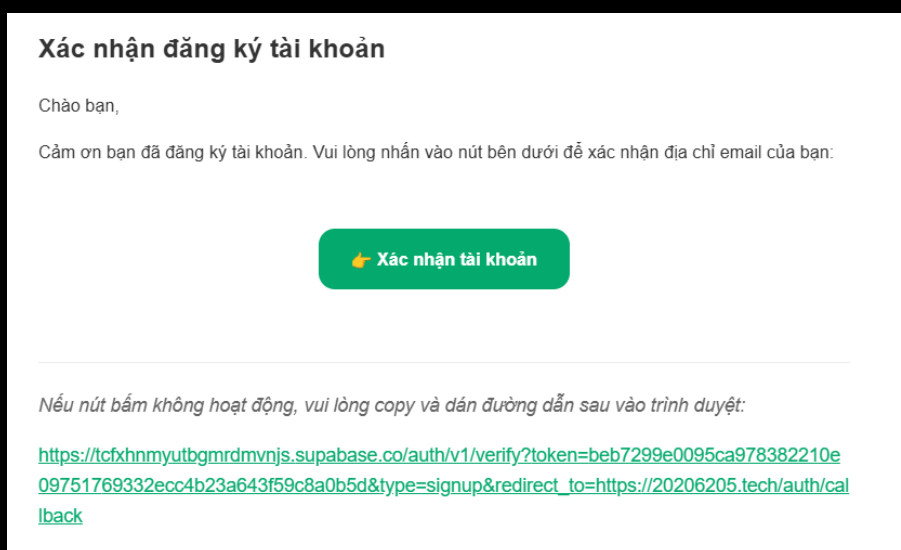
- **Tải lên ảnh đại diện:** Tải tệp tin ảnh của người dùng lên thư mục `avatars` trên Supabase Storage. Tên tệp được sinh ngẫu nhiên kết hợp với `userId` để tránh trùng lặp. Endpoint tải lên: `/storage/v1/object/avatars/{fileName}` Đường dẫn công khai nhận về: `/storage/v1/object/public/avatars/{fileName}`

Chi tiết về cấu hình Terraform dùng để quản lý hạ tầng Supabase:

- Sử dụng Terraform Cloud với workspace `infra-supabase` để lưu trữ và quản lý trạng thái hạ tầng một cách tập trung.
- Tự động tạo các dự án cho dev và prod dựa trên `project_names`.
- Cấu hình mã vùng (region) là `ap-southeast-1` tại Singapore để tối ưu tốc độ kết nối cho người dùng tại Việt Nam.
- Bỏ qua sự thay đổi của mật khẩu nhằm tránh việc Terraform yêu cầu cập nhật hoặc thay đổi mật khẩu CSDL sau mỗi lần chạy cấu hình.
- Tích hợp đăng nhập qua Google OAuth (Client ID và Client Secret).
- Thiết lập giới hạn gửi email (rate limit).
- Cấu hình Hệ thống Email (SMTP) đang được cấu hình mặc định là Gmail.
- Ghi đè các mẫu email HTML cho "Xác nhận tài khoản" và "Đặt lại mật khẩu" để đồng nhất email tương tự như trong dịch vụ thanh toán.



Hình 1.12: Tổng quan cấu hình Terraform quản lý hạ tầng Supabase



Hình 1.13: Mẫu email HTML được cấu hình

Chi tiết về cấu hình migration CSDL Supabase:

Sử dụng Supabase CLI tạo dự án quản lý migration CSDL bằng lệnh `npm run supabase init`. Trong thư mục migrations, các file SQL được thiết kế kết hợp với chính sách Bảo mật cấp hàng RLS (Row-Level Security):

- Tạo bảng `admin_whitelist` lưu thông tin email của quản trị viên (ADMIN).
- Tạo bucket `avatars` lưu thông tin file ảnh đại diện người dùng.
- Tạo bảng `profiles` lưu thông tin của người dùng.
- Tạo bucket `persona_avatars` lưu thông tin file ảnh đại diện nhân vật.
- Tạo bucket `persona_audios` lưu thông tin file âm thanh lời chào nhân vật.

Kết quả bucket cấu hình trên Supabase:

NAME	POLICIES	FILE SIZE LIMIT	ALLOWED MIME TYPES
persona_audios PUBLIC	2	Unset (50 MB)	Any
persona_avatars PUBLIC	0	Unset (50 MB)	Any
avatars PUBLIC	4	Unset (50 MB)	Any

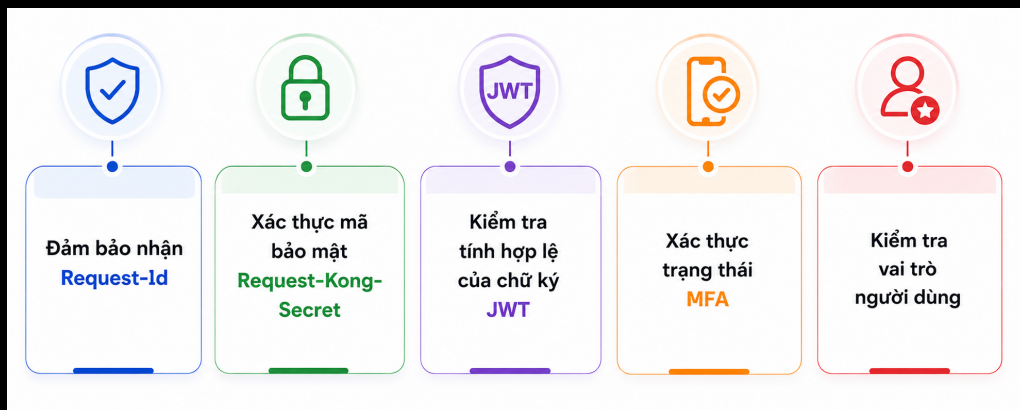
Hình 1.14: Cấu hình các Storage Bucket trên Supabase

Kết quả CSDL trên Supabase:

id	full_name	avatar_url
6b85706c-aaf9-489d-a20f-f40d8...	Nghia Vu	https://lh3.googleusercontent.com/a/AC...
c36ce92d-704d-452a-883e-67a9...	Nghia Vu	https://prod-persona.20206205.tech/ima...
29cb86d1-9345-48bb-bc8d-6881...	VVN_Music Game	https://lh3.googleusercontent.com/a/AC...

Hình 1.15: Cấu hình bảng CSDL trên Supabase

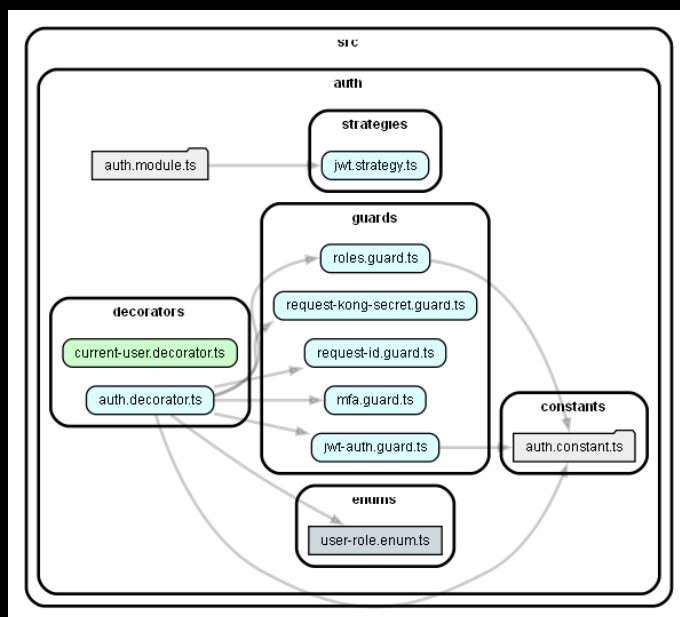
Để tối ưu hóa quá trình phát triển, các gói thư viện dùng chung (Shared Packages) đã được xây dựng để các dịch vụ khác dễ dàng tái sử dụng và quản lý. Hiện tại trong dự án, có 2 gói thư viện dùng chung được xây dựng và đăng ký trên npm/PyPI là NestJS (20206205tech-nestjs-auth) và FastAPI trong Python (20206205tech-python-auth). Các thư viện này thực hiện các bước kiểm duyệt bảo mật tự động bao gồm:



Hình 1.16: Quy trình kiểm tra bảo mật trong gói thư viện dùng chung

- Đảm bảo nhận Request-Id được chuyển tiếp từ Kong API Gateway.
- Xác thực mã bảo mật Request-Kong-Secret để đảm bảo yêu cầu thực sự đi qua API Gateway.
- Kiểm tra tính hợp lệ của chữ ký JWT của người dùng thông qua Supabase.
- Xác thực trạng thái MFA (yêu cầu bắt buộc để nâng cao bảo mật dữ liệu).
- Kiểm tra vai trò người dùng (ví dụ: yêu cầu vai trò quản trị viên ADMIN đối với các API quản lý).

Kết quả sơ đồ tệp và thư mục trong gói chung NestJS Auth được tạo tự động từ depcruise:

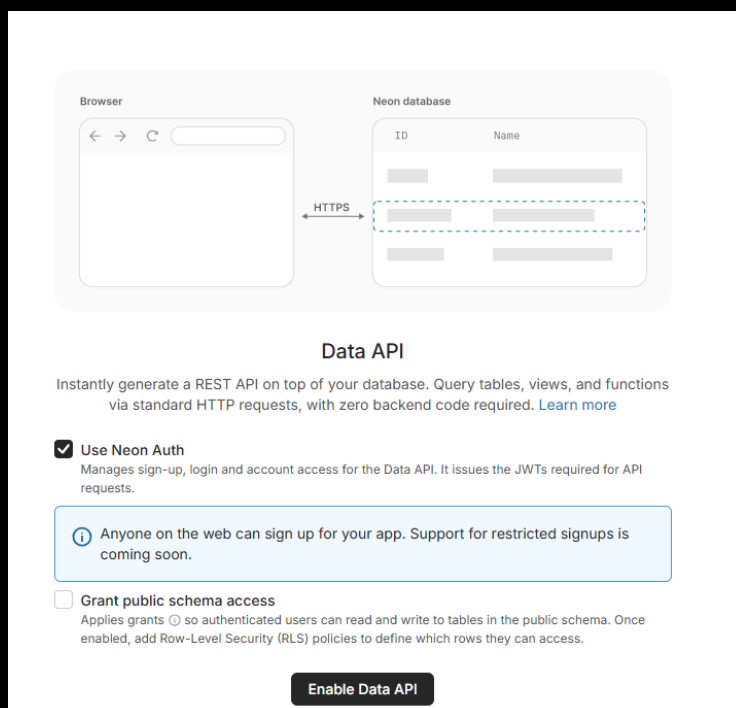


Hình 1.17: Sơ đồ tệp và thư mục trong gói chung NestJS

Cấu hình trong Kong API Gateway Tạo Service trong Kong trở về URL của dự án Supabase ([https://\[PROJECT_REF\].supabase.co](https://[PROJECT_REF].supabase.co)). Thiết lập Route: Tạo một Route liên kết với Service vừa tạo. Tham số `strip_path=true` đảm bảo Kong sẽ xóa tiền tố khỏi URI trước khi chuyển tiếp yêu cầu đến Supabase. Dùng plugin `request-transformer` để tự động thêm `apikey` vào header trước khi gửi đến Supabase.

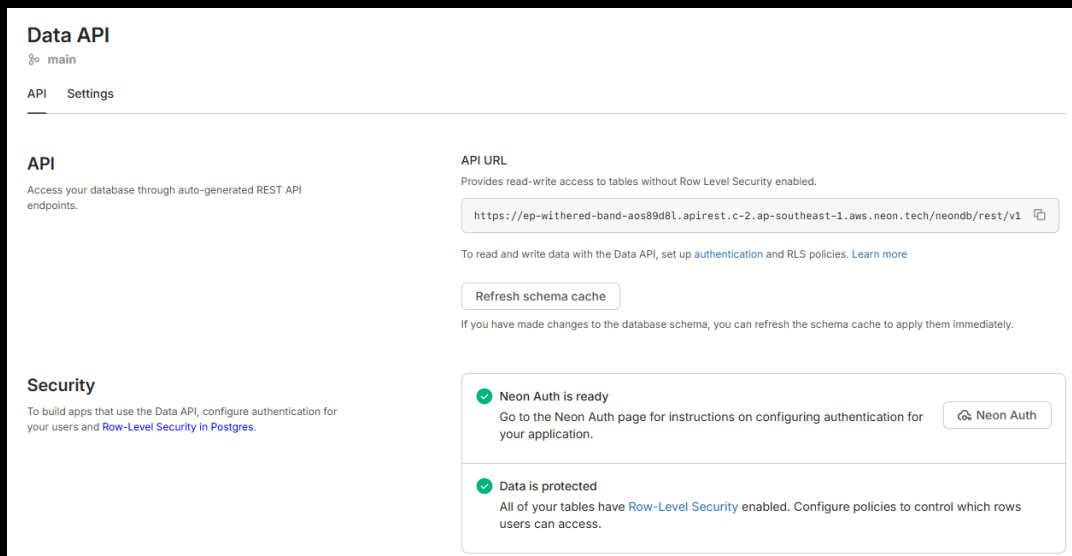
1.1.3 Dịch vụ cài đặt (setting service - dùng Neon Data API)

Sử dụng Neon Data API (REST API kết nối CSDL Postgres) để người dùng có thể thực hiện các yêu cầu `GET` và `POST` nhằm lưu trữ và truy xuất các cấu hình cài đặt (dưới dạng key-value). Kích hoạt thủ công Data API trên bảng điều khiển của Neon:



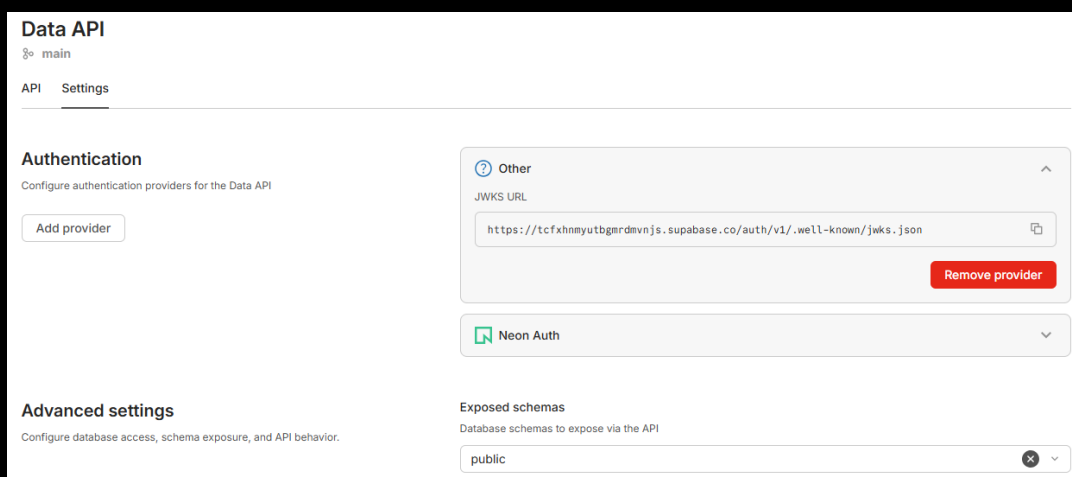
Hình 1.18: Kích hoạt Data API trên Neon

Lấy thông tin URL của REST API trên giao diện web của Neon:



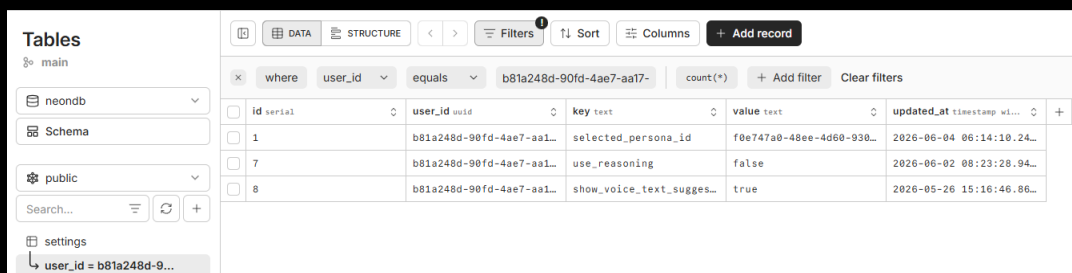
Hình 1.19: Thông tin URL REST API trên Neon

Neon hỗ trợ tích hợp cấu hình thông tin xác thực từ Supabase:



Hình 1.20: Tích hợp cấu hình xác thực Supabase trong Neon

Sử dụng SQL để tạo bảng `settings` gồm các cột: `id`, `user_id`, `key`, `value`, `updated_at`. Kích hoạt bảo mật cấp hàng RLS (Row-Level Security) để đảm bảo chỉ chủ sở hữu mới có quyền thay đổi dữ liệu cài đặt của họ:



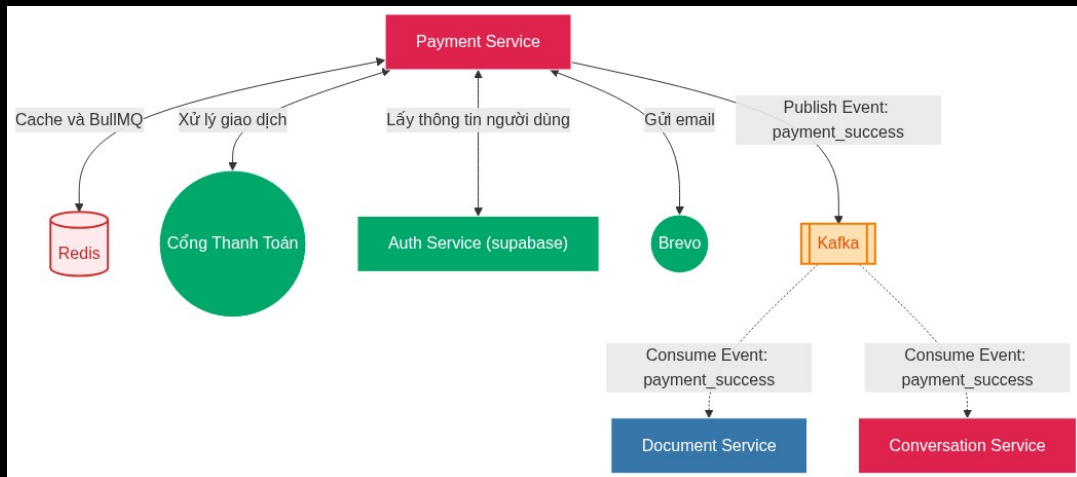
Hình 1.21: Bật RLS cho bảng settings trên Neon

Cấu hình trong Kong API Gateway Tạo Service trong Kong trỏ trực tiếp đến endpoint REST API của bảng `settings` trên Neon. Thay vì tạo một Route chung cho tất cả các yêu cầu, hệ thống được cấu hình để phân loại riêng biệt luồng đọc (GET) và luồng ghi (POST). Tham số

`strip_path=true` giúp Kong loại bỏ tiền tố URI trước khi chuyển tiếp yêu cầu đến Neon. Cấu hình Plugin xử lý Upsert tự động: Trong REST API dựa trên Postgres, để thực hiện hành động Upsert (chèn hoặc cập nhật), client cần gửi kèm các tham số cụ thể trên URL (`on_conflict=user_id,key`) và Header (`Prefer: resolution=merge-duplicates`). Sử dụng plugin `request-transformer` để Kong API Gateway tự động đính kèm các tham số và header phức tạp này trước khi gửi đến Neon.

1.1.4 Dịch vụ thanh toán (payment service)

Mục đích: Dịch vụ thanh toán (payment service) chịu trách nhiệm quản lý thông tin các gói dịch vụ (**plans**), quản lý thuê bao người dùng (**subscriptions**) của người dùng và tích hợp các cổng thanh toán. Sơ đồ tổng quan về Dịch vụ thanh toán (payment service):



Hình 1.22: Sơ đồ tổng quan dịch vụ thanh toán

Quản lý Gói dịch vụ (Plans) Cho phép hệ thống định nghĩa và cung cấp các gói dịch vụ khác nhau tới người dùng.

- **Tạo mới gói dịch vụ:** Cho phép Quản trị viên (ADMIN) thiết lập cấu hình gói dịch vụ mới bao gồm các thông số: tên gói (ví dụ: VIP 1 tháng), thời hạn sử dụng, giá tiền (VND), trạng thái hoạt động và các tính năng đi kèm (ví dụ: Quyền sử dụng mô hình suy luận AI, Voice, Xử lý tài liệu nâng cao).
- **Truy xuất danh sách gói:** Lấy danh sách toàn bộ các gói dịch vụ hiện có trên hệ thống. Chức năng này được phân trang thông qua hai tham số `skip` và `limit`.
- **Cơ chế Cache với Redis:** Lưu thông tin cache danh sách các plan vào Redis. Khi người dùng yêu cầu xem danh sách gói đăng ký, hệ thống sẽ ưu tiên đọc từ Redis. Nếu cache trống (cache miss), hệ thống sẽ truy xuất CSDL, trả về kết quả cho client và lưu lại kết quả đó vào Redis cho các yêu cầu sau.
- **Lưu trữ gói thuê bao VIP:** Cho phép quản trị viên tạm dừng các gói dịch vụ cũ.

Plans			^
POST	/code-payment-service/plans		🔒
GET	/code-payment-service/plans		🔒
DELETE	/code-payment-service/plans/{plan_id}		🔒
Subscriptions			^
POST	/code-payment-service/subscriptions/purchase		🔒
GET	/code-payment-service/subscriptions		🔒
GET	/code-payment-service/subscriptions/history		🔒
POST	/code-payment-service/subscriptions/manual-activate/{transaction_id}		🔒
Payment Controllers			^

Hình 1.23: Tài liệu Swagger API Dịch vụ thanh toán

Quản lý Thuê bao (Subscriptions) và Giao dịch Quản lý thông tin đăng ký dịch vụ của người dùng và các giao dịch phát sinh:

- **Mua gói:** Người dùng khởi tạo yêu cầu thanh toán cho một gói dịch vụ cụ thể. Hệ thống tiếp nhận mã gói (`plan_id`) và đường dẫn chuyển hướng (`redirect_url`) để tạo link thanh toán và đưa người dùng tới cổng thanh toán. Tích hợp chức năng tự động hủy giao dịch nếu hết thời gian chờ thanh toán (`timeout`) cấu hình trong Redis. Gửi sự kiện thanh toán thành công lên Kafka. Gửi email thông báo xác nhận thanh toán thành công cho người dùng.
- **Tra cứu thông tin Thuê bao:** Cho phép người dùng xem thông tin chi tiết à thời hạn của gói dịch vụ hiện tại mà họ đang sở hữu.
- **Tra cứu lịch sử giao dịch:** Cung cấp danh sách lịch sử các giao dịch thanh toán trước đó của người dùng, hỗ trợ phân trang để dễ dàng theo dõi.
- **Kích hoạt giao dịch thủ công:** Tính năng dành cho ADMIN để can thiệp kích hoạt thủ công một giao dịch (`transaction_id`) thành công, dùng trong các trường hợp đối soát thủ công hoặc xử lý sự cố.

Tích hợp Cổng thanh toán

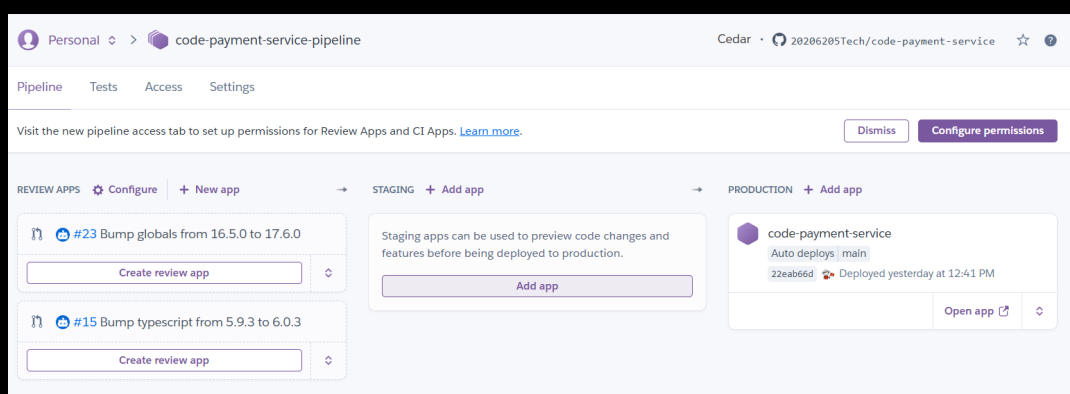
- **Xử lý Callback (IPN):** Cung cấp các endpoint (nhận cả phương thức `GET` và `POST`) để cổng thanh toán có thể âm thầm gọi về hệ thống (giao tiếp server-to-server) nhằm cập nhật trạng thái giao dịch một cách chính xác và bảo mật nhất.
- **Xử lý Return URL:** Cung cấp các endpoint (nhận cả `GET` và `POST`) để tiếp nhận và điều hướng giao diện người dùng ngay sau khi họ hoàn tất thao tác thanh toán trên trang của đối tác.

Trong quá trình phát triển dự án, do các dịch vụ thanh toán sẵn có chưa hỗ trợ tốt các cổng thanh toán phổ biến tại Việt Nam (như VNPAY, ZaloPay, MoMo, SePay, ...), dịch vụ thanh toán này đã được tự thiết kế, phát triển và triển khai độc lập trên nền tảng Heroku.

Quy trình đóng gói và vận hành:

- Dịch vụ được đóng gói tự động bằng Docker.
- Tự động chạy và triển khai lên Heroku khi vượt qua các bước kiểm thử (tests) trong CI/CD của GitHub Actions.
- Kết nối trực tiếp với CSDL Postgres (Neon), Kafka và Redis để vận hành hàng đợi BullMQ.

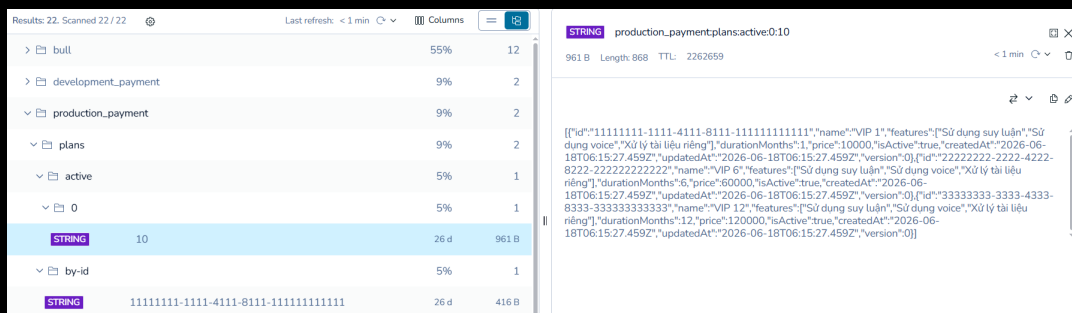
Triển khai Dịch vụ thanh toán (payment service) độc lập trên Heroku:



Hình 1.24: Trạng thái deploy dịch vụ thanh toán trên Heroku

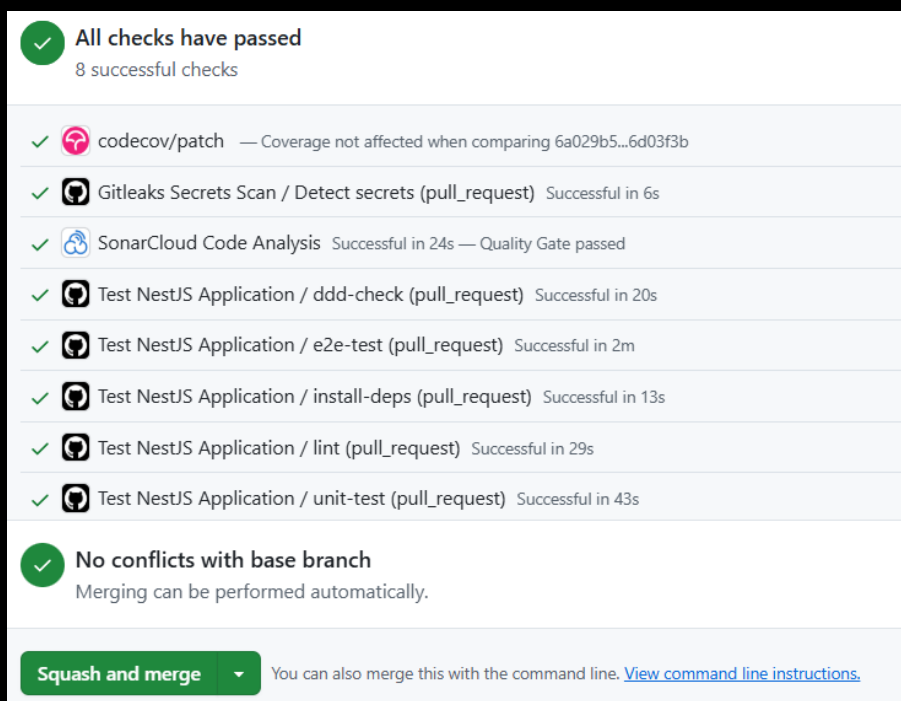
Các bước CI/CD tự động chạy kiểm thử trên GitHub Actions: Hệ thống sử dụng **Testcontainers** (cho Postgres, Redis và Kafka) để tự động khởi chạy các container Docker độc lập ngay trong quá trình chạy integration tests, đảm bảo tính đúng đắn của mã nguồn trước khi deploy. Hàng đợi tác vụ bất đồng bộ **BullMQ** (vận hành trên Redis) được tích hợp để quản lý và tự động xử lý hủy các giao dịch thanh toán quá hạn nếu hết thời gian chờ.

Dữ liệu gói thuê bao VIP được cache trong Redis:

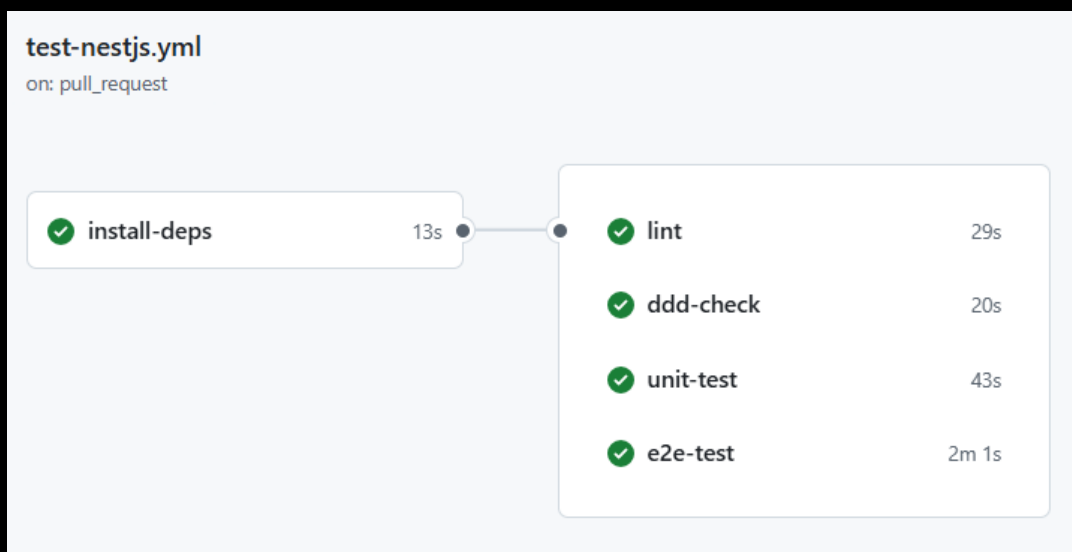


Hình 1.25: Dữ liệu gói thuê bao VIP được cache trong Redis

Thông tin kiểm tra tự động của Pull Request của payment service trên GitHub:

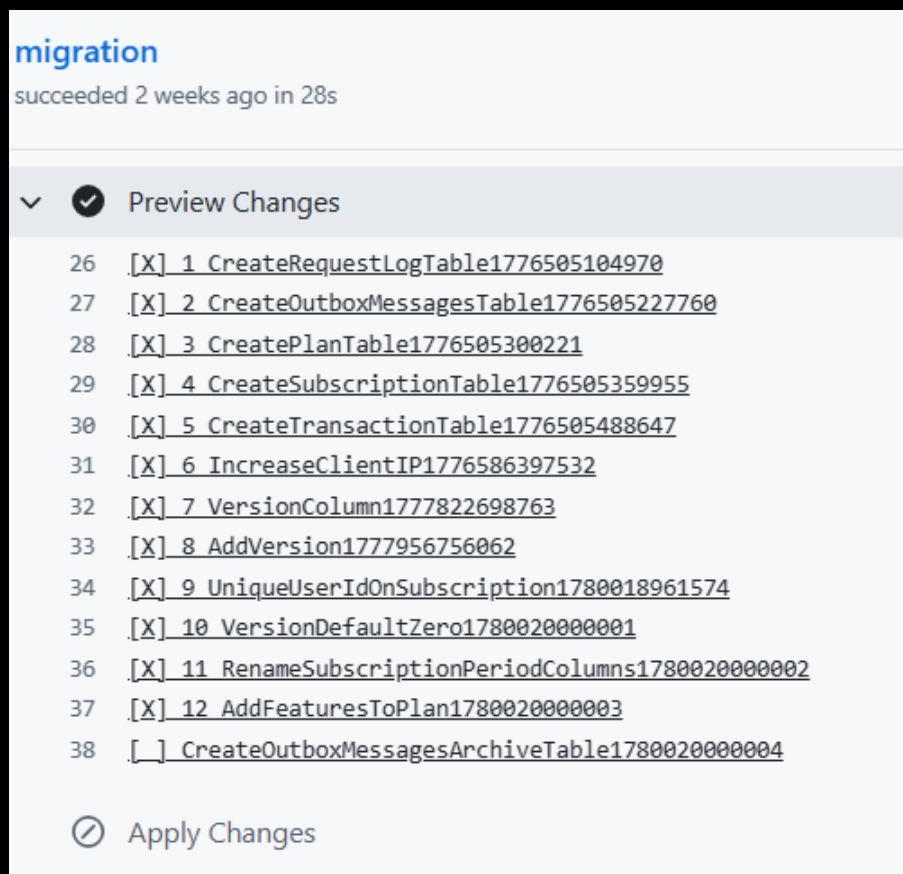


Hình 1.26: Kiểm tra Pull Request của payment service trên GitHub



Hình 1.27: Quy trình chạy Test trong GitHub Actions

Quản lý migrations CSDL tự động trong GitHub Actions:



Hình 1.28: Quản lý database migrations trên GitHub Actions

Các bảng trong CSDL của Dịch vụ thanh toán (payment service):

Tables

main

neondb

Schema

public

Search...

migrations

outbox_messages

outbox_messages_archive

plan

request_logs

subscription

transaction

DATA

STRUCTURE

<

>

Filters

Sort

Columns

+ Add record

☐	id serial	timestamp bigint	name varchar	
☐	1	1776505104970	CreateRequestLogTable1...	
☐	2	1776505227760	CreateOutboxMessagesTa...	
☐	3	1776505300221	CreatePlanTable1776505...	
☐	4	1776505359955	CreateSubscriptionTabl...	
☐	5	1776505488647	CreateTransactionTable...	
☐	6	1776586397532	IncreaseClientIP177658...	
☐	7	1777822698763	VersionColumn177782269...	
☐	8	1777956756062	AddVersion1777956756062	
☐	9	1780018961574	UniqueUserIdOnSubscrip...	
☐	10	1780020000001	VersionDefaultZero1780...	
☐	11	1780020000002	RenameSubscriptionPeri...	
☐	12	1780020000003	AddFeaturesToPlan17800...	
☐	13	1780020000004	CreateOutboxMessagesAr...	

Hình 1.29: Cấu hình các bảng CSDL của dịch vụ thanh toán

Sự kiện Thanh toán thành công (Subscription Purchased Event) Khi giao dịch thanh toán gói dịch vụ thành công, Dịch vụ thanh toán sẽ tạo và phát hành (publish) sự kiện lên Kafka để đồng bộ quyền hạn người dùng.

Chi tiết sự kiện

- **Tên Sự kiện:** SubscriptionPurchasedEvent

- **Topic:** `dev-payment-events` (phát triển) hoặc `prod-payment-events` (vận hành)
- **Mã hóa (Compression):** GZIP
- **Phân vùng khóa (Message Key):** `userId` (đảm bảo tính tuần tự của các sự kiện liên quan đến cùng một người dùng trên cùng một partition).

Cấu trúc dữ liệu sự kiện Dữ liệu sự kiện được gửi dưới dạng chuỗi JSON có cấu trúc như sau:

Thuộc tính	Kiểu dữ liệu	Mô tả
<code>subscriptionId</code>	String (UUID)	Mã định danh duy nhất của thuê bao mới tạo
<code>userId</code>	String (UUID)	Mã định danh người dùng mua gói
<code>planId</code>	String (UUID)	Mã định danh gói dịch vụ đã mua
<code>periodStart</code>	String (ISO 8601)	Thời điểm bắt đầu hiệu lực gói dịch vụ
<code>periodEnd</code>	String (ISO 8601)	Thời điểm kết thúc hiệu lực gói dịch vụ
<code>version</code>	Number	Số phiên bản của sự kiện (dùng để quản lý xung đột)
<code>occurredAt</code>	String (ISO 8601)	Thời điểm sự kiện phát sinh thực tế trên máy chủ

Bảng 1: Cấu trúc dữ liệu sự kiện `SubscriptionPurchasedEvent`

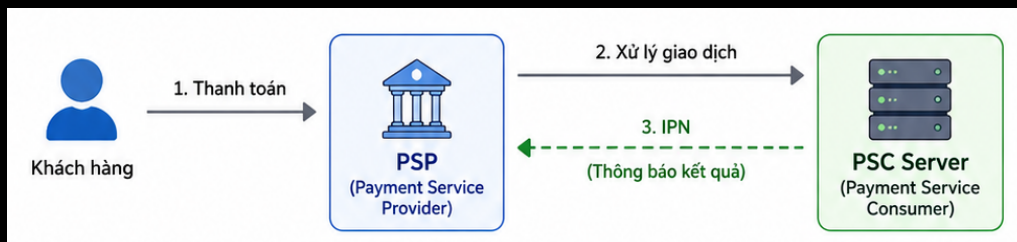
Ví dụ *JSON Payload*:

```
{
  "subscriptionId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "a1b2c3d4-e5f6-7a8b-9c0d-e1f2a3b4c5d6",
  "planId": "c9a64e1f-9c02-4c28-94ef-23e595a8947f",
  "periodStart": "2026-06-15T02:16:31.000Z",
  "periodEnd": "2026-07-15T02:16:31.000Z",
  "version": 1,
  "occurredAt": "2026-06-15T02:16:32.124Z"
}
```

Cách các dịch vụ khác tiếp nhận và xử lý sự kiện Sự kiện này được tiêu thụ (consume) song song bởi hai dịch vụ: **Dịch vụ văn bản (document service - Python)** (Consumer Group: `document-service-group`) và **Dịch vụ trò chuyện (conversation service - NestJS)** (Consumer Group: `conversation-service-group`) để đồng bộ quyền lợi gói VIP của người dùng theo các bước tuần tự sau:

1. **Lắng nghe và giải mã:** Consumer lắng nghe sự kiện từ topic tương ứng, phân tích và giải mã JSON payload nhận được.
2. **Khởi tạo giao dịch và khóa dòng:** Khởi tạo một giao dịch CSDL (**Database Transaction**) để thực hiện cập nhật bảng thông tin thuê bao của người dùng. Để tránh tranh chấp dữ liệu (Race Condition) khi có nhiều sự kiện xử lý song song, hệ thống kích hoạt khóa ghi bi quan (**Pessimistic Write Lock** hoặc cơ chế tương đương) trên dòng dữ liệu của `userId` đó.
3. **So sánh phiên bản lớn hơn:** So sánh trường `version` của sự kiện nhận được với `version` hiện tại trong CSDL. Nếu phiên bản của sự kiện lớn hơn ($>$), tiến hành cập nhật thông tin thuê bao (`period_start`, `period_end`, `subscription_id`, `version`) và đặt trạng thái kích hoạt VIP (`isActive = true` / `is_active = True`).
4. **Xử lý phiên bản nhỏ hơn hoặc bằng:** Nếu phiên bản nhận được nhỏ hơn hoặc bằng ($\leq$), hệ thống sẽ bỏ qua (SKIP) sự kiện vì đã tồn tại bản cập nhật mới hơn trong CSDL (cơ chế Version Control giúp ngăn chặn lỗi out-of-order event).
5. **Kích hoạt quyền VIP:** Sau khi cập nhật CSDL thành công, hệ thống mở khóa người dùng và kích hoạt các quyền lợi VIP tương ứng (như trò chuyện thoại năng cao, mô hình suy luận, bóc tách và tóm tắt văn bản).

Cơ chế Instant Payment Notification (IPN) IPN (Instant Payment Notification) được sử dụng để Đơn vị cung cấp dịch vụ thanh toán - Payment Service Provider (PSP) thông báo kết quả giao dịch một cách không đồng bộ cho Đơn vị sử dụng dịch vụ - Payment Service Consumer (PSC).



Hình 1.30: Sơ đồ nguyên lý hoạt động của IPN

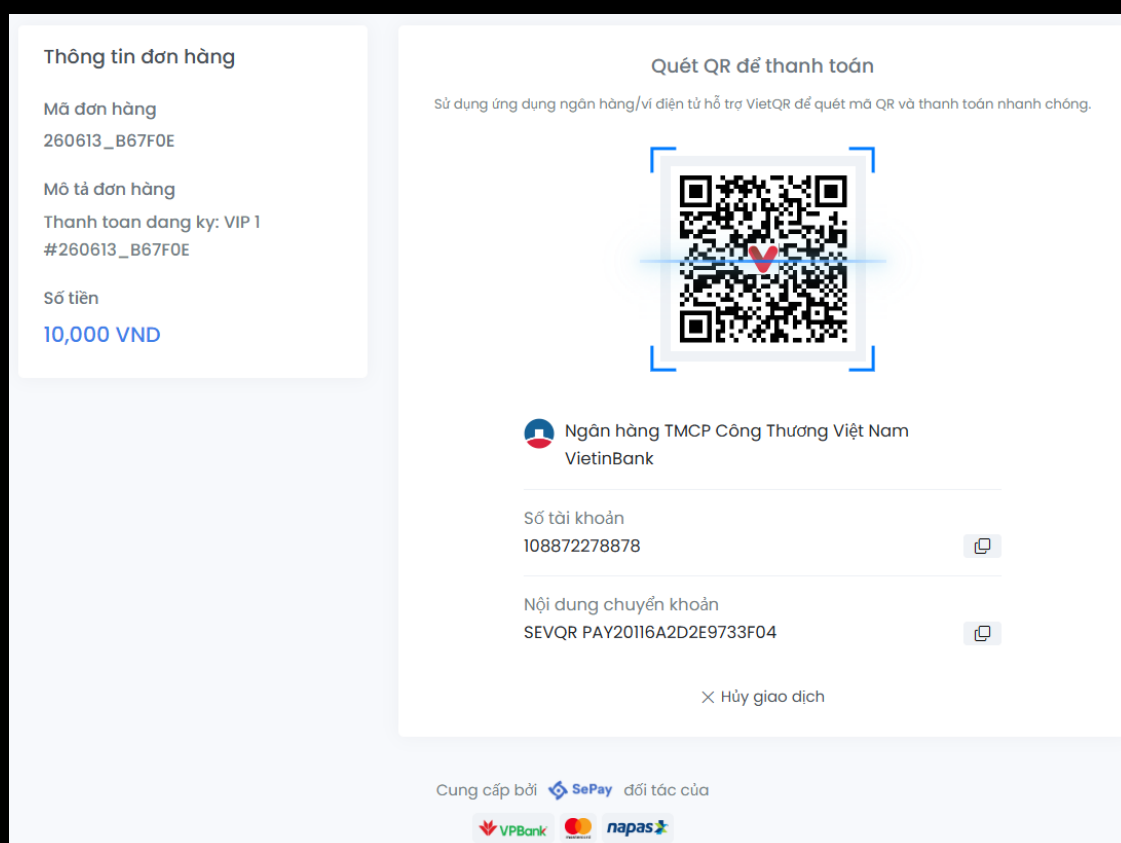
Sau khi khách hàng hoàn tất thanh toán, kết nối mạng của họ có thể bị gián đoạn hoặc họ có thể tắt trình duyệt trước khi được chuyển hướng về trang Web của hệ thống (Return URL). Cơ chế IPN đảm bảo rằng máy chủ của hệ thống vẫn nhận được kết quả cuối cùng một cách độc lập từ đối tác để cập nhật trạng thái đơn hàng một cách chính xác.



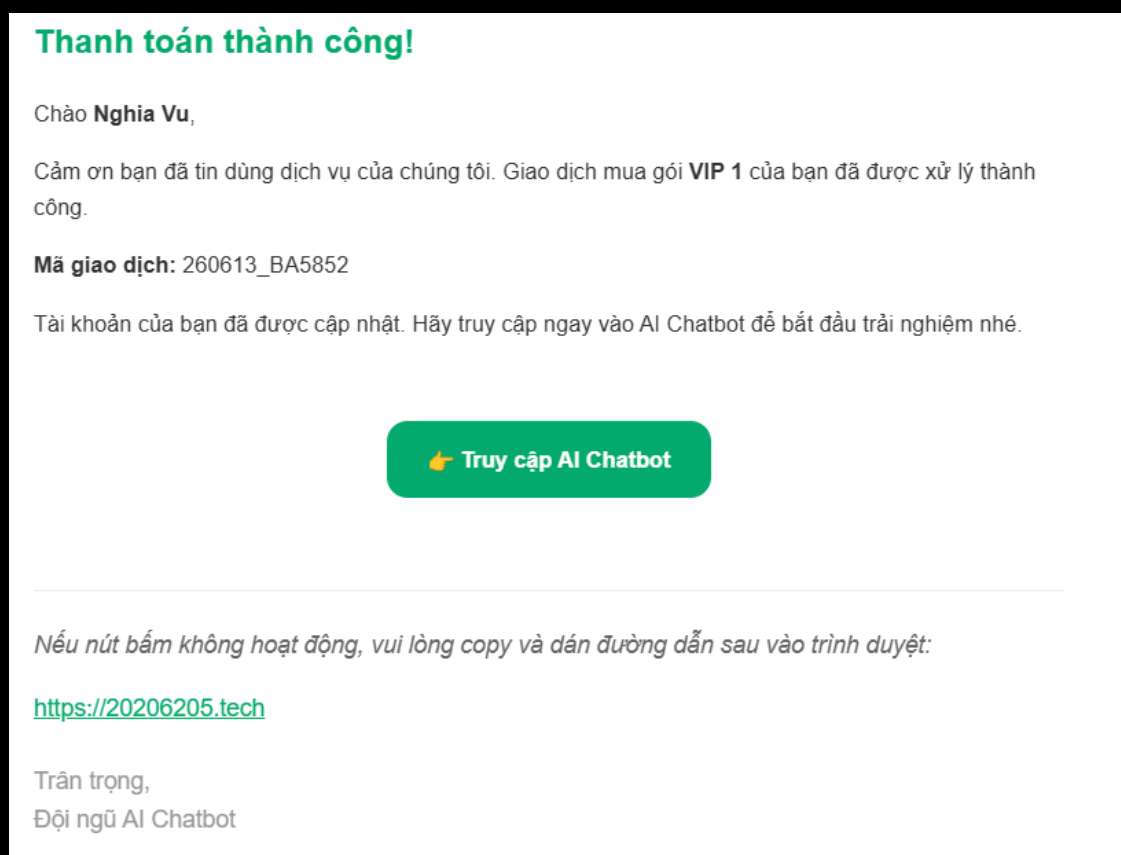
Hình 1.31: Mô phỏng luồng cập nhật trạng thái qua IPN

Thiết kế hệ thống theo Domain-Driven Design (DDD) Dự án được thiết kế theo mô hình Phát triển hướng tên miền (Domain-Driven Design - DDD) kết hợp với kiến trúc Lục giác (Hexagonal Architecture). Để demo khả năng thay đổi linh hoạt các dịch vụ tích hợp bên ngoài, hệ thống cho phép cấu hình cổng thanh toán mặc định thông qua biến môi trường `PAYMENT_DEFAULT_PROVIDER`. Nhờ áp dụng Dependency Inversion thông qua các Interface, việc chuyển đổi hoặc bổ sung các cổng thanh toán mới được thực hiện vô cùng dễ dàng mà không làm ảnh hưởng đến nghiệp vụ cốt lõi.

Hình 1.32: Giao diện thanh toán VNPay



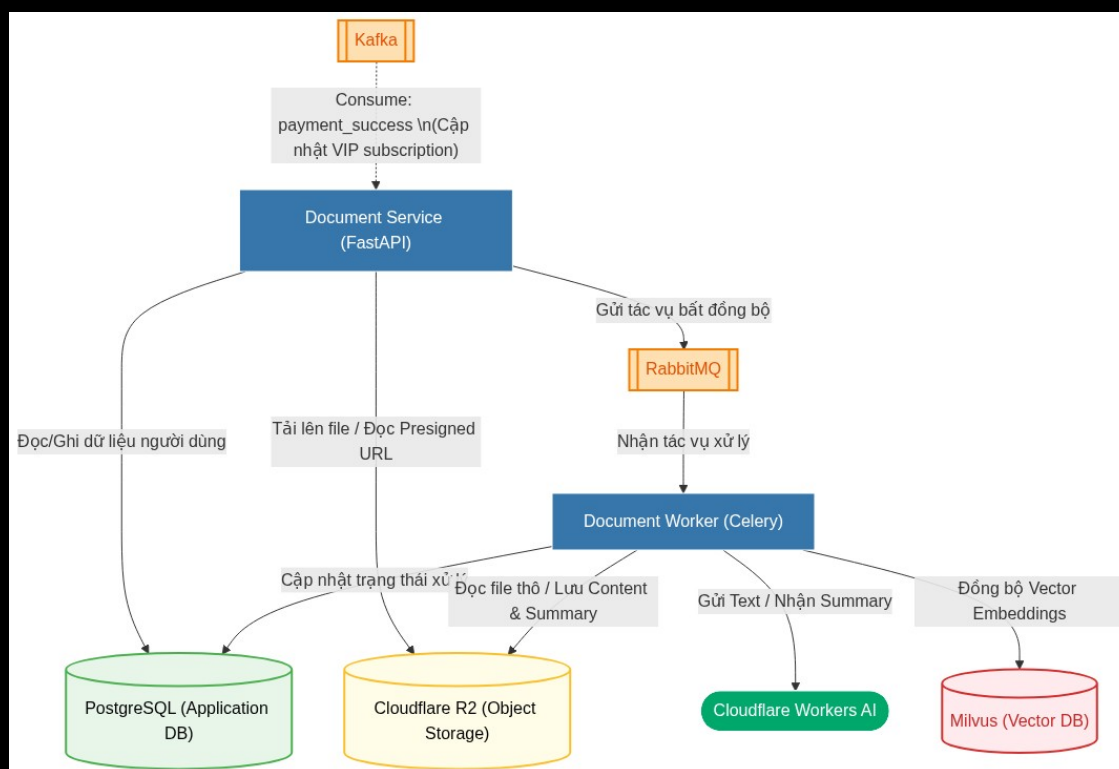
Hình 1.33: Giao diện thanh toán SePay



Hình 1.34: Email thông báo thanh toán thành công

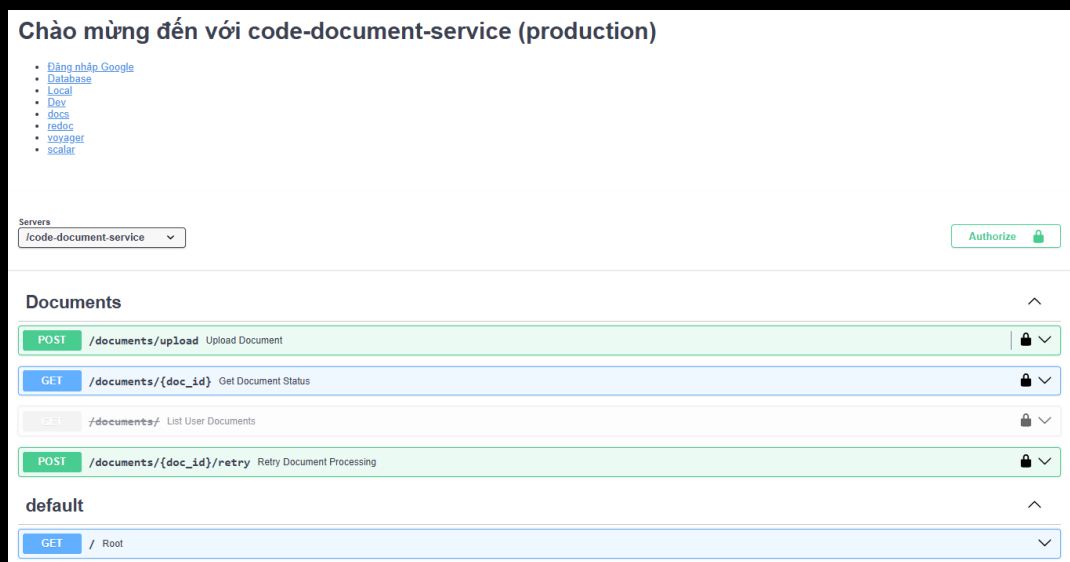
1.1.5 Dịch vụ văn bản (document service)

Sơ đồ tổng quan về Dịch vụ văn bản (Document Service):



Hình 1.35: Sơ đồ tổng quan Dịch vụ văn bản

Mục đích: Tiếp nhận, lưu trữ và quản lý luồng xử lý tài liệu của người dùng. Dịch vụ này không chỉ lưu trữ tệp tin mà còn hỗ trợ quy trình trích xuất nội dung sang định dạng Markdown, tự động tạo bản tóm tắt và thực hiện vector hóa tài liệu.



Hình 1.36: Tài liệu Swagger API Dịch vụ văn bản

Vì chức năng tải lên tài liệu chỉ dành riêng cho người dùng sở hữu gói VIP, dịch vụ văn bản (Document Service) thực hiện lắng nghe sự kiện thanh toán thành công từ Kafka, lưu thông tin vào CSDL để kiểm tra và xác thực quyền VIP của người dùng hiện tại.

- **Tải lên tài liệu mới:** Người dùng gửi tệp tài liệu lên hệ thống. Sau khi tiếp nhận thành công, hệ thống sẽ trả về mã định danh duy nhất (`doc_id`), tên tệp gốc và trạng thái khởi

tạo ban đầu, đồng thời đưa tác vụ vào hàng đợi xử lý.

- **Truy xuất trạng thái xử lý:** Dựa vào `doc_id`, người dùng có thể truy vấn trạng thái xử lý hiện tại của tài liệu. Hệ thống sẽ báo cáo chi tiết về sự tồn tại của các thành phần dữ liệu thông qua các cờ trạng thái: `has_file`, `has_content`, `has_summary`.
- **Thử lại tác vụ lỗi:** Trong trường hợp quá trình bóc tách hoặc tóm tắt gặp sự cố, hệ thống cung cấp một API chuyên biệt để kích hoạt lại quy trình xử lý cho tài liệu đó mà không cần người dùng phải tải lên lại tệp gốc.

Document Worker Dịch vụ văn bản kết hợp với **Document Worker** (nhận các tác vụ nền từ Celery thông qua RabbitMQ) để thực hiện quy trình xử lý tài liệu phục vụ hệ thống RAG:

1. **Tải tệp tin:** Tải tệp tin từ Cloudflare R2 và sử dụng thư viện `Docling` để bóc tách cấu trúc và nội dung văn bản.
2. **Tóm tắt văn bản:** Sử dụng mô hình ngôn ngữ trên Cloudflare Workers AI để tự động tạo bản tóm tắt nội dung tài liệu bằng tiếng Việt.
3. **Chia nhỏ văn bản (Chunking):** Sử dụng bộ chia `RecursiveCharacterTextSplitter` của `LangChain` để chia nhỏ văn bản thành các đoạn phù hợp.
4. **Trích xuất đặc trưng (Embedding):** Tạo vector biểu diễn ngữ nghĩa cho từng đoạn văn bản bằng mô hình `keepitreal/vietnamese-sbert`.
5. **Lưu trữ Vector:** Quản lý và lưu trữ các vector embedding vào CSDL vector Milvus để phục vụ tìm kiếm ngữ nghĩa sau này.

Cơ chế tối ưu & Độ bền bỉ (Optimization & Resilience)

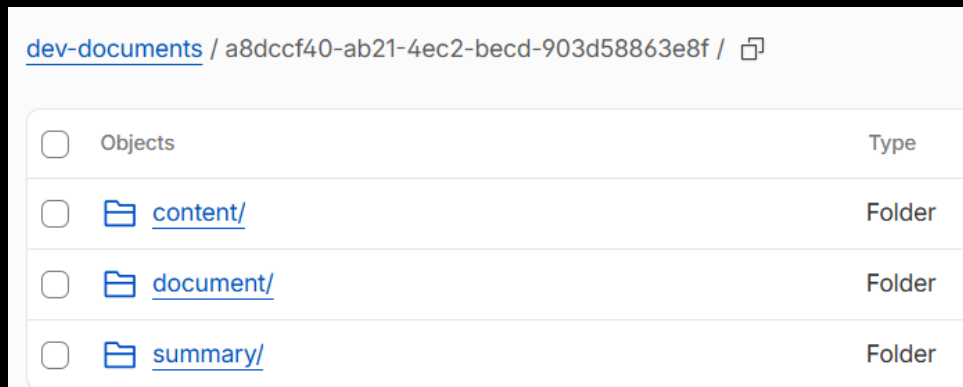
- **Tái sử dụng kết quả (Deduplication):** Để tối ưu chi phí và hiệu năng, trước khi chạy parser và AI, worker kiểm tra xem người dùng đã từng tải lên tệp tin có cùng dung lượng (`file_size`) và băm (`file_hash`) hay chưa. Nếu đã có tài liệu trùng khớp đã xử lý thành công, hệ thống sao chép trực tiếp kết quả bóc tách và tóm tắt trên Cloudflare R2 sang đường dẫn của tài liệu mới mà không cần chạy lại mô hình AI.
- **Cấu hình hàng đợi Celery bền bỉ:** Thiết lập `task_acks_late=True` để task chỉ được xóa khỏi RabbitMQ khi worker đã hoàn thành thực tế (tránh mất mát tác vụ); thiết lập `worker_prefetch_multiplier=1` ép worker chỉ nhận 1 tài liệu tại một thời điểm để cân bằng tải; tự động thử lại tối đa 3 lần (mỗi lần chờ 60 giây) nếu gặp sự cố kết nối ngoại vi.
- **Giám sát Langfuse:** Toàn bộ luồng tóm tắt văn bản bằng AI (Cloudflare Workers AI) được theo dõi vết và giám sát chi tiết thông qua nền tảng **Langfuse** để kiểm soát chất lượng và độ trễ.

Giao diện quản lý vector trên Milvus:

text	pk	Actions
CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM Độc lập – Tự do – Hạnh phúc **GIẤY ỦY QUYỀN** Hôm ...	4662434980633	Q ✎
Họ và tên : : ----- ----- -----	4662434980633	Q ✎
Địa chỉ thường trú : :	4662434980633	Q ✎
NỘI DUNG ỦY QUYỀN: Bên ủy quyền đồng ý ủy quyền cho bà **Vũ Thị Thủy** thay mặt công ty liê...	4662434980633	Q ✎

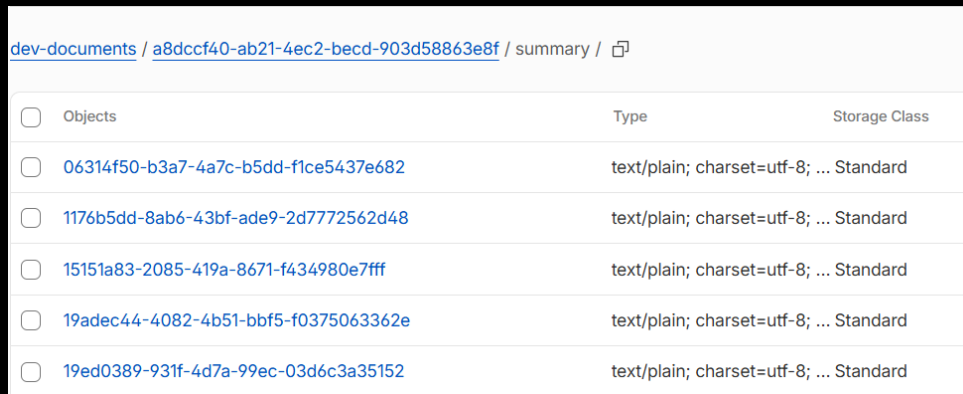
Hình 1.37: Dữ liệu vector lưu trữ trên Milvus

Quản lý các file tài liệu gốc trên Cloudflare R2:



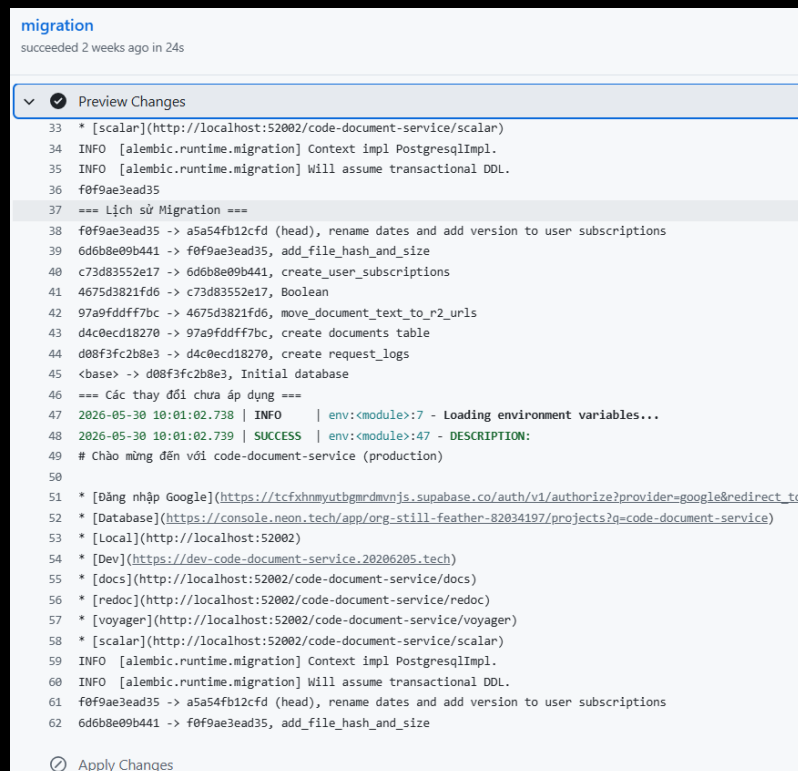
Hình 1.38: Tập tài liệu gốc lưu trữ trên Cloudflare R2

Quản lý nội dung tóm tắt lưu trữ trên Cloudflare R2:



Hình 1.39: Bản tóm tắt lưu trữ trên Cloudflare R2

Quản lý migration CSDL của dịch vụ văn bản:



Hình 1.40: Quản lý migration CSDL của dịch vụ văn bản

Kết quả các bảng trong CSDL tài liệu:

Table	Columns
alembic_version	version_num (varchar(32)), a5a54fb12cfd
documents	
request_logs	
user_subscriptions	

Hình 1.41: Kết quả các bảng trong CSDL tài liệu

1.1.6 Dịch vụ nhân vật (persona service)

Mục đích: Chịu trách nhiệm quản lý hệ thống nhân vật (personas), kho giọng đọc (Voices), các nền tảng tổng hợp giọng nói (TTS Engines) và xử lý trực tiếp các yêu cầu chuyển đổi văn bản thành âm thanh (Text-to-Speech).

Category	Method	Endpoint	Description	Lock
Public	GET	/personas	Get All Persona	
Persona Admin	POST	/personas	Create Persona	🔒
	PUT	/personas/{persona_id}	Update Persona	🔒
	DELETE	/personas/{persona_id}	Delete Persona	🔒
	POST	/personas/upload-avatar	Upload Persona Avatar	🔒
	POST	/personas/upload-audio	Upload Persona Audio	🔒
Audio	GET	/audio/v1/	Root	
	POST	/audio/v1/audio/speech	Generate Speech	🔒
	POST	/audio/v1/tts	Admin Generate Audio	🔒
Engine Admin				
Voice Admin				

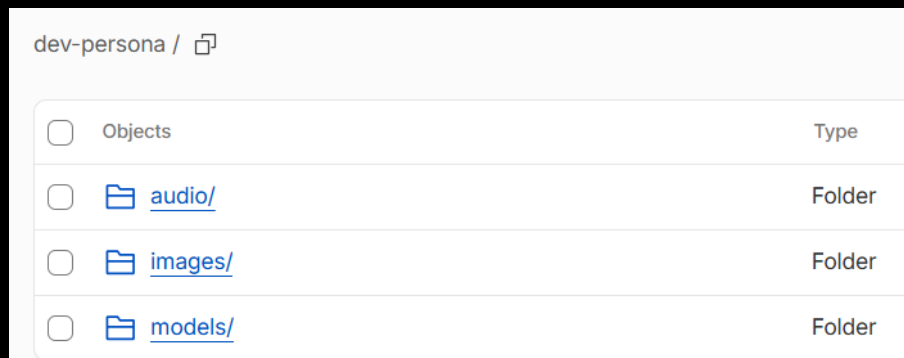
Hình 1.42: Tài liệu Swagger API Dịch vụ nhân vật

Các chức năng chính

- **Truy xuất danh sách nhân vật:** Cung cấp danh sách các nhân vật trong hệ thống, hỗ trợ phân trang và lọc theo `persona_id`.
- **Quản lý nhân vật (Dành cho ADMIN):** Cho phép tạo mới, cập nhật thông tin hoặc xóa bỏ các nhân vật khỏi hệ thống. Tải lên hình ảnh đại diện (**avatar**) và file âm thanh câu chào mẫu (welcome audio) cho từng nhân vật.
- **Quản lý công cụ TTS (engines):** Cho phép quản trị viên ADMIN định nghĩa, cập nhật, xóa và truy xuất danh sách các nền tảng dịch vụ cung cấp TTS.
- **Quản lý giọng đọc (voices):** Cho phép tạo, cập nhật, xóa và truy xuất danh sách các giọng đọc cụ thể, hỗ trợ bộ lọc theo công cụ/nền tảng.
- **Đồng bộ ElevenLabs:** Tích hợp API đồng bộ danh sách giọng đọc mới nhất từ nhà cung cấp ElevenLabs trực tiếp về hệ thống CSDL.

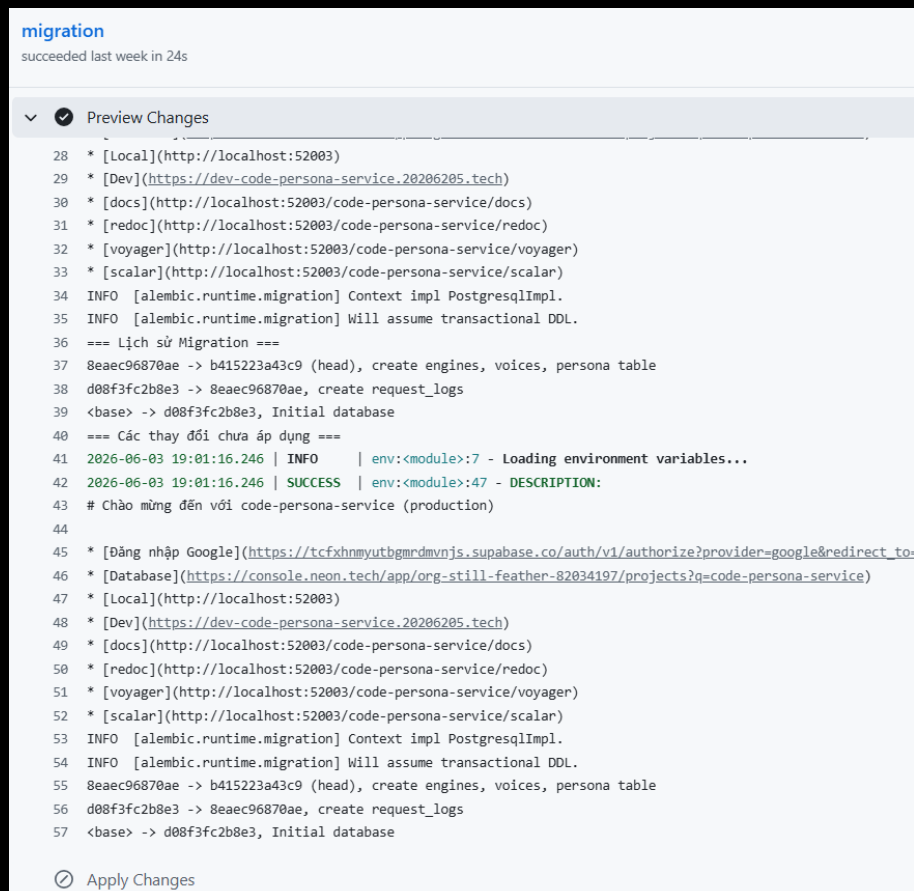
- **Chuyển đổi Text-to-Speech:** Cung cấp chức năng tạo ra âm thanh từ văn bản đầu vào thông qua hai thư viện: **edge-tts** (tổng hợp online) và **piper-tts** (tổng hợp offline chất lượng cao). Với **piper-tts**, hệ thống yêu cầu tệp mô hình giọng đọc được lưu trữ tại Cloudflare R2 và tự động tải xuống máy chủ cục bộ khi dịch vụ bắt đầu khởi động.
- **Cơ chế Cache tối ưu (TTS Caching):** Sử dụng **Disk Cache** (thư viện **diskcache** cơ chế LRU, kích thước giới hạn 1GB) để lưu giữ các file âm thanh đã tổng hợp lên ổ đĩa nhằm phần hồi tức thì các yêu cầu trùng lặp; và sử dụng **Memory Cache** để lưu giữ các đối tượng mô hình Piper đã load vào bộ nhớ RAM (**_loaded_voices**), tránh việc phải nạp lại tệp tin mô hình **.onnx** nặng sau mỗi yêu cầu, tối ưu tốc độ phát âm thanh.
- **Lưu trữ tài nguyên:** Toàn bộ dữ liệu mẫu âm thanh (**audio**), hình ảnh (**images**) và mô hình (**models**) phục vụ cho dịch vụ nhân vật được lưu trữ an toàn tại Cloudflare R2.

Quản lý thư mục tài nguyên của dịch vụ nhân vật trên Cloudflare R2:



Hình 1.43: Tài nguyên nhân vật lưu trữ trên Cloudflare R2

Quản lý migrations CSDL của dịch vụ nhân vật:



Hình 1.44: Quản lý migration CSDL của dịch vụ nhân vật

Cấu trúc các bảng trong CSDL của Dịch vụ nhân vật:

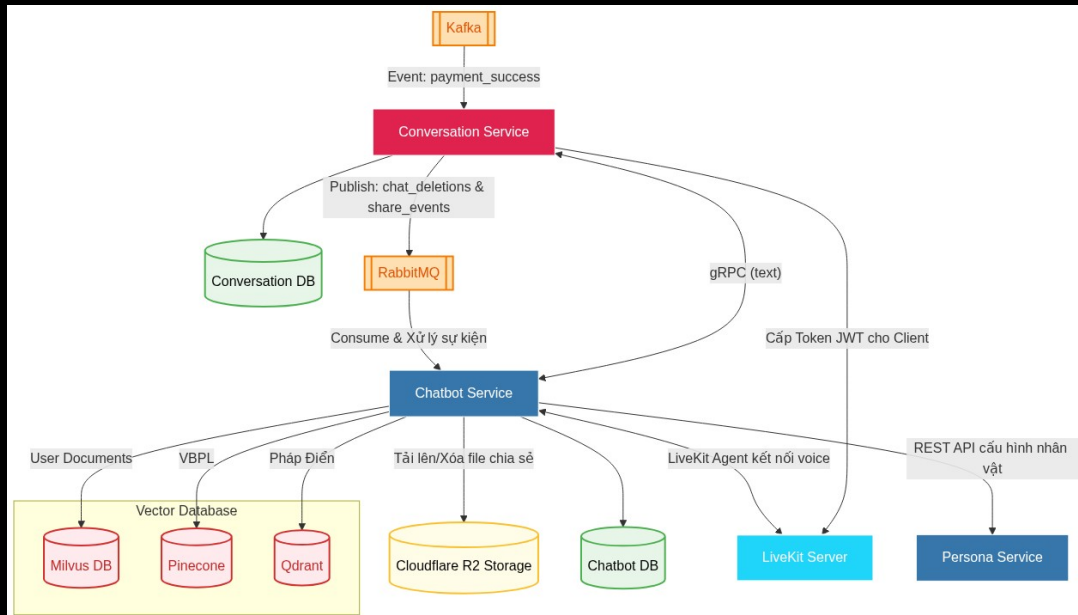
Tables					
main					
neondb					
Schema					
public					
Search...					
alembic_version					
engines					
persona					
request_logs					
voices					

id	code	name	is_active	voices
a12e3456-789a-bcde-f01...	edge_tts	Edge TTS	TRUE	voices
b12e3456-789a-bcde-f01...	elevenlabs	ElevenLabs	TRUE	voices
e12e3456-789a-bcde-f01...	pipet_tts	Piper TTS	TRUE	voices

Hình 1.45: Cấu trúc các bảng trong CSDL của Dịch vụ nhân vật

1.1.7 Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service)

Sơ đồ tổng quan về Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service):



Hình 1.46: Sơ đồ tổng quan Dịch vụ trò chuyện và Dịch vụ chatbot

Dịch vụ trò chuyện (conversation service) Vì chức năng trò chuyện văn bản suy luận (reasoning) và trò chuyện bằng giọng nói chỉ dành riêng cho tài khoản VIP, nên dịch vụ trò chuyện (conversation service) thực hiện lắng nghe sự kiện thanh toán thành công từ Kafka, lưu lại thông tin vào CSDL và kiểm tra quyền VIP của người dùng hiện tại trước khi xử lý yêu cầu.

Mục đích: Chịu trách nhiệm quản lý luồng giao tiếp giữa người dùng và hệ thống Agentic RAG. Dịch vụ này xử lý trực tiếp các cuộc trò chuyện văn bản theo cơ chế streaming, cấp phát token truy cập cho luồng giao tiếp giọng nói và cung cấp các tiện ích lưu trữ, chia sẻ trò chuyện.

Các tính năng chính:

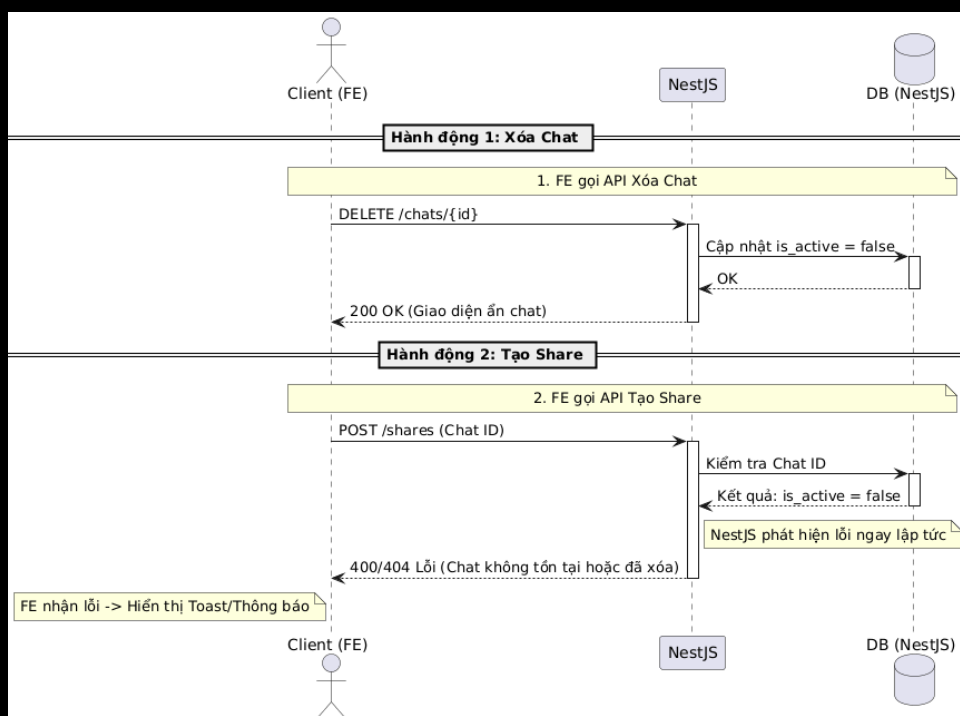
- **Khởi tạo trò chuyện:** Bắt đầu một luồng trò chuyện mới trong hệ thống.
- **Cấp phát token thoại:** Cấp phát token bảo mật phục vụ kết nối đàm thoại bằng giọng nói với máy chủ LiveKit.
- **Định tuyến tin nhắn:** Tiếp nhận tin nhắn từ người dùng và kết nối với Dịch vụ chatbot (chatbot service) thông qua giao thức gRPC để xử lý thông tin.
- **Xóa cuộc trò chuyện:** Hỗ trợ xóa mềm (soft delete) một cuộc trò chuyện khỏi hệ thống.
- **Quản lý thư mục:** Cho phép người dùng tạo các thư mục lưu trữ để phân loại và sắp xếp các cuộc trò chuyện.

- **Đánh dấu và Ghi chú:** Đánh dấu một cuộc trò chuyện quan trọng và ghi chú các nội dung tóm tắt đi kèm.
- **Chia sẻ trò chuyện:** Tạo ra một bản snapshot chia sẻ đoạn trò chuyện với người dùng khác.
- **Đường dẫn chia sẻ:** Cung cấp URL công khai chứa tham số **shareId** và token bảo mật đi kèm. Chủ sở hữu đoạn chat có thể xem danh sách các liên kết đã tạo và thu hồi quyền truy cập bất cứ lúc nào.



Hình 1.47: Tài liệu Swagger API Dịch vụ trò chuyện

Sơ đồ trình tự kiểm duyệt nghiệp vụ khi xóa cuộc trò chuyện và chặn đứng việc chia sẻ cuộc trò chuyện đã bị xóa:

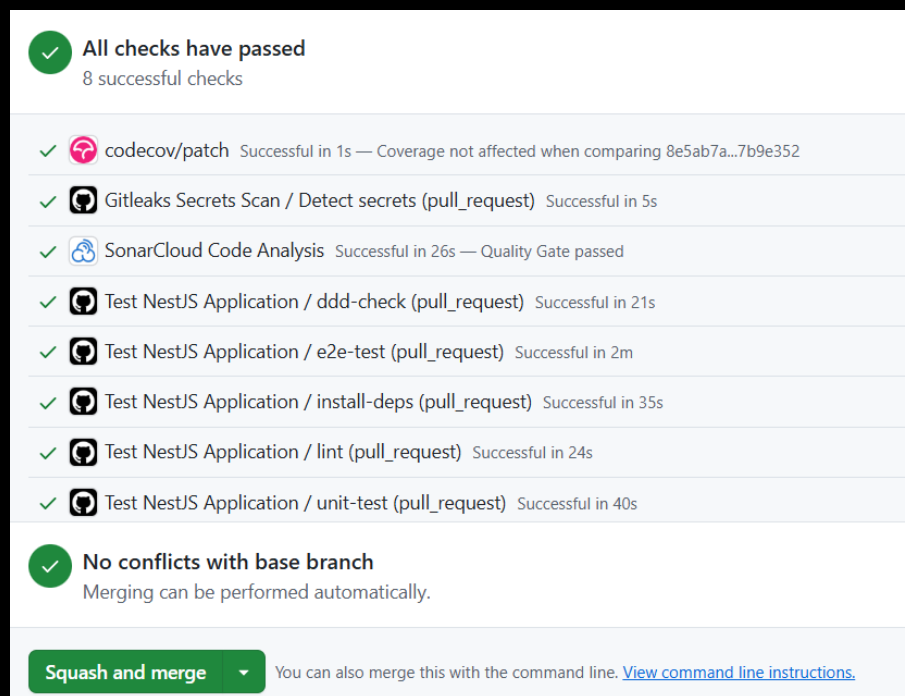


Hình 1.48: Sơ đồ trình tự xóa cuộc trò chuyện và kiểm duyệt chia sẻ

Chi tiết luồng xử lý và kiểm duyệt (Validation Logic):

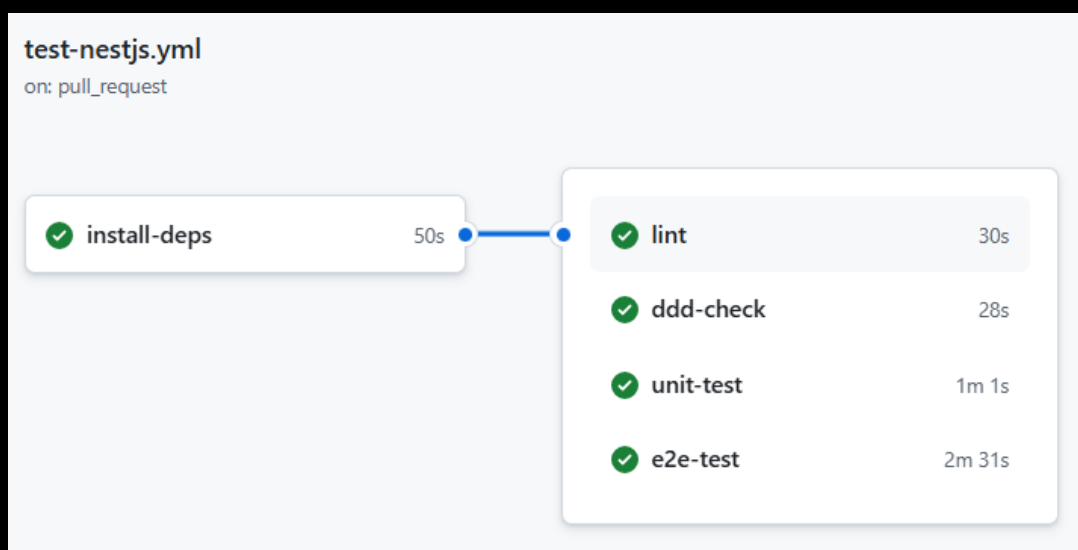
1. **Xóa cuộc trò chuyện (Delete Chat):** Client gửi yêu cầu DELETE /chats/{id} đến Dịch vụ trò chuyện (NestJS). Hệ thống tiến hành cập nhật trường trạng thái của cuộc trò chuyện trong Database thành `isActive = false` (thực hiện cơ chế xóa mềm - soft delete) và phản hồi 200 OK. Lúc này, cuộc trò chuyện sẽ lập tức được ẩn đi trên giao diện người dùng.
2. **Kiểm duyệt tạo liên kết chia sẻ (Validate Share Link):** Khi client gửi yêu cầu POST /shares (kèm theo chat_id) để tạo link chia sẻ công khai, trước khi lưu bản ghi chia sẻ, `GenerateLinkShareCommandHandler` sẽ truy vấn thông tin cuộc trò chuyện từ Database dựa trên chat_id. Nếu cuộc trò chuyện không tồn tại hoặc có trạng thái `isActive = false` (đã bị xóa mềm), hệ thống sẽ chặn đứng hành động ngay lập tức và ném ra lỗi `ChatNotFoundException` (trả về mã lỗi 400/404 cho client kèm theo thông báo lỗi chi tiết trên giao diện UI).

Thông tin kiểm tra tự động Pull Request của dịch vụ trò chuyện trên GitHub:



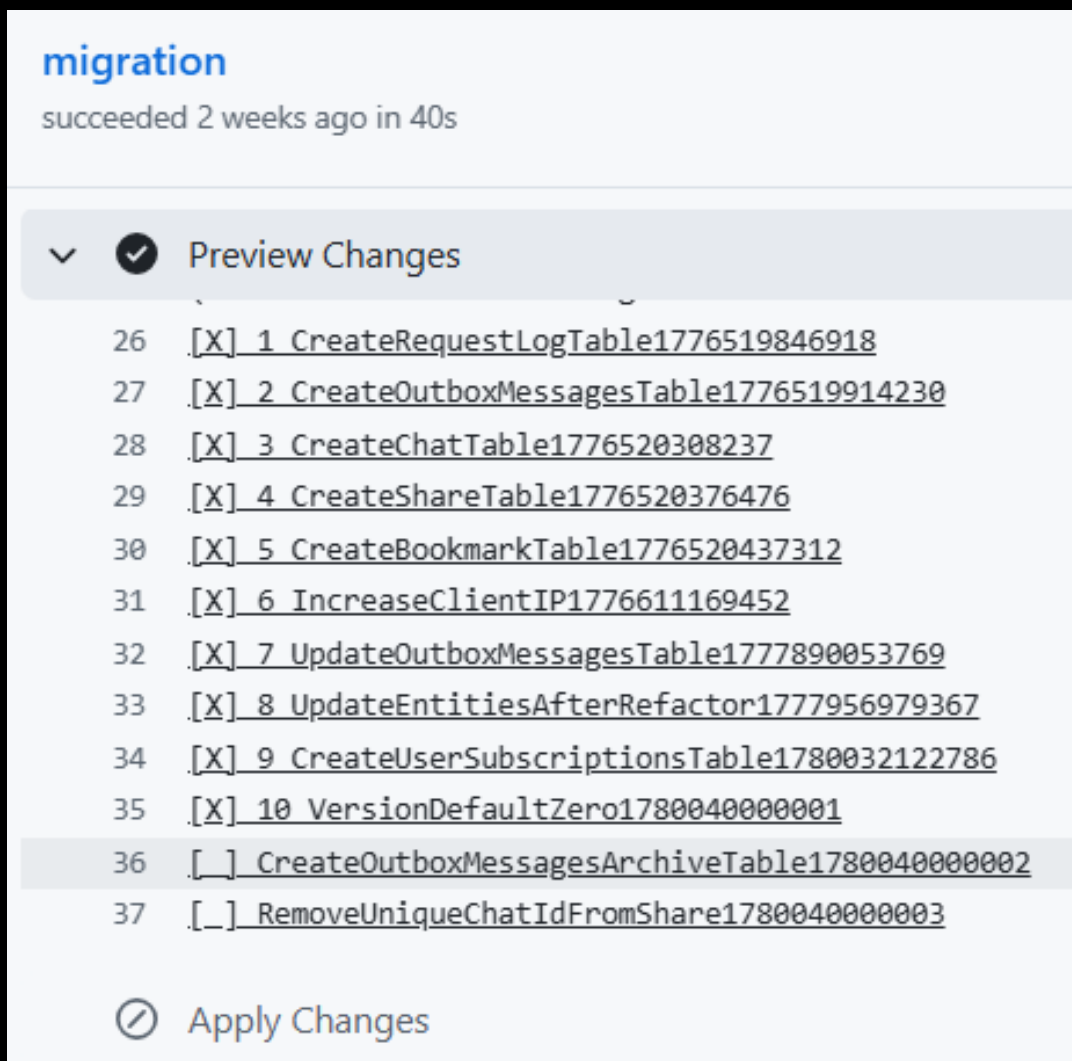
Hình 1.49: Kiểm tra Pull Request của conversation service trên GitHub

Kiểm tra cú pháp nguồn (linting) tự động trong CI/CD:



Hình 1.50: Kiểm tra code linting dịch vụ trò chuyện

Quản lý migrations CSDL của dịch vụ trò chuyện:



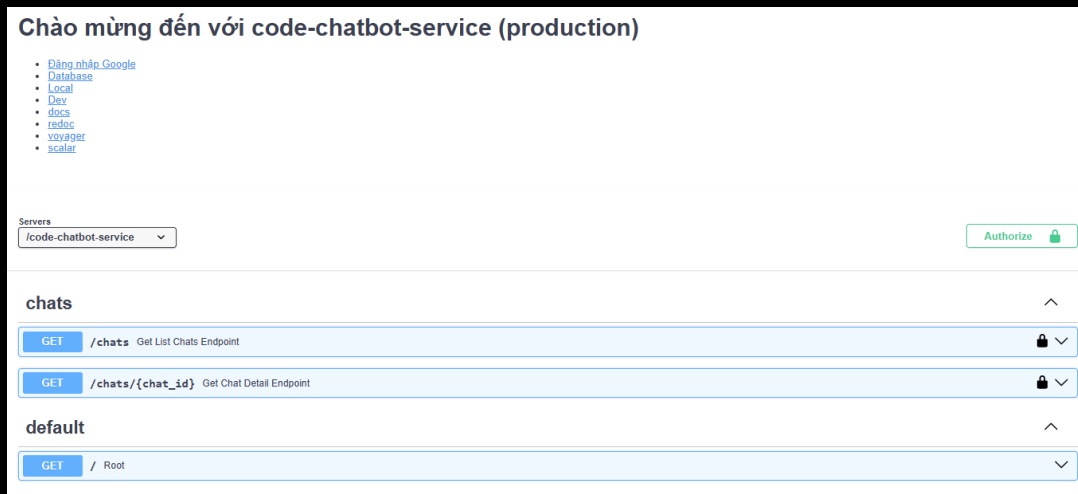
Hình 1.51: Quản lý database migrations dịch vụ trò chuyện

Cơ cấu các bảng trong CSDL của Dịch vụ trò chuyện:

Tables		DATA STRUCTURE Filters Sort Columns Add record			
neondb Schema public Search... bookmark_folder bookmark_item chat migrations outbox_messages outbox_messages_archive request_logs shared_chat user_subscriptions		☐ id serial ☐ timestamp bigint ☐ name varchar			
		☐ 1	1776519846918	CreateRequestLogTable1...	
		☐ 2	1776519914230	CreateOutboxMessagesTa...	
		☐ 3	1776520308237	CreateChatTable1776520...	
		☐ 4	1776520376476	CreateShareTable177652...	
		☐ 5	1776520437312	CreateBookmarkTable177...	
		☐ 6	1776611169452	IncreaseClientIP177661...	
		☐ 7	1777890053769	UpdateOutboxMessagesTa...	
		☐ 8	1777956979367	UpdateEntitiesAfterRef...	
		☐ 9	1780032122786	CreateUserSubscription...	
		☐ 10	1780040000001	VersionDefaultZero1780...	
		☐ 11	1780040000002	CreateOutboxMessagesAr...	
		☐ 12	1780040000003	RemoveUniqueChatIdFrom...	

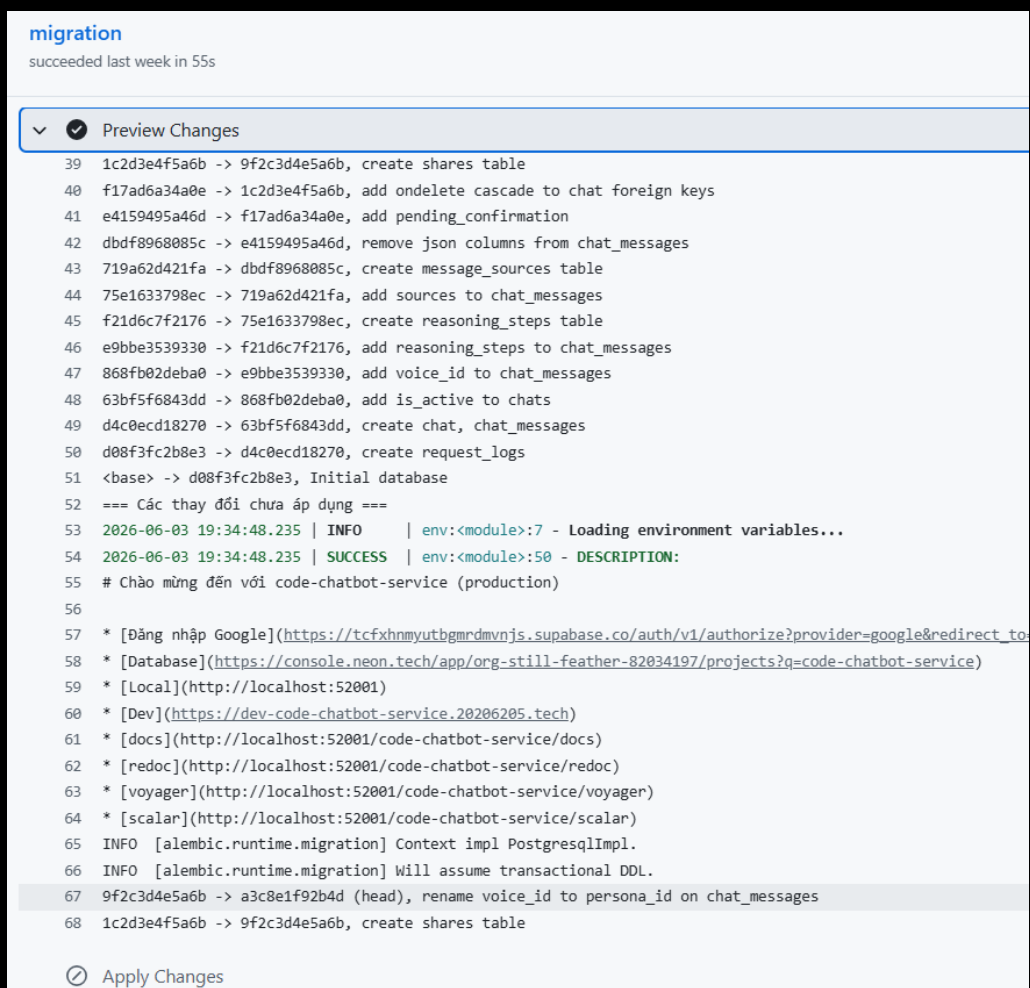
Hình 1.52: Cấu hình các bảng trong CSDL conversation

Dịch vụ chatbot (chatbot service) Mục đích: Sử dụng các mô hình AI để xử lý và phản hồi thông minh các câu hỏi của người dùng. Sau đó, lưu lại lịch sử trò chuyện và cung cấp các API truy xuất lịch sử, thông tin suy luận chi tiết (reasoning details) và nguồn tài liệu tham khảo (references) của từng câu trả lời. Dịch vụ chatbot lắng nghe RabbitMQ để thực hiện việc tạo và thu hồi các bản snapshot chia sẻ trò chuyện, đồng thời kết nối với LiveKit để vận hành đàm thoại giọng nói với người dùng.



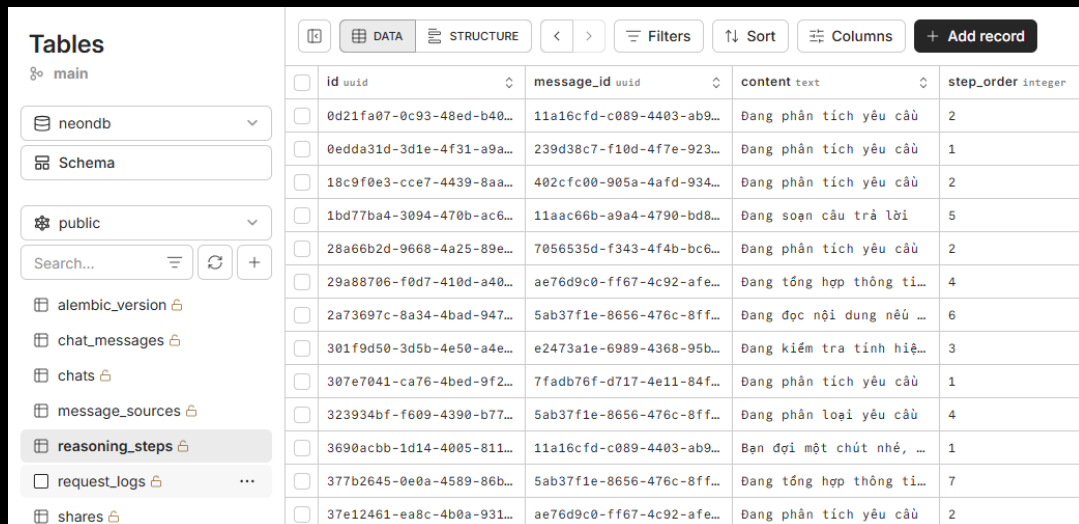
Hình 1.53: Tài liệu Swagger API Dịch vụ chatbot

Quản lý migrations CSDL của dịch vụ chatbot:



Hình 1.54: Quản lý database migrations dịch vụ chatbot

Cơ cấu các bảng trong CSDL của Dịch vụ chatbot:



id	message_id	content	step_order
0d21fa07-0c93-48ed-b40...	11a16cfd-c089-4403-ab9...	Đang phân tích yêu cầu	2
0edda31d-3d1e-4f31-a9a...	239d38c7-f10d-4f7e-923...	Đang phân tích yêu cầu	1
18c9f0e3-cce7-4439-8aa...	402cfc00-905a-4afd-934...	Đang phân tích yêu cầu	2
1bd77ba4-3094-470b-ac6...	11aac66b-a9a4-4790-bd8...	Đang soạn câu trả lời	5
28a66b2d-9668-4a25-89e...	7056535d-f343-4f4b-bc6...	Đang phân tích yêu cầu	2
29a88706-f0d7-410d-a40...	ae76d9c0-ff67-4c92-afe...	Đang tổng hợp thông ti...	4
2a73697c-8a34-4bad-947...	5ab37f1e-8656-476c-8ff...	Đang đọc nội dung nếu ...	6
301f9d50-3d5b-4e50-a4e...	e2473a1e-6989-4368-95b...	Đang kiểm tra tính hiệ...	3
307e7041-ca76-4bed-9f2...	7fad76f-d717-4e11-84f...	Đang phân tích yêu cầu	1
323934bf-f609-4390-b77...	5ab37f1e-8656-476c-8ff...	Đang phân loại yêu cầu	4
3690acbb-1d14-4005-811...	11a16cfd-c089-4403-ab9...	Bạn đợi một chút nhé, ...	1
377b2645-0e0a-4589-86b...	5ab37f1e-8656-476c-8ff...	Đang tổng hợp thông ti...	7
37e12461-ea8c-4b0a-931...	ae76d9c0-ff67-4c92-afe...	Đang phân tích yêu cầu	2

Hình 1.55: Cấu hình các bảng trong CSDL chatbot

Hệ thống trao đổi thông điệp qua RabbitMQ (RabbitMQ Messaging) Hệ thống sử dụng RabbitMQ làm Message Broker để thực hiện các giao tiếp bất đồng bộ, đáng tin cậy giữa **Dịch vụ trò chuyện (NestJS)** và **Dịch vụ chatbot (Python)**. Các tên hàng đợi được tiền tố hóa tự động theo môi trường (dev_, test_, prod_) dựa trên cấu hình hệ thống.

Danh sách các Hàng đợi trong hệ thống

- [prefix]_chat_deletions: Nhận yêu cầu xóa mềm (soft delete) lịch sử chat từ phía người dùng (Durable: true).
- [prefix]_share_generated: Yêu cầu tạo trang chia sẻ trò chuyện công khai (đóng gói JSONL và tải lên Cloudflare R2).
- [prefix]_share_generated_retry: Hàng đợi trung gian để xử lý lại (retry) khi gặp lỗi lệch thứ tự sự kiện (out of order). Cấu hình độ trễ cố định `x-message-ttl = 20s` trước khi gửi trả lại queue chính.
- [prefix]_share_generated_dlq: Dead Letter Queue nhận thông điệp tạo share thất bại hoàn toàn sau `MAX_RETRY_COUNT = 5` lần thử lại.
- [prefix]_share_upload_completed: Báo cáo kết quả đóng gói và upload file chia sẻ từ Python về lại NestJS.
- [prefix]_share_upload_completed_dlq: Dead Letter Queue nhận thông điệp báo cáo kết quả upload bị xử lý thất bại ở NestJS.
- [prefix]_share_revoked: Nhận yêu cầu thu hồi link chia sẻ, tiến hành xóa file tương ứng trên Cloudflare R2 và cập nhật DB (Durable: true).

Sự kiện Xóa trò chuyện

- Queue: [prefix]_chat_deletions
- Publisher: code-conversation-service (NestJS)
- Consumer: code-chatbot-service (Python)
- Payload Schema (JSON):

```
{
  "chat_id": "string", // Mã ID cuộc trò chuyện cần xóa
  "user_id": "string", // Mã ID người dùng sở hữu cuộc trò chuyện
  "version": 0 // Phiên bản sự kiện để kiểm tra thứ tự
}
```

Sự kiện Yêu cầu chia sẻ (Share Generated Event)

- **Queue:** [prefix]_share_generated
- **Publisher:** code-conversation-service (NestJS)
- **Consumer:** code-chatbot-service (Python)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID duy nhất của liên kết chia sẻ
  "chat_id": "string", // ID cuộc trò chuyện được chia sẻ
  "user_id": "string", // ID người dùng tạo chia sẻ
  "version": 1, // Phiên bản sự kiện (khởi tạo thường là 1)
  "last_message_id": "string" // ID tin nhắn cuối cùng
}
```

- **Cơ chế xử lý lệch thứ tự (Out-of-Order):** Consumer Python so sánh phiên bản sự kiện nhận được với phiên bản hiện tại trong DB. Nếu `event_version == current_version + 1`, tiến hành đóng gói lịch sử chat, upload R2 và gửi kết quả về NestJS. Nếu `event_version <= current_version`, bỏ qua vì đã xử lý. Nếu `event_version > current_version + 1`, sự kiện đến sai thứ tự, ném ra ngoại lệ `OutOfOrderException` để đưa tin nhắn vào `retry` tạm chờ 20 giây trước khi thử lại. Sau 5 lần thất bại liên tục sẽ bị chuyển vào DLQ và gửi cảnh báo về hệ thống.

Sự kiện Thu hồi liên kết (Share Revoked Event)

- **Queue:** [prefix]_share_revoked
- **Publisher:** code-conversation-service (NestJS)
- **Consumer:** code-chatbot-service (Python)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID liên kết chia sẻ cần thu hồi
  "file_url": "string", // URL tệp tin JSONL cần xóa trên Cloudflare R2
  "version": 2 // Phiên bản sự kiện (thường là phiên bản kế tiếp)
}
```

- **Cơ chế xử lý:** Xóa tệp tin trên Cloudflare R2, đánh dấu `is_revoked = True` trong DB. Hỗ trợ cơ chế tự động gửi lại hàng đợi chính (requeue) nếu phát hiện sự kiện đến lệch thứ tự (revoke đến trước khi generate kịp ghi DB).

Sự kiện Hoàn tất Upload (Share Upload Completed Event)

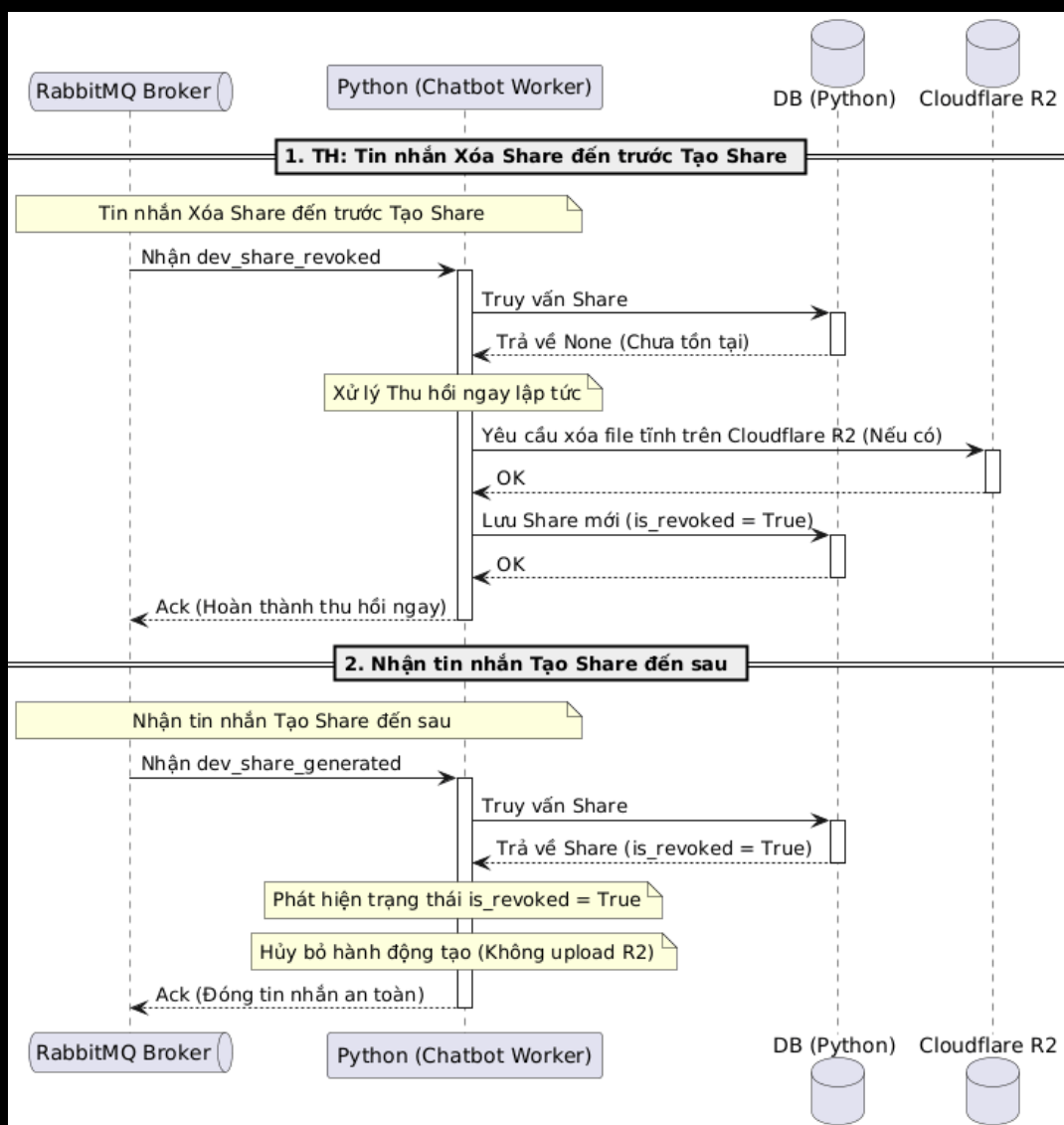
- **Queue:** [prefix]_share_upload_completed
- **Publisher:** code-chatbot-service (Python)
- **Consumer:** code-conversation-service (NestJS)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID liên kết chia sẻ tương ứng
  "file_url": "string", // URL tệp tin JSONL
  "is_success": true // Trạng thái thành công (true / false)
}
```

- **Cơ chế xử lý:** NestJS nhận thông điệp sẽ tiến hành cập nhật trạng thái của bản ghi chia sẻ thành công hoặc thất bại trong DB (sử dụng Manual Ack để đảm bảo không bị mất dữ liệu).

Xử lý lệch thứ tự sự kiện Tạo Share và Thu hồi/Xóa Share (Out-of-Order Handling)

- **Tình huống:** Người dùng vừa tạo Share và lập tức thực hiện xóa/thu hồi Share đó. Do tính chất bất đồng bộ của Message Broker, sự kiện Thu hồi Share (`share_revoked`) có thể đến Python worker **trước** sự kiện Tạo Share (`share_generated`).
- **Cơ chế kiểm tra trạng thái Lũy đẳng (State-based Idempotency):** Vì việc thu hồi liên kết thường liên quan đến vấn đề rò rỉ thông tin nhạy cảm của người dùng, nghiệp vụ yêu cầu lệnh thu hồi phải được thực hiện **ngay lập tức và ưu tiên cao nhất**. Do đó, hệ thống không áp dụng cơ chế tuần tự kiểm tra version phức tạp mà sử dụng cờ trạng thái `is_revoked` để triệt tiêu sự kiện:



Hình 1.56: Sơ đồ xử lý lệch thứ tự tạo và thu hồi share

1. **Khi sự kiện Thu hồi Share đến trước:** Python kiểm tra trạng thái trong DB (chưa có bản ghi nào), tiến hành gọi xóa tệp tin trên Cloudflare R2 (nếu đã được tạo trước đó), lưu bản ghi Share vào DB với trạng thái `is_revoked = True` và phản hồi Ack để hoàn thành tin nhắn.
2. **Khi sự kiện Tạo Share đến sau:** Python kiểm tra DB và phát hiện bản ghi đã tồn tại với trạng thái `is_revoked = True`. Khi đó, áp dụng **Cơ chế Triệt tiêu sự kiện (Event Cancellation)**: lập tức bỏ qua (drop) hoàn toàn hành động tạo và upload tệp lên R2 để bảo vệ tuyệt đối dữ liệu nhạy cảm của người dùng, sau đó gửi Ack để đóng tin nhắn một cách an toàn.

Thiết lập hàng đợi ưu tiên (Priority Queue) trên RabbitMQ

Để đảm bảo các thông điệp thu hồi khẩn cấp được đẩy lên xử lý trước trong trường hợp hàng đợi bị nghẽn (backlog), hệ thống hỗ trợ tích hợp **Priority Queue**:

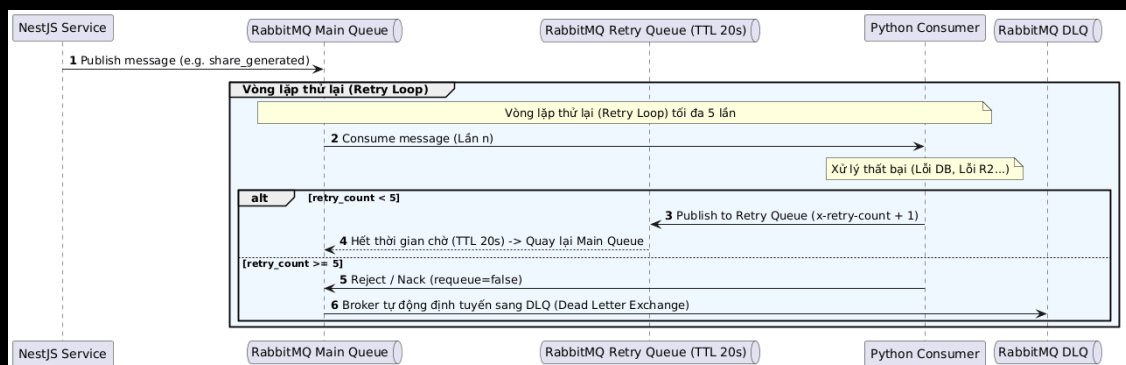
1. **Khai báo hàng đợi hỗ trợ ưu tiên:** Khi khởi tạo queue trên RabbitMQ, gán đối số cấu hình mức ưu tiên cao nhất (khuyến nghị tối đa là 10 để tránh tốn tài nguyên RAM/CPU):

```
channel.queue_declare(
    queue='[prefix]_share_events_queue',
    arguments={'x-max-priority': 10}
)
```

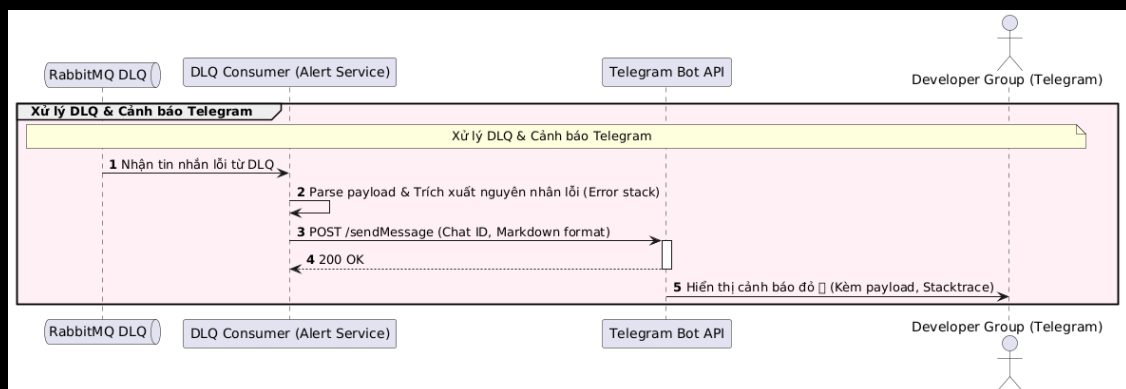
2. **Gửi sự kiện kèm mức độ ưu tiên (Producer):** Sự kiện Thu hồi Share (`share_revoked`) được gửi với độ ưu tiên khẩn cấp (`priority = 10`); Sự kiện Tạo Share (`share_generated`) được gửi với độ ưu tiên bình thường (`priority = 1`).
3. **Cấu hình QOS cho Consumer:** Thiết lập thuộc tính `prefetch_count = 1` trên consumer để bắt buộc RabbitMQ chỉ gửi đúng 1 thông điệp tại một thời điểm cho worker. Điều này giúp hàng đợi có cơ hội sắp xếp và đẩy các tin nhắn có độ ưu tiên cao (`priority = 10`) lên xử lý trước.

Luồng xử lý hàng đợi thư chết (DLQ) kết hợp Cảnh báo Telegram

- **Mục đích:** Khi một tin nhắn gặp lỗi xử lý lặp đi lặp lại nhiều lần (lỗi logic, lỗi kết nối CSDL kéo dài, lỗi phân tích cú pháp dữ liệu), hệ thống cần có lập tin nhắn đó để tránh làm nghẽn hàng đợi chính, đồng thời phát cảnh báo tức thời cho đội ngũ vận hành (Developers/DevOps).
- **Quy trình hoạt động:** Sự kết hợp giữa cơ chế Dead Letter Queue (DLQ) của RabbitMQ và Telegram Bot API được thực hiện như sau:



Hình 1.57: Sơ đồ vòng lặp thử lại (Retry Loop)



Hình 1.58: Sơ đồ xử lý DLQ và cảnh báo Telegram

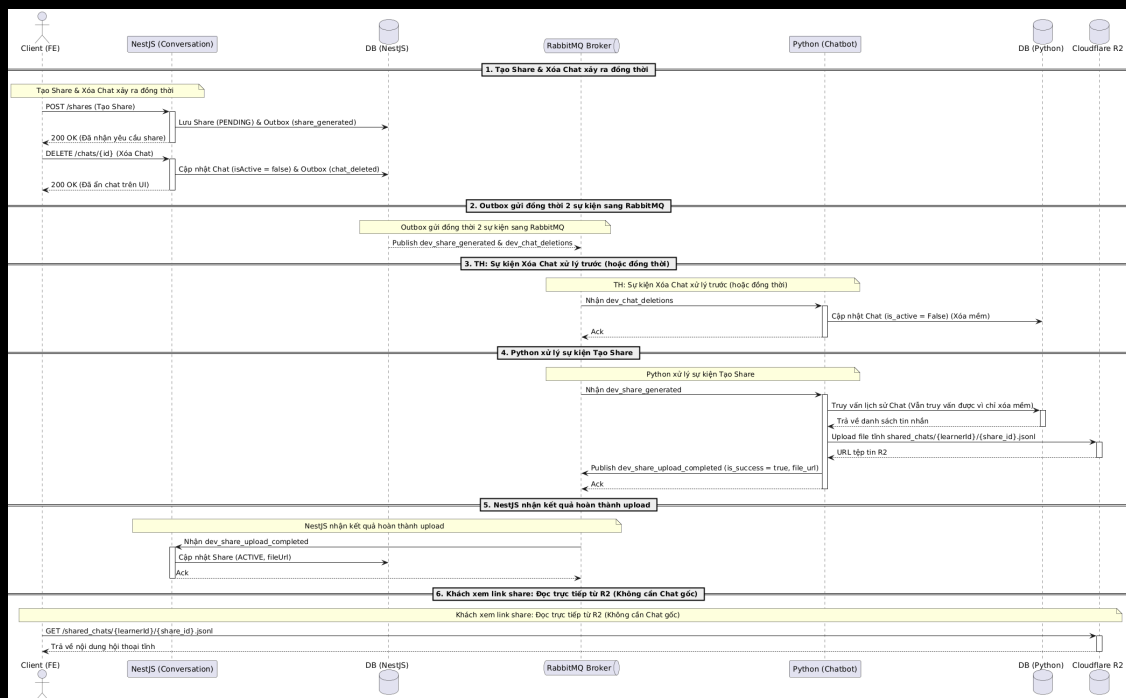
1. **Ghi nhận và Thử lại (Retry Loop):** Khi consumer (Python/NestJS) xử lý tin nhắn thất bại, tin nhắn sẽ được gửi sang hàng đợi Retry với độ trễ (TTL) cố định là 20 giây, đồng thời số lần thử lại (`x-retry-count`) được lưu và tăng dần trong header của tin nhắn.

- Chuyển sang DLQ:** Sau khi thử lại thất bại quá số lần quy định (`MAX_RETRY_COUNT = 5`), consumer sẽ từ chối tin nhắn không điều kiện (`message.reject(requeue=False)`). RabbitMQ sẽ tự động chuyển hướng tin nhắn này sang hàng đợi thư chết tương ứng (`_dlq`) dựa trên cấu hình `x-dead-letter-routing-key` lúc khởi tạo queue.
- Tiêu thụ DLQ và Cảnh báo:** Một consumer chuyên biệt (`process_share_generated_dlq`) sẽ lắng nghe hàng đợi DLQ. Khi có tin nhắn đi vào DLQ, consumer này sẽ trích xuất thông tin lỗi, payload của sự kiện, thời gian xảy ra sự cố và gửi yêu cầu HTTP POST tới Telegram Bot API để thông báo tức thời cho đội ngũ vận hành.

Xử lý concurrency khi Tạo Share và Xóa Chat đồng thời (Concurrency & Snapshot Architecture) Trong thực tế vận hành, người dùng có thể thực hiện thao tác tạo liên kết chia sẻ (Share) và ngay lập tức xóa cuộc trò chuyện (Delete Chat). Lúc này, cơ chế Outbox sẽ đẩy đồng thời hai sự kiện `share_generated` và `chat_deleted` vào RabbitMQ. Để giải quyết bài toán Race Condition (xung đột tài nguyên) khi thứ tự xử lý trên RabbitMQ không cố định, hệ thống áp dụng mẫu kiến trúc **Snapshot/Static Generation**:

Cơ chế hoạt động chi tiết:

- Không phụ thuộc vào trạng thái hoạt động của Chat:** Khi NestJS hoặc Python nhận sự kiện xóa chat, họ chỉ thực hiện **xóa mềm (soft delete)** bằng cách cập nhật trường `isActive = false/is_active = False` trong CSDL. Bản ghi và dữ liệu tin nhắn vẫn tồn tại vật lý trong database.
- Tạo Snapshot tĩnh (Static Generation):** Dù sự kiện `chat_deleted` có được xử lý trước sự kiện `share_generated` trên Python worker, Python vẫn có thể truy vấn đầy đủ nội dung lịch sử chat đã xóa mềm từ database để đóng gói thành tệp JSONL tĩnh và tải lên Cloudflare R2.
- Tính độc lập của liên kết chia sẻ:** Sau khi upload thành công lên Cloudflare R2, liên kết chia sẻ trở trực tiếp đến tệp tĩnh này. Khi người nhận xem liên kết chia sẻ, Frontend sẽ fetch trực tiếp tệp tĩnh từ R2 (qua CDN) mà không cần truy vấn vào database gốc (Neon) hay đi qua NestJS. Do đó, nghiệp vụ chấp nhận trạng thái "Có liên kết Share hoạt động nhưng Chat gốc đã bị xóa".



Hình 1.59: Sơ đồ xử lý đồng thời tạo share và xóa chat

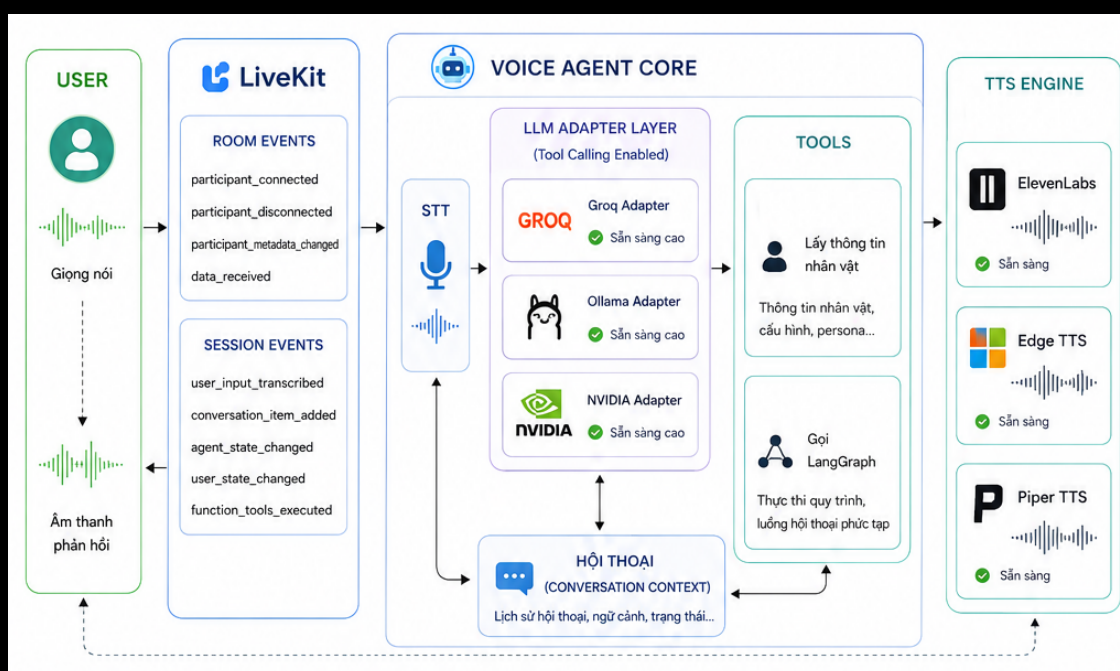
Kết quả các tệp tin chia sẻ trò chuyện lưu trữ trên Cloudflare R2:

[dev-share](#) / [shared_chats](#) / c36ce92d-704d-452a-883e-67a984645e4e /

☐ Objects	Type	Storage Class
☐ 01c2cadb-6450-4534-b173-f21190b8b7a0.jsonl	application/x-ndjson; cha...	Standard
☐ 01ee2872-36bb-4491-8c4f-283f09e5eee5.jsonl	application/x-ndjson; cha...	Standard
☐ 35945422-ccbf-4b91-8a0a-00fd1d5d5766.jsonl	application/x-ndjson; cha...	Standard
☐ 451ad427-7eb1-4d99-95be-578b0e57fc12.jsonl	application/x-ndjson; cha...	Standard
☐ 5811c0d9-d658-4e8b-9cdd-bf394ec3f10b.jsonl	application/x-ndjson; cha...	Standard
☐ 5c04c7a3-0ba8-44c6-83d6-9f7dafb1e91e.jsonl	application/x-ndjson; cha...	Standard

Hình 1.60: Lưu trữ các tệp tin chia sẻ trên Cloudflare R2

Chi tiết chức năng đàm thoại giọng nói (Voice Agent) Hệ thống thiết lập các bộ chuyển đổi (Adapters) cho phép kết nối linh hoạt với nhiều nhà cung cấp mô hình ngôn ngữ (Groq, Ollama và NVIDIA) có hỗ trợ gọi công cụ (Tool Calling) để đảm bảo tính sẵn sàng cao của hệ thống. AI Agent tích hợp sẵn công cụ lấy thông tin nhân vật (Get Persona) và công cụ gọi luồng xử lý LangGraph để phản hồi câu hỏi. Hệ thống hỗ trợ tự động chuyển đổi công cụ TTS giữa ElevenLabs, Edge TTS và Piper TTS để đáp ứng linh hoạt theo cấu hình nhân vật của người dùng. Sơ đồ tổng quan luồng tích hợp LiveKit đàm thoại giọng nói:



Hình 1.61: Sơ đồ tích hợp LiveKit đàm thoại giọng nói

Các sự kiện của LiveKit được hệ thống Voice Agent tiếp nhận và xử lý:

Sự kiện phòng (Room Events) Quản lý vòng đời kết nối và sự hiện diện của người dùng trong phòng đàm thoại:

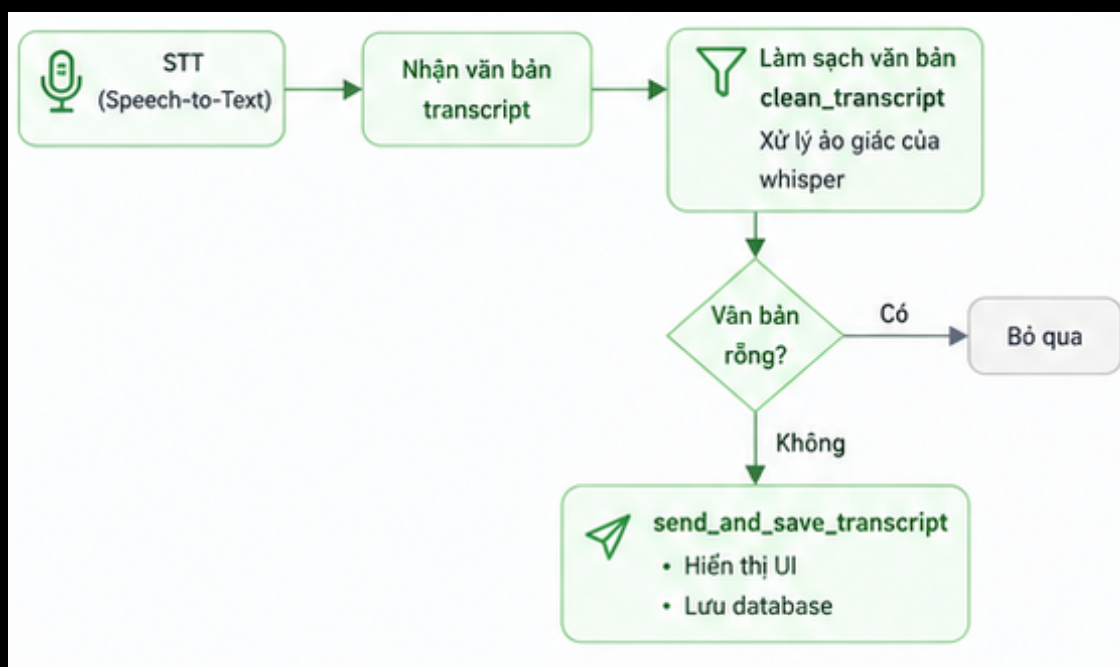
- **participant_connected:** Kích hoạt khi người dùng tham gia vào phòng đàm thoại thành công. Hệ thống tự động gọi hàm chủ động phát giọng nói chào hỏi người dùng (Lần đầu kết nối: WELCOME_MESSAGE = "Xin chào, tôi là trợ lý pháp luật. Bạn có cần tôi giúp đỡ gì không?", Các lần kết nối lại sau đó: REJOIN_MESSAGE = "Chào mừng bạn đã quay trở lại. Bạn có cần tôi giúp đỡ gì không?").

- **participant_disconnected**: Kích hoạt khi người dùng rời khỏi phòng hoặc bị ngắt kết nối đột ngột.
- **participant_metadata_changed**: Lắng nghe sự thay đổi siêu dữ liệu (metadata) từ phía người dùng để cập nhật cấu hình giọng đọc, nhân vật mới một cách tức thời.
- **data_received**: Nhận dữ liệu truyền qua kênh data channel (thường là tin nhắn văn bản định dạng JSON). Cho phép nhà phát triển (developer) gửi tin nhắn văn bản thay cho âm thanh nói để debug luồng xử lý của AI một cách thuận tiện.

Sự kiện phiên thoại (Session Events) Quản lý các sự kiện trong luồng đàm thoại giữa người dùng và AI Agent:

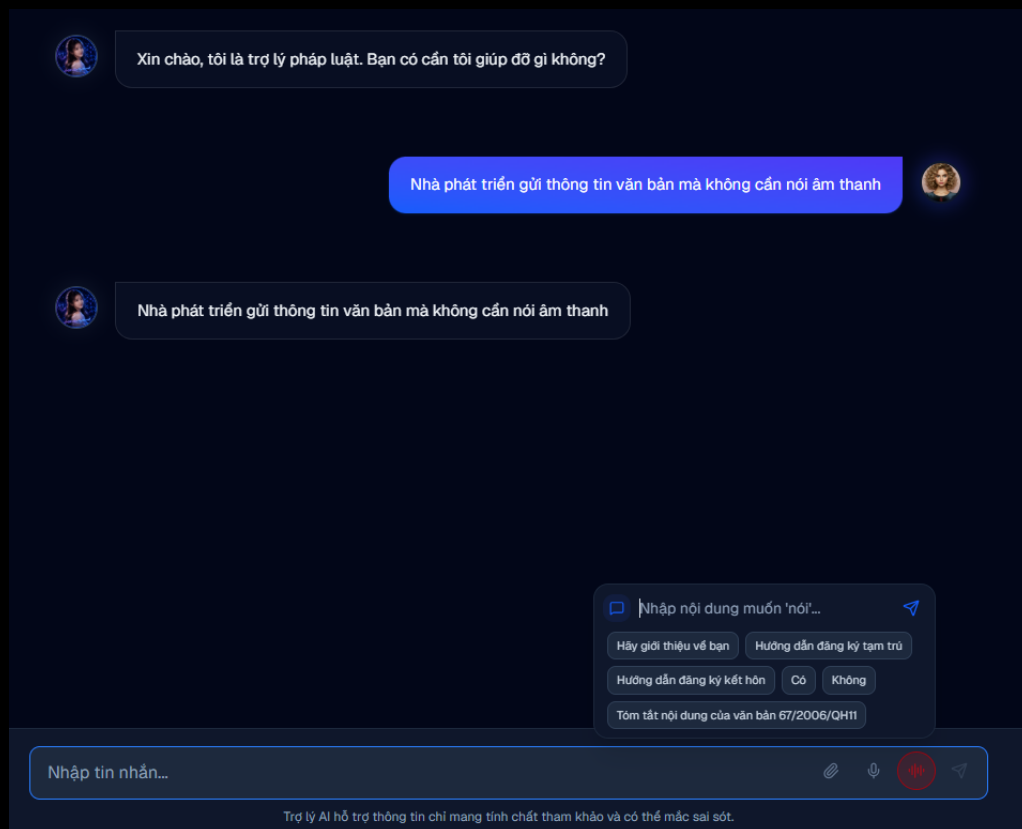
- **user_input_transcribed**: Nhận văn bản sau khi hệ thống Speech-to-Text (STT) chuyển đổi thành công giọng nói của người dùng. Áp dụng bộ lọc **clean_transcript** để loại bỏ các đoạn văn bản ảo giác của Whisper, bỏ qua nếu chuỗi rỗng và gọi **send_and_save_transcript** để hiển thị nội dung văn bản tương ứng lên giao diện UI và lưu vào CSDL.
- **conversation_item_added**: Kích hoạt mỗi khi có một tin nhắn mới (từ người dùng hoặc AI Assistant) được thêm vào ngữ cảnh trò chuyện. Đối với phản hồi từ AI Assistant, hệ thống tiến hành trích xuất nội dung tin nhắn, kiểm tra và lọc bỏ các bước suy luận trung gian của LangGraph. Khi có câu trả lời cuối cùng, hệ thống gọi **send_and_save_transcript** để hiển thị trên UI và lưu CSDL.
- **agent_state_changed** và **user_state_changed**: Theo dõi sự thay đổi trạng thái hoạt động (như Speaking, Listening, Thinking) của cả AI Agent và người dùng để đồng bộ hiển thị trạng thái động trên UI.
- **function_tools_executed**: Kích hoạt khi AI Agent quyết định gọi một công cụ bên ngoài (Function/Tool Calling) thành công.

Bộ lọc làm sạch văn bản **clean_transcript** xử lý loại bỏ ảo giác trong văn bản dịch từ giọng nói:

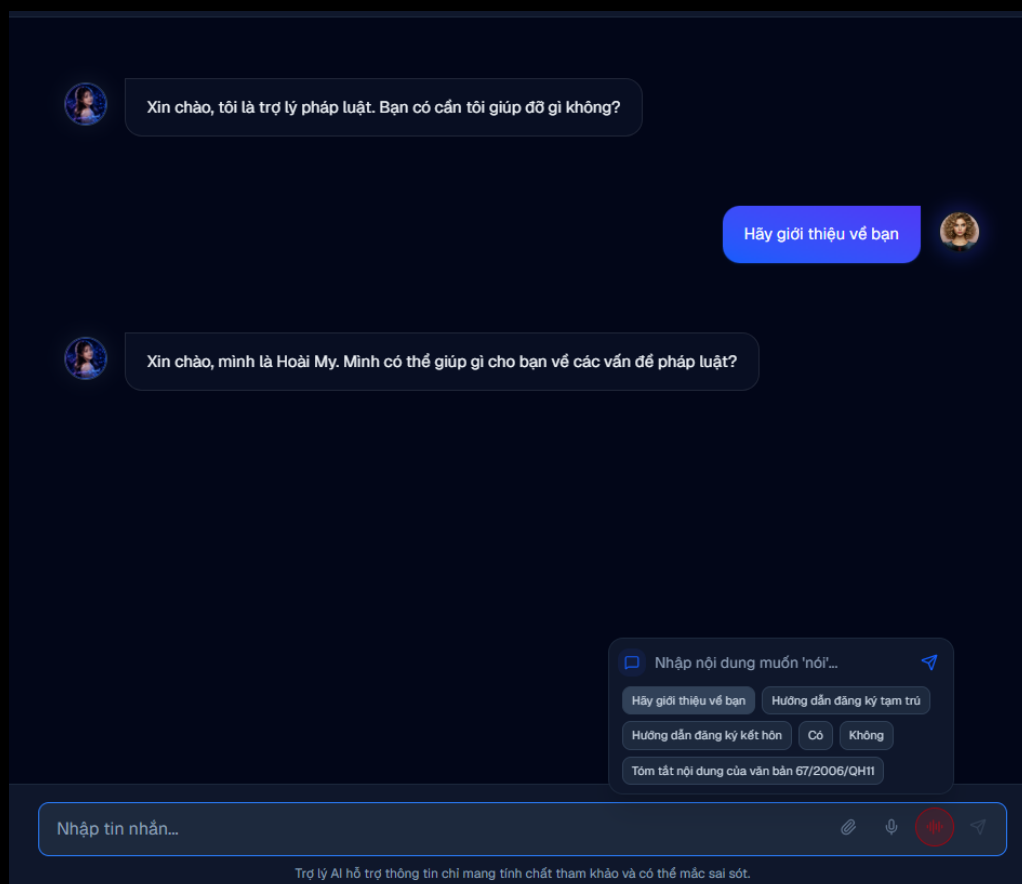


Hình 1.62: Bộ lọc làm sạch văn bản **clean_transcript**

Nhà phát triển gửi thông tin văn bản qua data channel mà không cần nói âm thanh:



Hình 1.63: Nhà phát triển gửi tin nhắn văn bản thay giọng nói để debug



Hình 1.64: Giao diện lấy thông tin nhân vật

Thông tin log terminal khi AI thực thi công cụ lấy thông tin nhân vật thành công:

```
2026-06-15 01:24:21.512 | DEBUG | app.voice.events.session.on_function_tools_executed:
on_function_tools_executed:5 - AI vừa sử dụng công cụ: type='function_tools_executed' function_calls=[FunctionCall(id='item_e82d4166f61c/fnc_0', type='function_call', call_id='fc_3c032004-6093-48e4-913f-62646d31a182', arguments='{}', name='get_persona_info', created_at=1781461461.1885478, extra={}, group_id=None)] function_call_outputs=[FunctionCallOutput(id='item_d388c3411560', type='function_call_output', name='get_persona_info', call_id='fc_3c032004-6093-48e4-913f-62646d31a182', output='Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?', is_error=False, created_at=1781461461.5105371)] created_at=1781461461.5115511
2026-06-15 01:24:23.007 - WARNING livekit.agents - no request_id provided for TTS livekit.plugins.openai.tts.TTS
2026-06-15 01:24:23.327 - WARNING livekit.agents - no request_id provided for TTS livekit.plugins.openai.tts.TTS
2026-06-15 01:24:28.977 - DEBUG livekit.agents - conversation_item_added {"role": "assistant", "text": "Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?"}
2026-06-15 01:24:28.977 | WARNING | app.voice.events.session.on_conversation_item_added:
on_conversation_item_added:45 - 🗣️ Agent (Final): Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp luật?
```

Hình 1.65: Terminal log của việc gọi công cụ lấy thông tin nhân vật

Cơ chế Lập lịch Dọn dẹp dữ liệu (Data Cleanup Scheduling) Vì tính năng xóa trò chuyện và thu hồi liên kết chia sẻ trong hệ thống sử dụng cơ chế **xóa mềm (soft delete)** để đảm bảo tính an toàn dữ liệu và tối ưu hiệu năng của các giao dịch tức thời, hệ thống triển khai các tác vụ lập lịch định kỳ (cron jobs / scheduler) trên cả **NestJS** và **Python** để dọn dẹp triệt để các dữ liệu đã bị xóa sau thời gian lưu trữ quy định.

Phía Dịch vụ trò chuyện (NestJS - code-conversation-service) NestJS sử dụng module `@nestjsjs/schedule` để quản lý các cron jobs dọn dẹp các bản ghi không còn hoạt động quá **30** ngày:

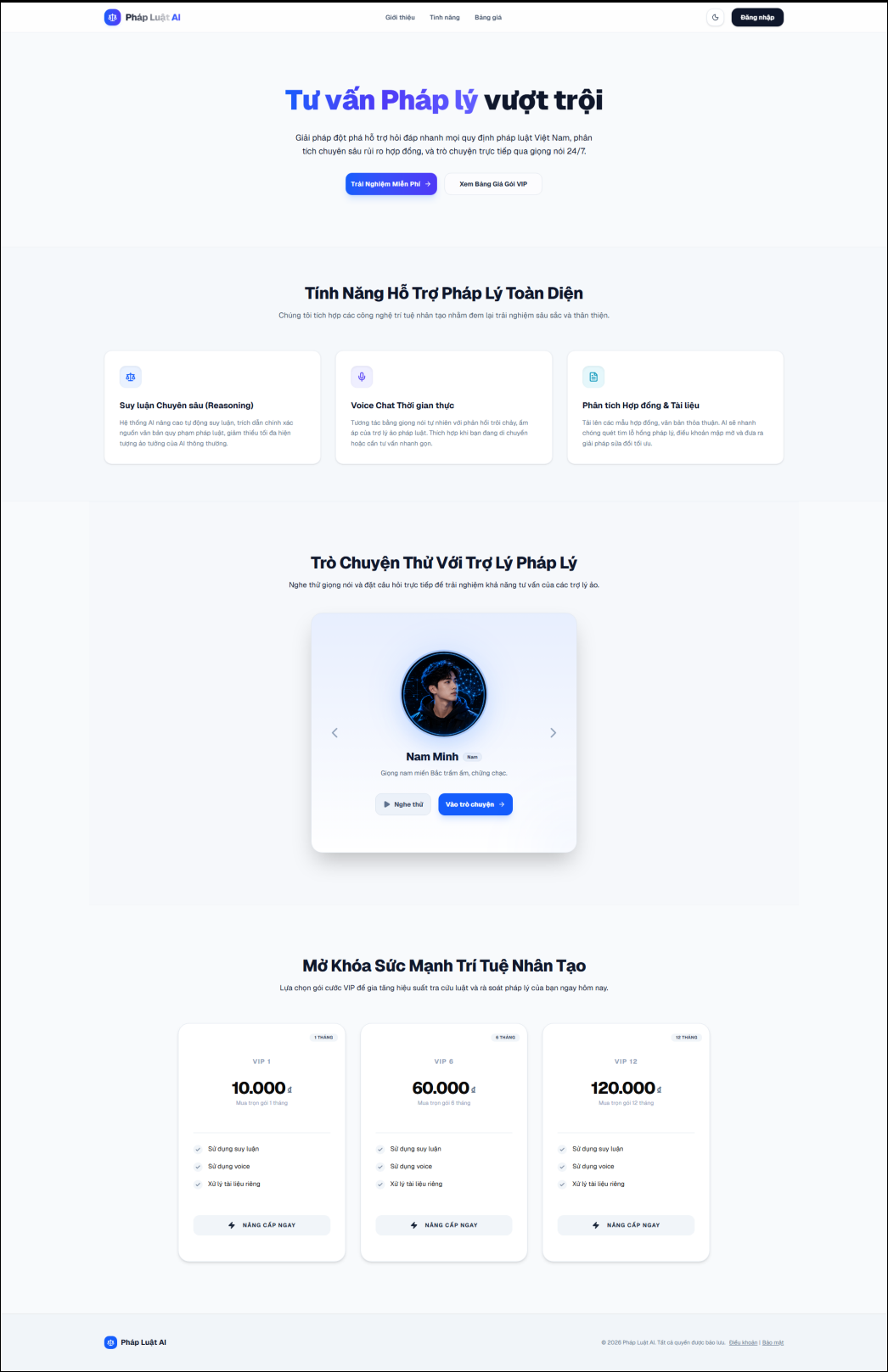
- **ChatCleanupCron:** Tìm và xóa vĩnh viễn các bản ghi Chat có `isActive = false` và thời gian cập nhật cách hiện tại từ 30 ngày trở lên. Chu kỳ chạy: Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **ShareCleanupCron:** Tìm và xóa vĩnh viễn các bản ghi liên kết Share có `isActive = false` (hoặc đã bị thu hồi) quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).

Phía Dịch vụ chatbot (Python - code-chatbot-service) Python sử dụng `APScheduler` (`AsyncIOScheduler`) chạy nền tích hợp trong vòng đời (lifespan) của ứng dụng để tự động dọn dẹp CSDL lưu trữ lịch sử:

- **run_chat_cleanup:** Xóa vĩnh viễn các bản ghi Chat trong CSDL chatbot có trạng thái `is_active = False` quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **run_share_cleanup:** Xóa vĩnh viễn các bản ghi Share trong CSDL chatbot có trạng thái `is_revoked = True` quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).

1.2 Giao diện chương trình

1.2.1 Người dùng khách và xác thực

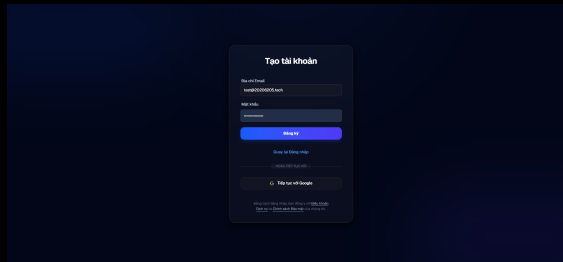


Hình 1.66: Trang chủ khách

Trang chủ khách

1. Người dùng truy cập trang chủ.

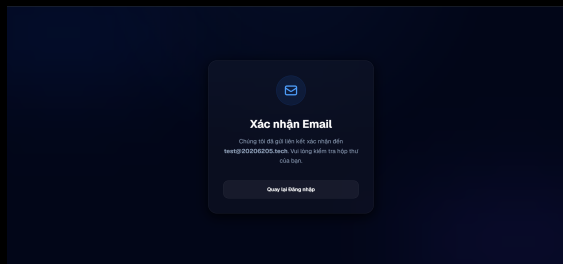
2. Xem phần giới thiệu, gói VIP và danh sách nhân vật.
3. Chọn đăng nhập hoặc đăng ký để bắt đầu sử dụng hệ thống.



Hình 1.67: Đăng ký tài khoản

Đăng ký tài khoản

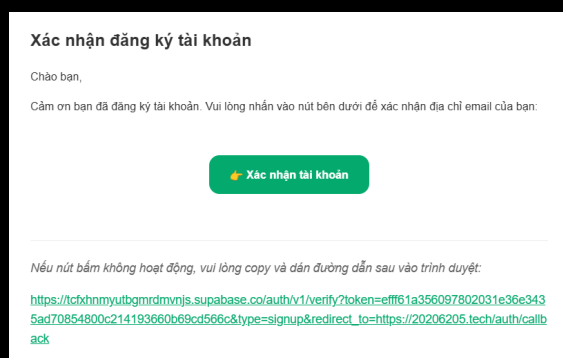
1. Nhập email và mật khẩu.
2. Bấm nút đăng ký.
3. Kiểm tra thông báo yêu cầu xác thực email.



Hình 1.68: Thông báo xác thực email

Thông báo xác thực email

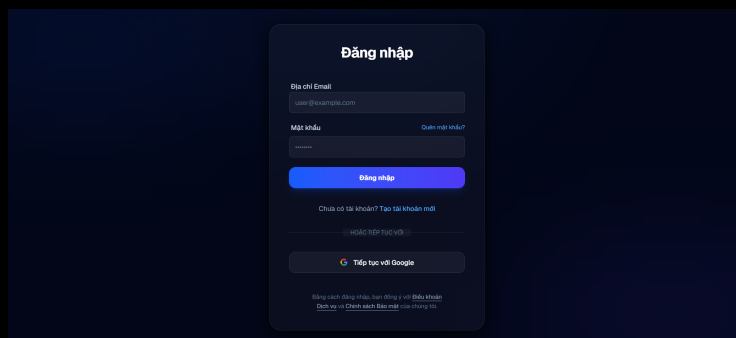
1. Sau khi đăng ký, hệ thống hiển thị trạng thái chờ xác thực.
2. Người dùng mở hộp thư email đã đăng ký.
3. Bấm liên kết xác thực để kích hoạt tài khoản.



Hình 1.69: Email xác thực đăng ký

Email xác thực đăng ký

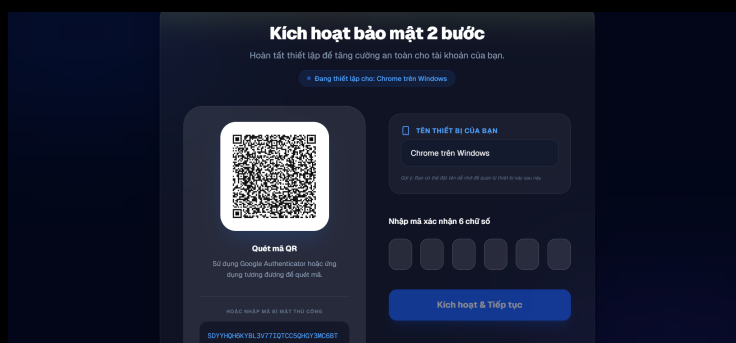
1. Mở email xác thực từ hệ thống.
2. Kiểm tra đúng địa chỉ email và nội dung xác thực.
3. Bấm liên kết xác nhận để quay lại hệ thống.



Hình 1.70: Đăng nhập

Đăng nhập

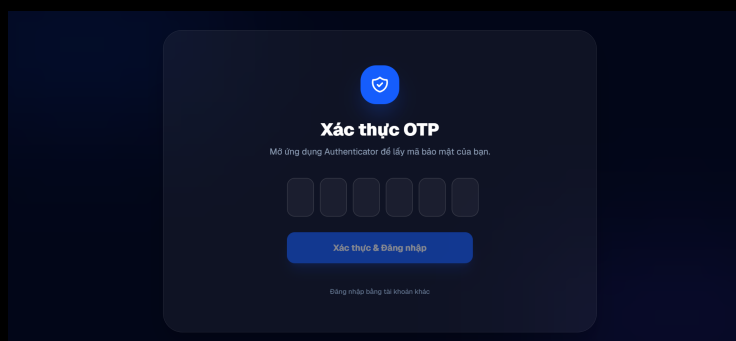
1. Nhập email và mật khẩu.
2. Bấm đăng nhập.
3. Nếu tài khoản bật MFA, hệ thống chuyển sang bước xác thực bổ sung.



Hình 1.71: Thiết lập MFA bằng mã QR

Thiết lập MFA bằng mã QR

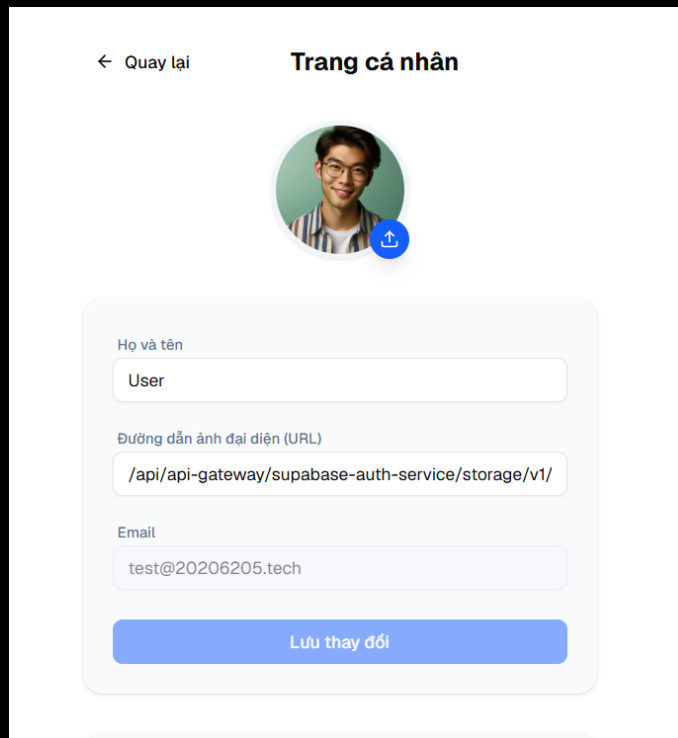
1. Mở ứng dụng xác thực trên điện thoại.
2. Quét mã QR hiển thị trên màn hình.
3. Lưu cấu hình MFA trong ứng dụng xác thực.



Hình 1.72: Xác minh mã MFA

Xác minh mã MFA

1. Lấy mã TOTP mới nhất từ ứng dụng xác thực.
2. Nhập mã TOTP vào màn hình xác minh.
3. Bấm xác nhận để hoàn tất đăng nhập hoặc kích hoạt MFA.



The screenshot shows a user profile page titled "Trang cá nhân". At the top left is a back arrow and the text "Quay lại". Below the title is a circular profile picture of a man with glasses, with a blue upload icon in the bottom right corner. The profile information is contained in a light blue rounded rectangle with the following fields:

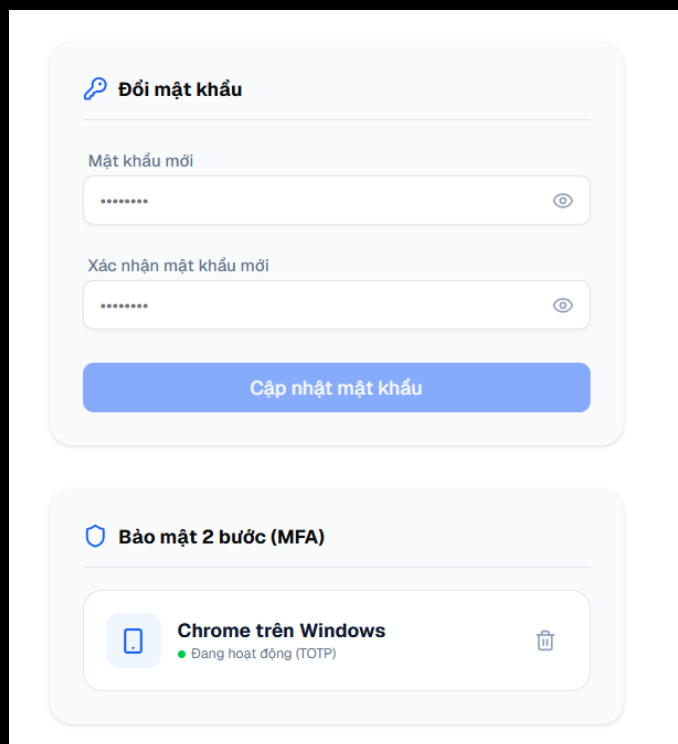
- Họ và tên** (Full Name): User
- Đường dẫn ảnh đại diện (URL)** (Profile Picture URL): /api/api-gateway/supabase-auth-service/storage/v1/
- Email**: test@20206205.tech

At the bottom of the form is a blue button labeled "Lưu thay đổi" (Save changes).

Hình 1.73: Hồ sơ người dùng

Hồ sơ người dùng

1. Mở trang hồ sơ cá nhân.
2. Xem hoặc cập nhật thông tin hiển thị.
3. Lưu thay đổi để hệ thống cập nhật hồ sơ.



The screenshot shows two sections of a security settings page:

Đổi mật khẩu (Change Password):

- Mật khẩu mới** (New Password): A text input field with a masked password "*****" and a toggle icon.
- Xác nhận mật khẩu mới** (Confirm new password): A text input field with a masked password "*****" and a toggle icon.
- A blue button labeled "Cập nhật mật khẩu" (Update password).

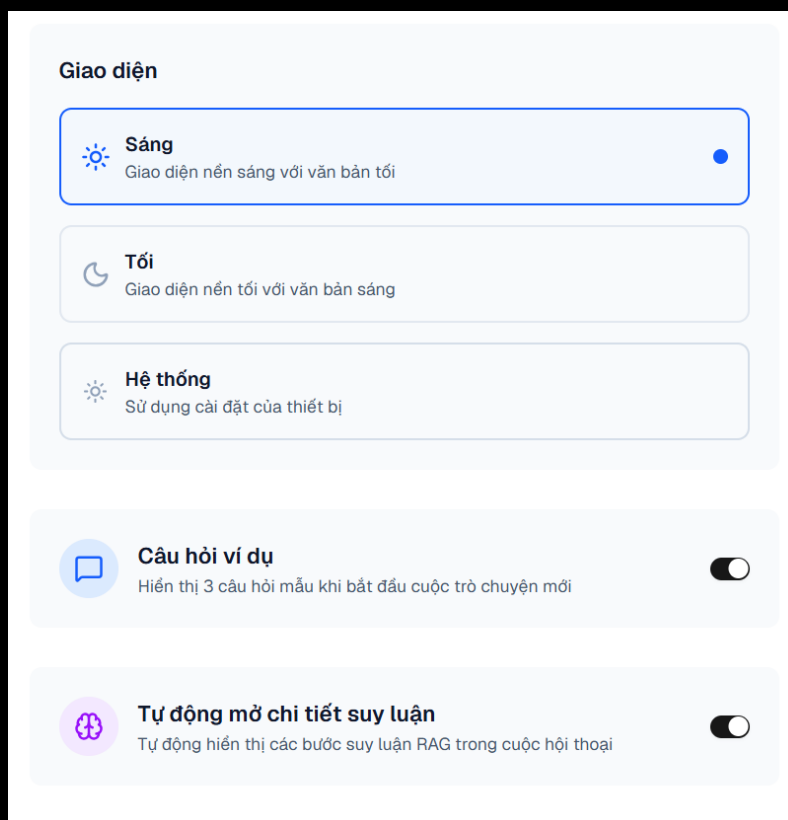
Bảo mật 2 bước (MFA) (Two-step security):

- A section titled "Chrome trên Windows" (Chrome on Windows) with a green status indicator "Đang hoạt động (TOTP)" (Active (TOTP)) and a trash icon.

Hình 1.74: Đổi mật khẩu và MFA

Đổi mật khẩu và MFA

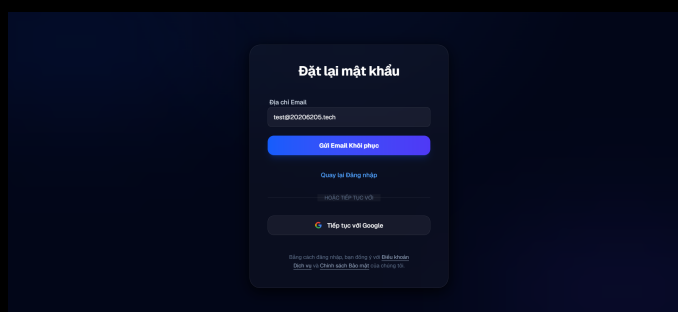
1. Mở phần bảo mật tài khoản.
2. Đổi mật khẩu hoặc quản lý xác thực MFA.
3. Xác nhận thao tác theo yêu cầu của hệ thống.



Hình 1.75: Cài đặt người dùng

Cài đặt người dùng

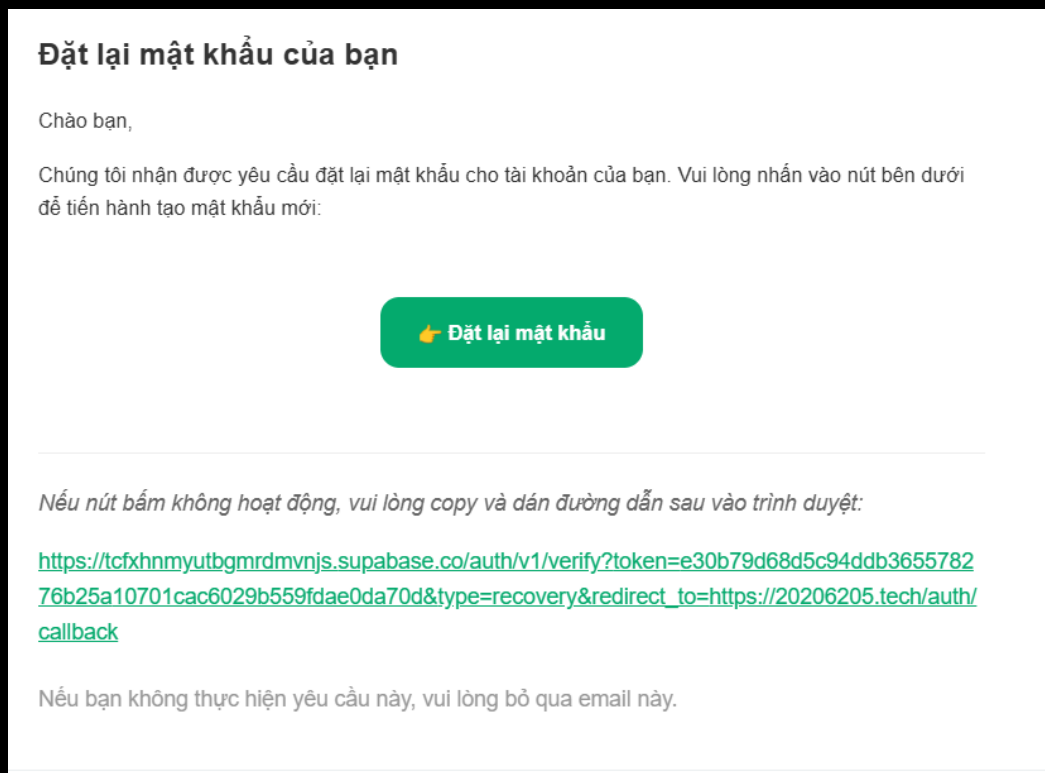
1. Mở trang cài đặt.
2. Điều chỉnh các tùy chọn cá nhân như giao diện, giọng nói hoặc cấu hình trò chuyện.
3. Bấm lưu để áp dụng cấu hình.



Hình 1.76: Khôi phục mật khẩu

Khôi phục mật khẩu

1. Nhập email đã đăng ký.
2. Bấm gửi yêu cầu khôi phục.
3. Mở email khôi phục mật khẩu và làm theo hướng dẫn.

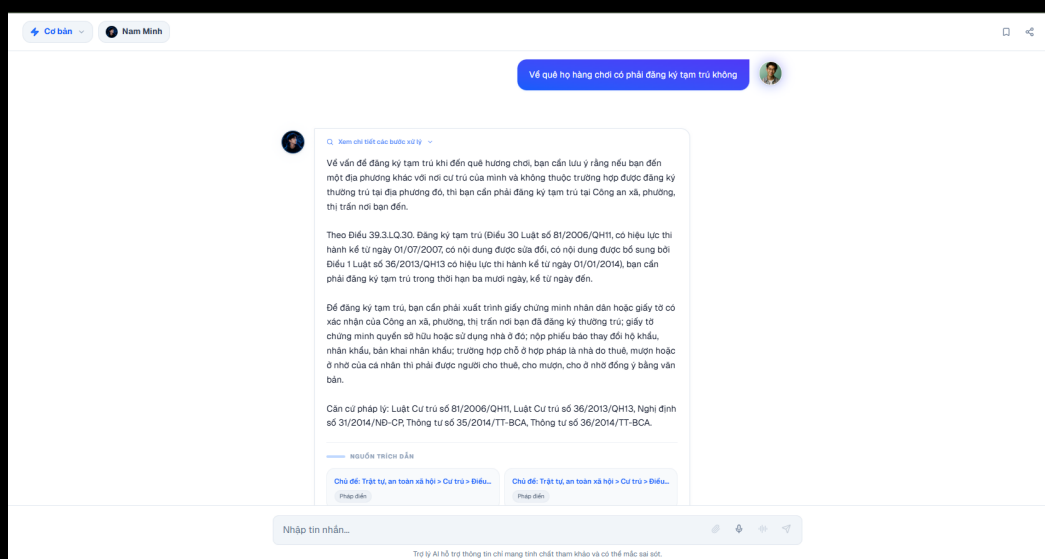


Hình 1.77: Email khôi phục mật khẩu

Email khôi phục mật khẩu

1. Mở email khôi phục mật khẩu.
2. Bấm liên kết đặt lại mật khẩu.
3. Nhập mật khẩu mới trên màn hình được chuyển hướng.

1.2.2 Trò chuyện, ghi chú và chia sẻ

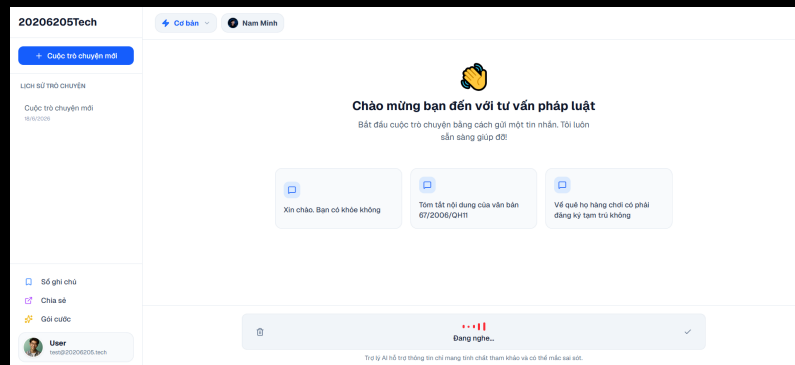


Hình 1.78: Trò chuyện văn bản

Trò chuyện văn bản

1. Nhập câu hỏi vào khung trò chuyện.

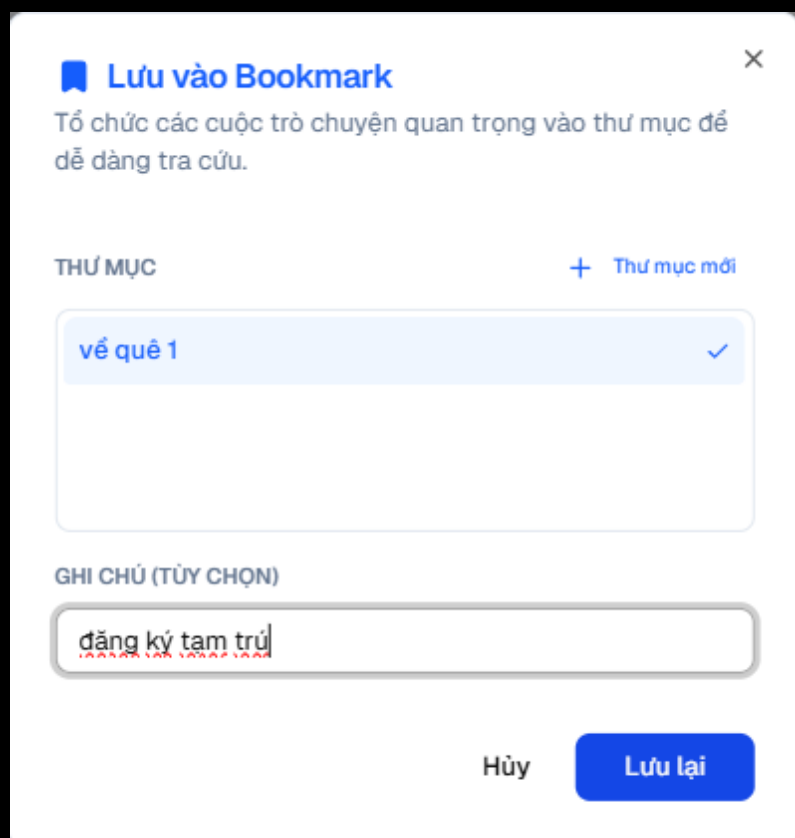
2. Chọn tùy chọn bổ sung nếu cần, ví dụ suy luận hoặc tài liệu đính kèm.
3. Gửi câu hỏi và theo dõi phản hồi dạng streaming.



Hình 1.79: Trò chuyện bằng micro

Trò chuyện bằng micro

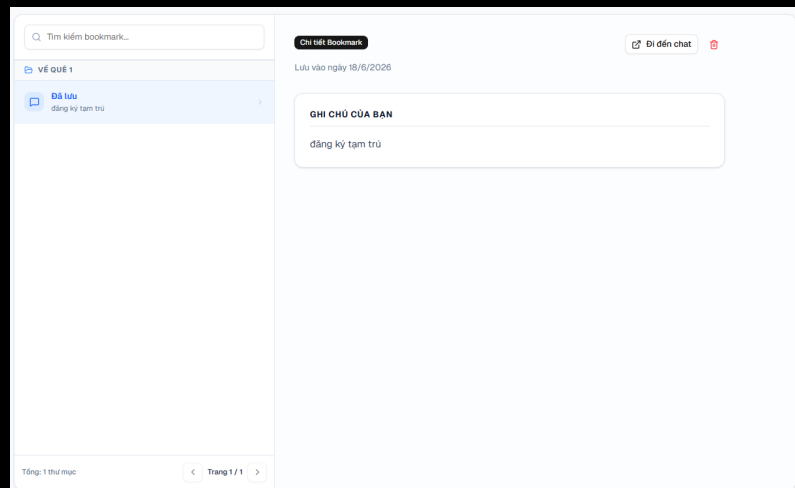
1. Bấm biểu tượng micro để bắt đầu nhập bằng giọng nói.
2. Cho phép trình duyệt truy cập micro nếu được hỏi.
3. Nói câu hỏi và chờ hệ thống xử lý.



Hình 1.80: Tạo ghi chú đánh dấu

Tạo ghi chú đánh dấu

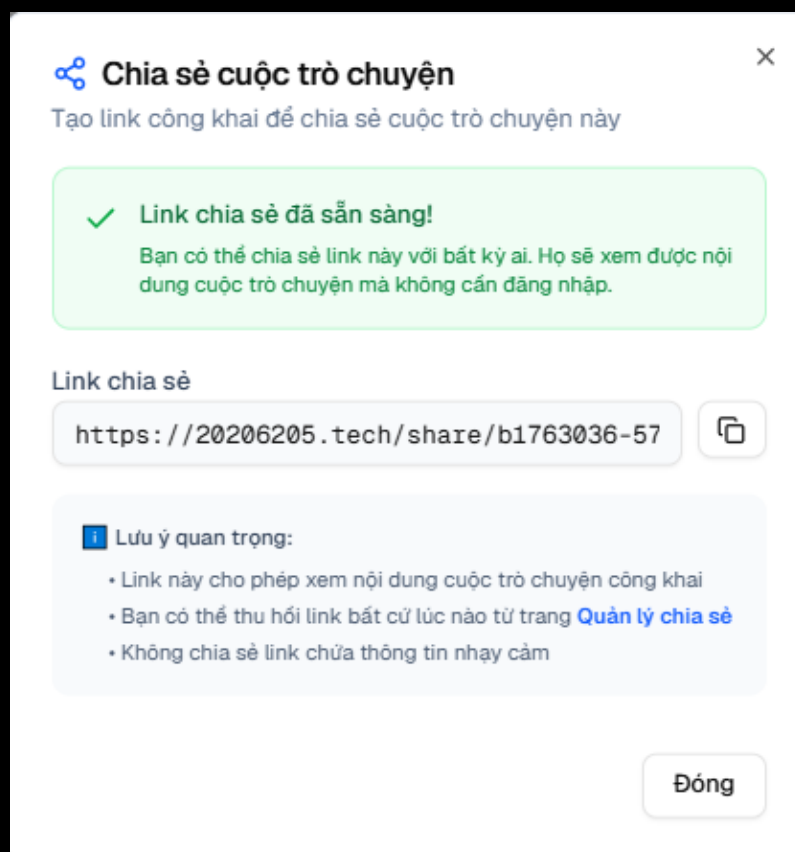
1. Chọn cuộc trò chuyện cần lưu.
2. Chọn hoặc tạo thư mục đánh dấu.
3. Nhập ghi chú và bấm lưu.



Hình 1.81: Quản lý ghi chú đánh dấu

Quản lý ghi chú đánh dấu

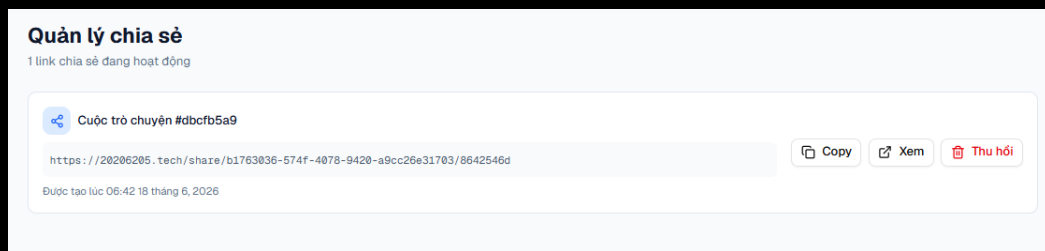
1. Mở danh sách đánh dấu.
2. Tìm cuộc trò chuyện hoặc thư mục cần quản lý.
3. Chỉnh sửa ghi chú, đổi thư mục hoặc mở lại cuộc trò chuyện.



Hình 1.82: Tạo liên kết chia sẻ

Tạo liên kết chia sẻ

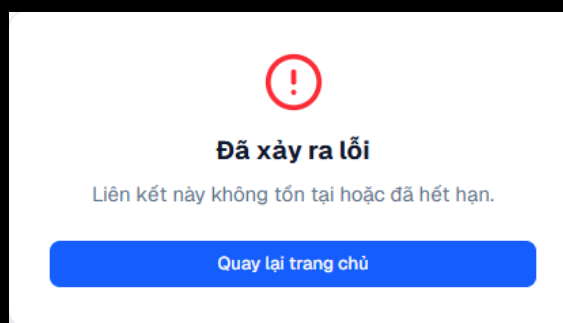
1. Mở cuộc trò chuyện cần chia sẻ.
2. Bấm tạo liên kết chia sẻ.
3. Sao chép liên kết sau khi hệ thống tạo xong.



Hình 1.83: Quản lý chia sẻ

Quản lý chia sẻ

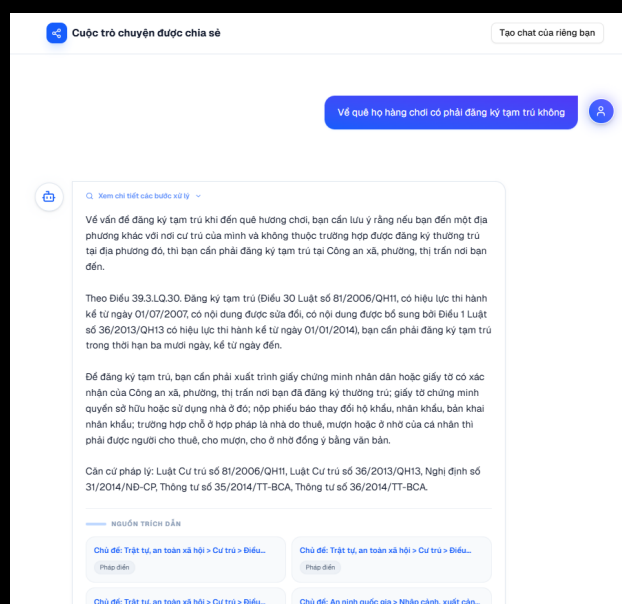
1. Mở danh sách liên kết chia sẻ.
2. Kiểm tra trạng thái từng liên kết.
3. Thu hồi liên kết nếu không muốn tiếp tục chia sẻ.



Hình 1.84: Liên kết chia sẻ không khả dụng

Liên kết chia sẻ không khả dụng

1. Người nhận mở liên kết chia sẻ.
2. Nếu liên kết đã bị thu hồi hoặc không tồn tại, hệ thống hiển thị thông báo không khả dụng.
3. Liên hệ người chia sẻ để nhận liên kết mới nếu cần.



Hình 1.85: Liên kết chia sẻ khả dụng

Liên kết chia sẻ khả dụng

1. Mở liên kết chia sẻ hợp lệ.
2. Xem nội dung trò chuyện được chia sẻ.
3. Dùng nội dung này để tham khảo mà không cần quyền chỉnh sửa cuộc trò chuyện gốc.



Hình 1.86: Thu hồi chia sẻ

Thu hồi chia sẻ

1. Chọn liên kết chia sẻ cần thu hồi.
2. Bấm thu hồi hoặc hủy chia sẻ.
3. Xác nhận thao tác để liên kết không còn truy cập được.



Hình 1.87: Xóa cuộc trò chuyện

Xóa cuộc trò chuyện

1. Chọn cuộc trò chuyện cần xóa.
2. Bấm thao tác xóa.
3. Xác nhận để hệ thống xóa hoặc ẩn cuộc trò chuyện khỏi danh sách.

1.2.3 Quản trị hệ thống

Quản lý gói cước					
Quản lý các gói đăng ký					
+ Thêm gói mới					
Tên gói	Thời hạn	Giá	Tính năng	Trạng thái	Thao tác
VIP 1	1 tháng	100.000 đ	- Sử dụng suy luận - Sử dụng voice - Xử lý tài liệu riêng	Đang bật	
VIP 6	6 tháng	550.000 đ	- Sử dụng suy luận - Sử dụng voice - Xử lý tài liệu riêng	Đang bật	
VIP 12	12 tháng	1.000.000 đ	- Sử dụng suy luận - Sử dụng voice - Xử lý tài liệu riêng	Đang bật	

Hình 1.88: Quản lý gói VIP

Quản lý gói VIP

1. Quản trị viên mở trang quản lý gói VIP.
2. Xem danh sách gói hiện có.
3. Tạo mới, cập nhật hoặc vô hiệu hóa gói theo nhu cầu.

Quản lý nhân vật						
Quản lý các nhân vật cho hệ thống						
+ Thêm nhân vật						
Avatar	Tên nhân vật	Giới tính	Engine	Voice ID	Trạng thái	Thao tác
	Nam Minh Giọng nam trầm ấm, Bắc trâm âm, chữ...	Nam	EDGE_TTS	v1-VN-NamMinhNeura1	Đang hoạt động	✎ 🗑️
	Hoài My Giọng nữ miền Nam nhẹ nhàng, trù...	Nữ	EDGE_TTS	v1-VN-HoaiMyNeura1	Đang hoạt động	✎ 🗑️
	Quốc Hùng Giọng nam trầm ấm, tự tin và năng ...	Nam	ELEVENLABS	IKne3meq5a5n9XLyUoCD	Đang hoạt động	✎ 🗑️
	Đức Kiên Giọng nam chắc chắn, uy quyền v...	Nam	ELEVENLABS	pNIInz6obpgDQGcFmaJgB	Đang hoạt động	✎ 🗑️
	Lan Anh Giọng nữ trưởng thành, đáng tin cậ...	Nữ	ELEVENLABS	EXAVITQu4vz4xnSDxMaL	Đang hoạt động	✎ 🗑️
	Thanh Trúc Giọng nữ chuyên nghiệp, tươi sáng ...	Nữ	ELEVENLABS	hpp4J3VqNfWUAU00d1Us	Đang hoạt động	✎ 🗑️
	Hùng Dũng Giọng nam trầm ấm, truyền cảm.	Nam	ELEVENLABS	cjVigY5qz086Huf00Wa1	Đang hoạt động	✎ 🗑️
	Ban Mai Giọng nữ nhẹ nhàng, trong trẻo.	Nữ	PIPER_TTS	banma1	Đang hoạt động	✎ 🗑️
	Thanh Phương Giọng nữ truyền cảm, ấm áp.	Nữ	PIPER_TTS	yannew	Đang hoạt động	✎ 🗑️
	Minh Quang Giọng nam trẻ trung, năng động.	Nam	PIPER_TTS	minhquang	Đang hoạt động	✎ 🗑️
Hiển thị 1 đến 10 trong tổng số 11 nhân vật						
Trang đầu < Trước Trang 1 / 2 Sau > Trang cuối						

Hình 1.89: Quản lý nhân vật

Quản lý nhân vật

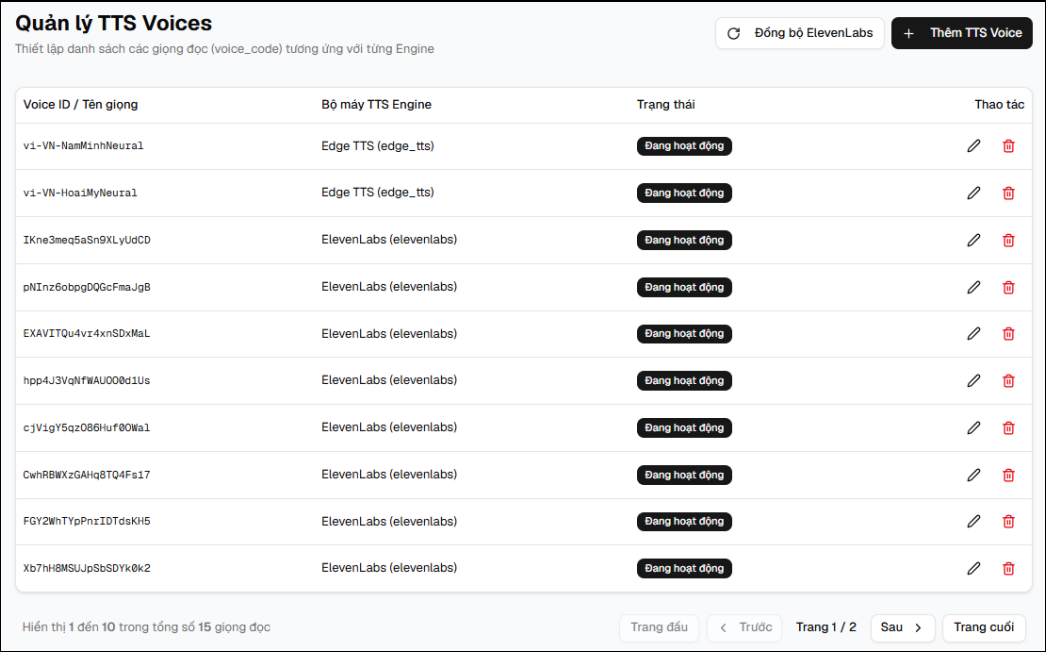
1. Quản trị viên mở trang quản lý nhân vật.
2. Nhập thông tin nhân vật.
3. Lưu để nhân vật xuất hiện trong hệ thống.

Quản lý TTS Engines			
Cấu hình các bộ máy chuyển đổi văn bản thành giọng nói (TTS)			
+ Thêm TTS Engine			
Mã Engine	Tên hiển thị	Trạng thái	Thao tác
edge_tts	Edge TTS	Đang hoạt động	✎ 🗑️
elevenlabs	ElevenLabs	Đang hoạt động	✎ 🗑️
piiper_tts	Piper TTS	Đang hoạt động	✎ 🗑️

Hình 1.90: Quản lý công cụ dịch vụ giọng nói

Quản lý công cụ dịch vụ giọng nói

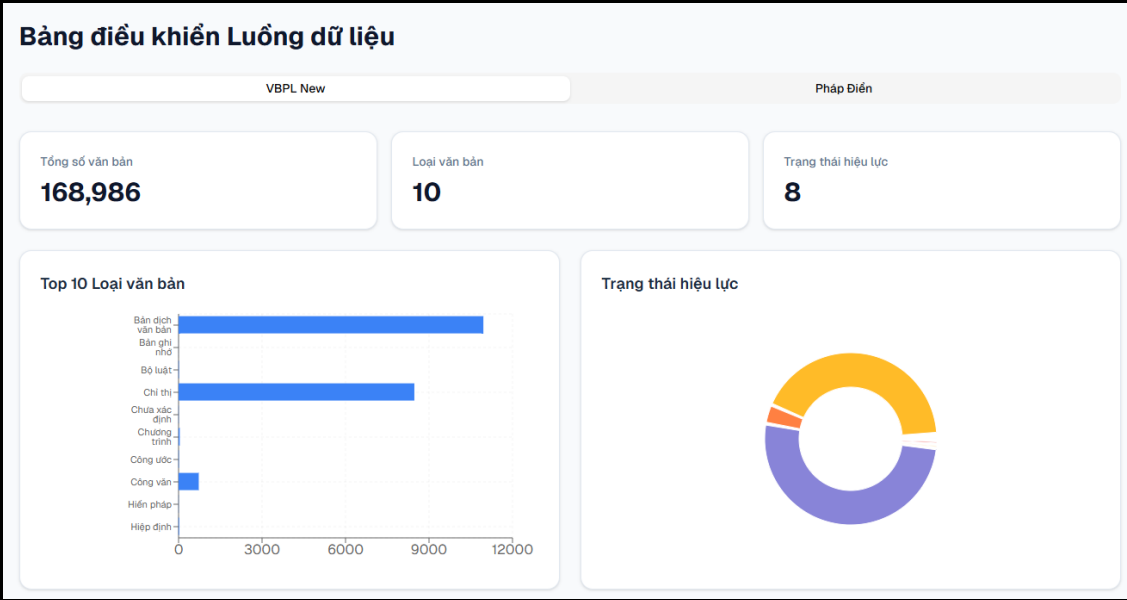
1. Quản trị viên mở trang cấu hình công cụ dịch vụ giọng nói.
2. Kiểm tra trạng thái các công cụ dịch vụ giọng nói đang hỗ trợ.
3. Bật, tắt hoặc cập nhật thông tin công cụ dịch vụ giọng nói.



Hình 1.91: Quản lý giọng nói

Quản lý giọng nói

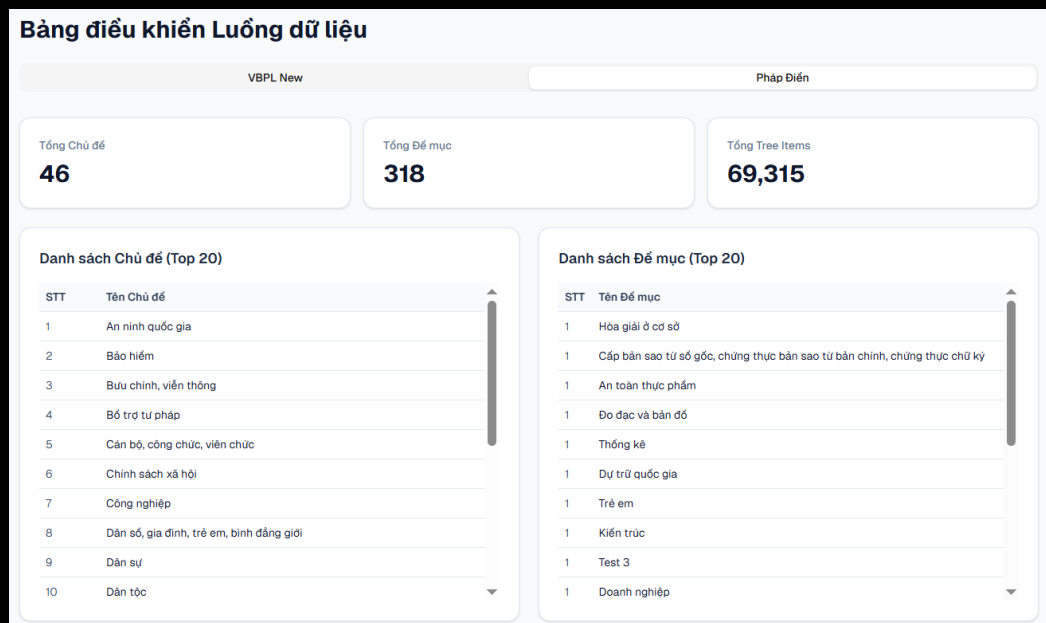
- 1. Quản trị viên mở trang giọng nói.
- 2. Đồng bộ hoặc cập nhật danh sách giọng nói.
- 3. Gắn giọng nói phù hợp cho nhân vật AI.



Hình 1.92: Đường ống dữ liệu VBPL

Đường ống dữ liệu VBPL

- 1. Quản trị viên mở trang đường ống dữ liệu VBPL.
- 2. Theo dõi trạng thái xử lý dữ liệu.
- 3. Kiểm tra tiến độ hoặc lỗi nếu có.

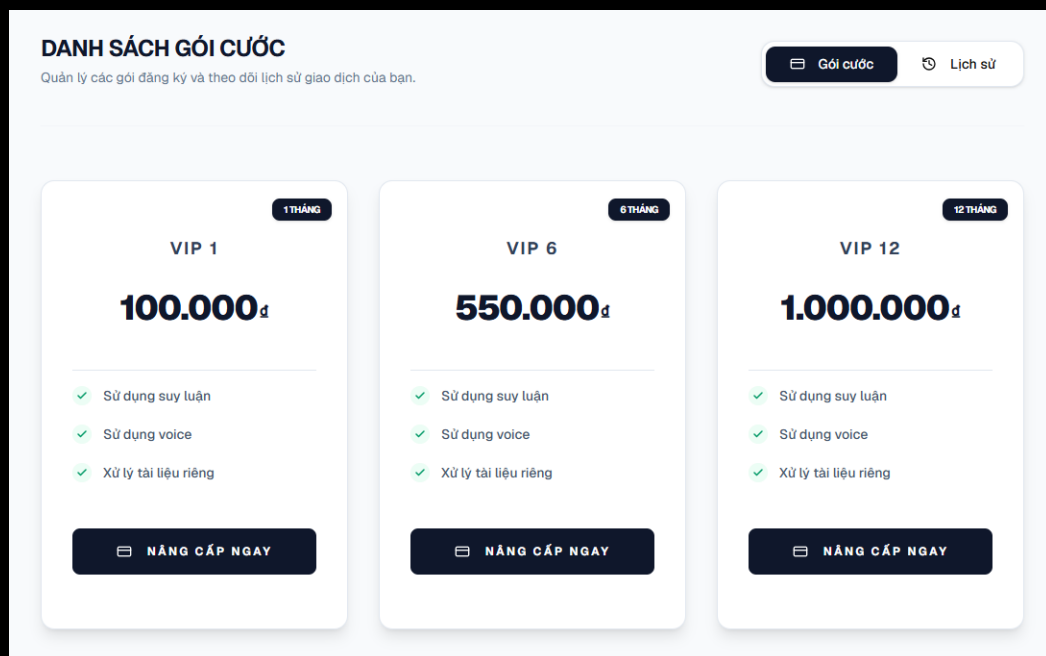


Hình 1.93: Đường ống dữ liệu Pháp Điển

Đường ống dữ liệu Pháp Điển

1. Quản trị viên mở trang đường ống Pháp Điển.
2. Xem thống kê cấu trúc và tiến độ xử lý.
3. Dùng thông tin này để theo dõi chất lượng dữ liệu nền.

1.2.4 Thanh toán và gói VIP



Hình 1.94: Danh sách gói thanh toán

Danh sách gói thanh toán

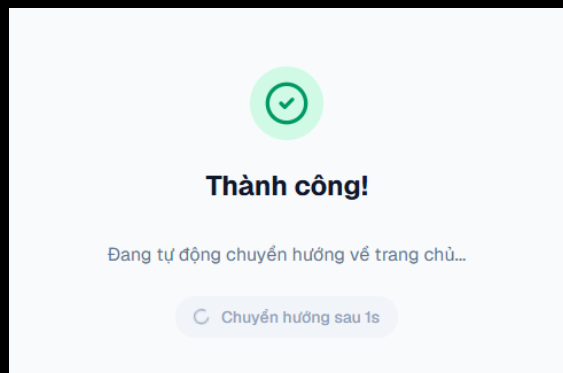
1. Người dùng mở trang gói VIP.
2. So sánh quyền lợi, giá và thời hạn từng gói.
3. Chọn gói phù hợp để tiến hành thanh toán.

DANH SÁCH GÓI CƯỚC			
Quản lý các gói đăng ký và theo dõi lịch sử giao dịch của bạn.			
		Gói cước	Lịch sử
Ngày mua	Gói cước	Số tiền	Trạng thái
16/06/2026 11:20	Gói cước	10.000 đ	Hết hạn

Hình 1.95: Lịch sử thanh toán

Lịch sử thanh toán

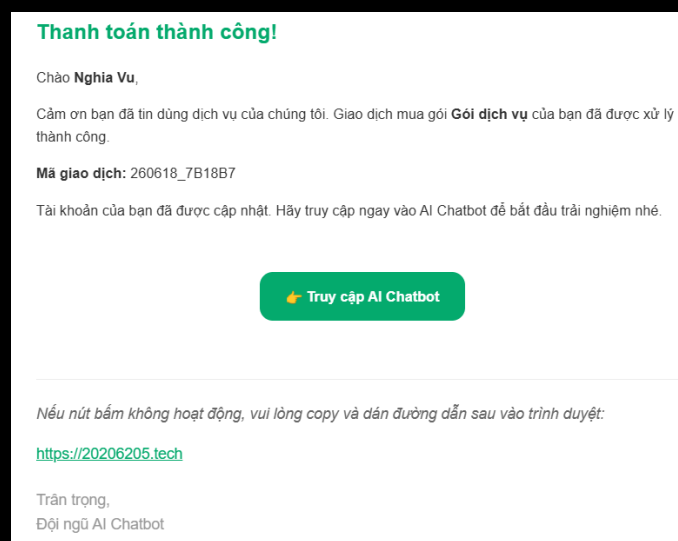
1. Mở trang lịch sử giao dịch.
2. Kiểm tra trạng thái, thời gian và gói đã thanh toán.
3. Dùng thông tin này để đối soát giao dịch khi cần.



Hình 1.96: Thanh toán thành công

Thanh toán thành công

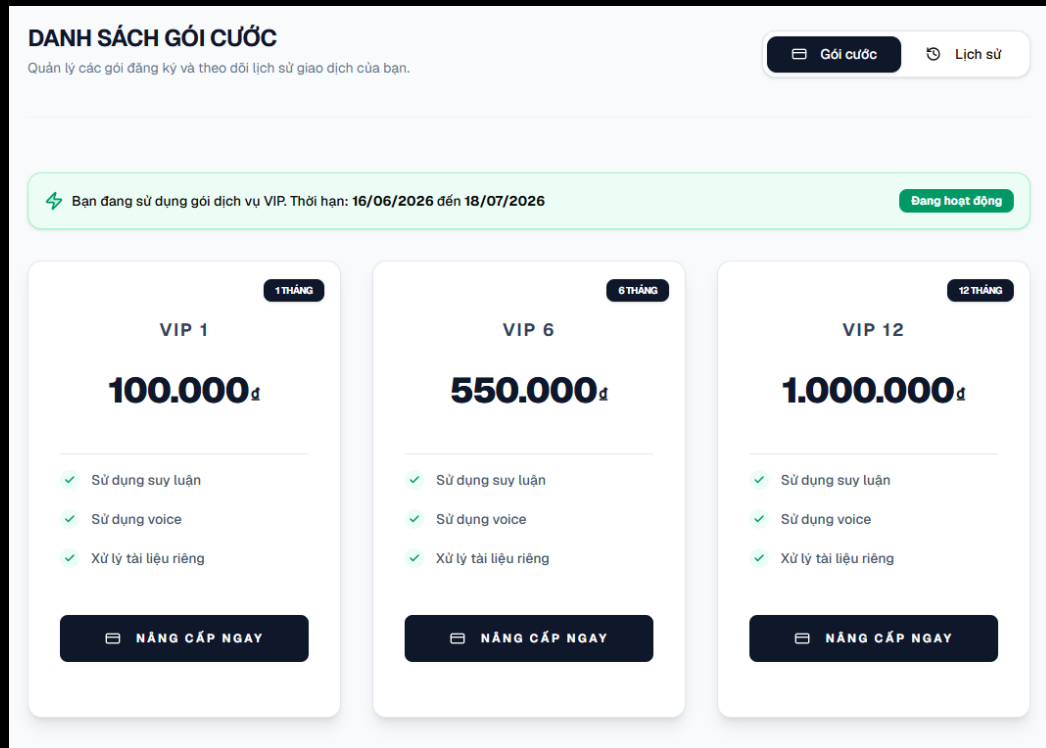
1. Sau khi thanh toán, hệ thống hiển thị kết quả thành công.
2. Kiểm tra thông tin gói VIP vừa kích hoạt.
3. Quay lại hệ thống để sử dụng các tính năng VIP.



Hình 1.97: Email thanh toán thành công

Email thanh toán thành công

1. Mở email thông báo thanh toán.
2. Kiểm tra thông tin gói, thời hạn và trạng thái giao dịch.
3. Lưu email để đối chiếu khi cần hỗ trợ.

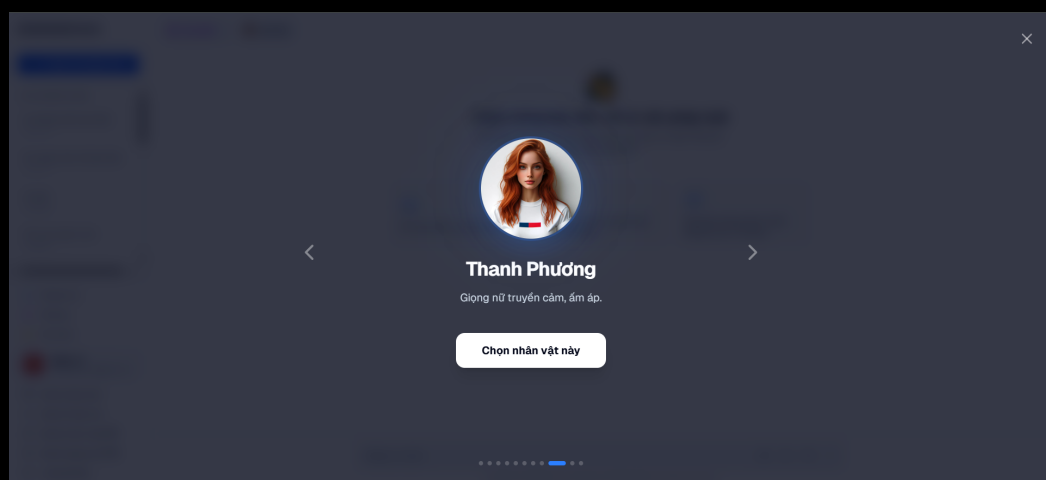


Hình 1.98: Gói VIP đang hoạt động

Gói VIP đang hoạt động

1. Mở trang gói VIP sau khi thanh toán.
2. Kiểm tra dấu hiệu gói đang hoạt động.
3. Sử dụng các tính năng yêu cầu VIP như suy luận, trò chuyện bằng giọng nói hoặc tài liệu cá nhân.

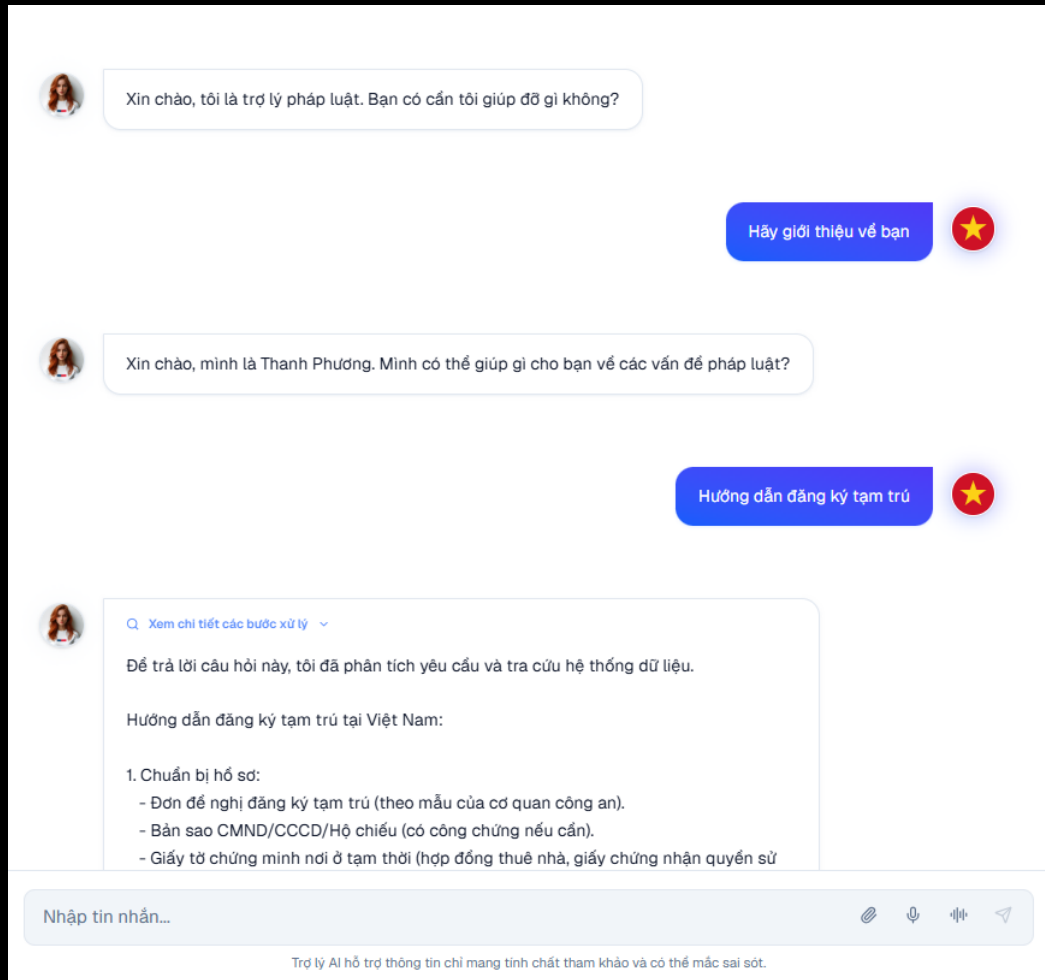
1.2.5 Trò chuyện bằng giọng nói



Hình 1.99: Chọn giọng nói

Chọn giọng nói

1. Mở phần chọn giọng nói hoặc nhân vật.
2. Nghe thử hoặc xem thông tin từng giọng.
3. Chọn giọng phù hợp cho phiên trò chuyện.

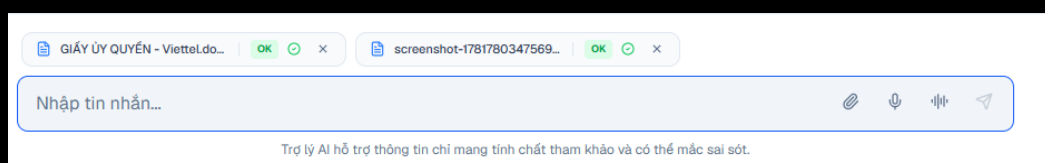


Hình 1.100: Trò chuyện bằng giọng nói

Trò chuyện bằng giọng nói

1. Bắt đầu phiên trò chuyện bằng giọng nói.
2. Nói câu hỏi hoặc yêu cầu vào micro.
3. Nghe phản hồi bằng giọng nói và theo dõi trạng thái xử lý trên giao diện.

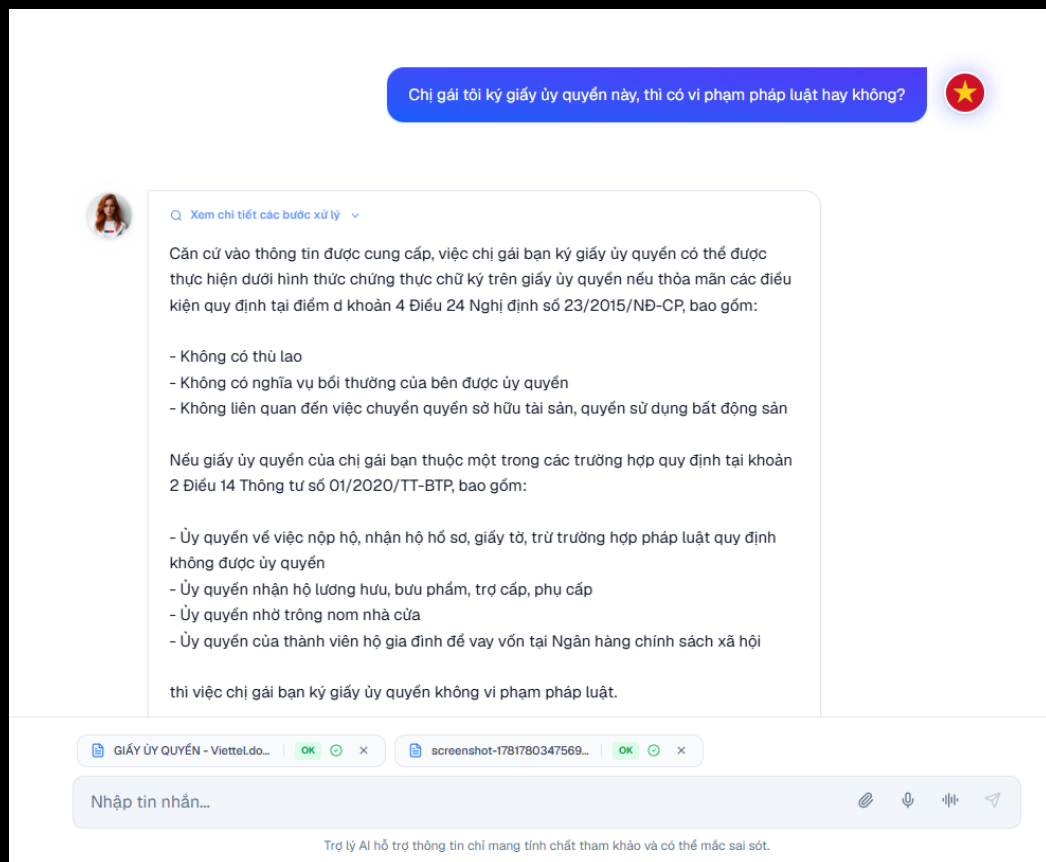
1.2.6 Tải lên tài liệu cá nhân



Hình 1.101: Tải lên tài liệu

Tải lên tài liệu

1. Chọn file tài liệu từ thiết bị.
2. Bấm tải lên.
3. Chờ hệ thống xử lý nền trước khi dùng tài liệu trong trò chuyện.



Hình 1.102: Trò chuyện với tài liệu

Trò chuyện với tài liệu

1. Chọn tài liệu đã xử lý thành công.
2. Đặt câu hỏi liên quan đến nội dung tài liệu.
3. Xem câu trả lời và nguồn tham chiếu do hệ thống trả về.